



Interactive Converged HPC: An Arkouda-Driven Approach for Exploratory Computing in Cloud Native Environments

Bachelor Thesis

submitted on May 7, 2024

Faculty for Business and Health

Degree program in Business Informatics

International Management for Business and Information Technology

by

JON ECKERTH

Practice supervisor :

⟨ Hewlett Packard Labs ⟩
⟨ Harumi Kuno, PhD ⟩
⟨ Principal Research Scientist ⟩

DHBW Stuttgart:

⟨ Dominic Viola ⟩

Signature of the practice supervisor:

Acknowledgements

Once again I want to thank Dr. Harumi Kuno, Principal Research Scientist at Hewlett Packard Labs, for her guidance and support throughout this project, without her this work would not have been possible. Having her as a mentor has been a great privilege and inspiration. I am grateful to once again have had the opportunity to work with her.

The same goes for my academic supervisor Dominic Viola who I want to thank for his invaluable insights and recommendations, which have been instrumental in shaping this project.

I extend this gratitude to the entire Hewlett Packard Labs and Hewlett Packard Enterprise for supporting me professionally and academically throughout my studies and making this bachelor thesis possible in the first place.

Abstract:

This thesis presents an exploration of the synergies between the worlds of High Performance Computing and Cloud Computing, aiming to devise an interactive exploratory computing environment within a cloud-native setting capable of handling HPC-scale data analysis tasks. The research addresses the challenge of merging the computational power and efficiency of HPC systems with the versatility and flexibility of Cloud Computing and the need for an easy-to-use and user-friendly interface; which enables real-time research in a familiar programming environment. For this purpose, an architecture, was designed from literature by way of a Conceptual Deductive Analysis, and the resulting system design was implemented in an iterative development process applying the gained insights to update the system design with the experimental knowledge. The artifact created in this way is a bare metal Kubernetes cluster, running a fleet of Arkouda worker instances, which can be interfaced with via a Jupyter Notebook with native a native integration providing a seamless experience for the user. The goal for this artifact being to extend the capabilities of the Pachykuda system designed in the previous work by the author, extending the GitOps based pipeline workflow orchestrator with the ability interactively explore and build data analysis pipelines within a Jupyter environment. This thesis contributes to the field by constructing an experimentally verified design for the integration of a computational notebook interfaces with Kubernetes and highly parallel compute frameworks and by providing a tangible artifact.

Contents

List of abbreviations	V
List of figures	VII
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Research Questions	2
1.4 Contributions	2
2 Methodology	3
2.1 Conceptual-Deductive Analysis	3
2.2 Prototyping	4
3 Conceptual Deductive Analysis	5
3.1 Cloud Computing	5
3.1.1 Virtualization and Isolation	6
3.1.2 Containerization	7
3.1.3 Container Runtimes	11
3.1.4 Orchestration	12
3.2 High Performance Computing	20
3.2.1 Introduction to High Performance Computing	20
3.2.2 HPC Infrastructure	21
3.2.3 HPC Software Stack	28
3.3 Exploratory Computing	33
3.4 Deductive Design of Conceptual Architecture	34
3.4.1 Goals	34
3.4.2 Implications	34
3.4.3 Suggested Architecture	35
4 Artifact Development	42
4.1 Previous Work	42
4.2 Existing Infrastructure	43
4.3 Implication for the Prototype	44
4.4 Iteration 1: Interactive Environment	45
4.4.1 Milestones and Goals	45
4.4.2 Implementation	45
4.4.3 Evaluation of Results	48
4.5 Iteration No 2: Performance Improvements	49
4.5.1 Milestones and Goals	49
4.5.2 Implementation	49
4.5.3 Investigating Arkouda Crash	53
4.5.4 Performance Testing	54
4.5.5 Evaluation of Results	57
4.5.6 Conclusion	58
4.6 Iteration No 3: Infrastructure and Interconnect	59

4.6.1	Milestones and Goals	59
4.6.2	Implementation	59
4.6.3	Evaluation of Results	61
4.6.4	Conclusion	65
4.7	Conclusion of the Prototyping Process	66
4.7.1	Summary of the Prototyping Process	66
4.7.2	Evaluating against the Deductively Designed System	67
5	Summary and Outlook	74
5.1	Summary	74
5.1.1	Contributions	77
5.2	Reflection	78
5.2.1	Methodology	78
5.2.2	Execution	78
5.2.3	Results	79
5.3	Outlook	79
5.3.1	Future Work	79
5.3.2	Conclusion	79
	List of terms	81
	List of literature	85

List of abbreviations

AI	Artificial Intelligence	iWARP	Internet Wide Area RDMA Protocol
API	Application Programming Interface	KVM	Kernel-based Virtual Machine
ARP	Address Resolution Protocol	LAN	Local Area Network
AUFS	Advanced Multi-Layered Unification File System	LXC	Linux Containers
CC	Cloud Computing	ML	Machine Learning
CDA	Conceptual-Deductive Analysis	MPI	Message Passing Interface
CNCF	Cloud Native Computing Foundation	MPP	Massively Parallel Processors
CNI	Container Network Interface	MVP	Minimum Viable Prototype
COW	Copy-On-Write	NIC	Network Interface Controller
CPU	Central Processing Unit	NUMA	Non-Uniform Memory Access
CRI-O	Container Runtime Interface - OCI	OCI	Open Container Initiative
CRI	Container Runtime Interface	OS	Operating System
DB	Database	PCI	Peripheral Component Interconnect
EC	Exploratory Computing	PDB	Pod Disruption Budget
FAM	Fabric Attached Memory	PF	Physical Function
FIFO	First In, First Out	PGAS	Partitioned Global Address Space
FOSS	Free and Open Source Software	PID	Process Identifier
FPGA	Field-Programmable Gate Array	QoL	Quality of Life
GASNet	Global Address Space Networking	QoS	Quality of Service
GPU	Graphics Processing Unit	RAD	Rapid Application Development
HPC	High Performance Computing	RBAC	Role-Based Access Control
I/O	Input/Output	RDMA	Remote Direct Memory Access
IB	InfiniBand	REPL	Read-Eval-Print Loop
ID	Identifier	RO	Read-Only
IP	Internet Protocol	RoCE	RDMA over Converged Ethernet
IPAM	IP Address Management	SIMD	Single Instruction, Multiple Data
IS	Information Systems	SLURM	Simple Linux Utility for Resource Management
IT	Information Technology	SMP	Symmetric Multi-Processing
		SPMD	Single Program, Multiple Data

SRIOV	Single Root I/O Virtualization	VENV	Virtual Environment
SSH	Secure Shell	VETH	Virtual Ethernet
TCP/IP	Transmission Control Protocol/Internet Protocol	VF	Virtual Function
UEFI	Unified Extensible Firmware Interface	VM	Virtual Machine
UMA	Uniform Memory Access	VXLAN	Virtual Extensible LAN
		WAN	Wide Area Network

List of Figures

1	Full Virtualization	6
2	Containerization	7
3	Namespace isolation ¹	9
4	High Level Architecture ²	14
5	Kubernetes Network Diagram ³	16
6	Symmetric Multi-Processing (SMP) Architecture	23
7	Three Tiered NUMA Architecture ⁴	23
8	Star Topology	27
9	Ring Topology	27
10	Connected Graph	27
11	Fat Tree Topology	28
12	Hypercube Topology	28
13	Fork-Join Model	30
14	High level architecture of Pachykouda ⁵	43
15	High Level Overview of the Jupyter Setup	47
16	Deployment time of a 2.86GB image ⁶	50
17	Deployment time of a 710 MB image ⁷	51
18	Argosort by Problem Size	55
19	Aggregated Speed by Data Type	56
20	Operation Data Transfer	57
21	Multus Container Network Interface (CNI) with SRIOV and Flannel ⁸	60
22	Simplified architecture of a SRIOV enabled Kubernetes Node	63

1 Introduction

This section clarifies the circumstances and the motivation for the following research, which problems are addressed, how we address them and what was achieved.

1.1 Motivation

The High Performance Computing (HPC) community and the Cloud Computing (CC) community have traditionally operated in separate worlds, with the HPC community focusing on raw performance and the CC community focusing on convenience, flexibility, and reliability. But with the rise of CC systems to run a significant portion of the computation workloads of the Internet, and a mature ecosystem that rivals the compute power of traditional HPC systems, both communities have realized the potential of combining the best of both worlds.

Especially in the field of Artificial Intelligence (AI) and Machine Learning (ML), where increasingly large systems are needed for training large models, innovative solutions are being created to run these workloads directly on CC systems. The recent acquisition of **Pachyderm** by **HPE** is an example of how one of the leading HPC vendors sees the potential benefits of CC systems which can be used to schedule and run Big Data workloads in a scalable and scheduled manner⁹.

Within the **Hewlett Packard Labs - Systems Architecture Lab** a research project was started in order to investigate the potential of bringing the flexibility and convenience of the CC ecosystem to the HPC world. While the **Pachykouda** project has successfully shown that the integration of the workflow management system **Pachyderm** and the highly parallel compute framework **Arkouda** is possible, further research was needed.

The existing solution is based on a GitOps approach, defining workflows, building containers and deploying them to the Kubernetes cluster, based on the changes in a Git repository. While this approach is highly reliable and ensures reproducibility of the results, it is limited by its static nature and a high latency between changes made and the deployment of the new workflow. This latency prevents the system from being used in an interactive manner, as the user has to wait for the deployment to finish before the results can be seen.

The goal of this research project is not to replace the existing solution, but to extend the system with a new interactive way of submitting jobs to the CC system, allowing the user to interact with the system in real-time and get immediate feedback on the results, without incurring the initial latency of the deployment, the build and deployment steps of the GitOps approach.

⁹cf. Nahren Khizeran 2024

1.2 Problem Statement

While low latency high performance computing has been a focus of the HPC as well as the CC community, resulting in systems such as **JupyterHub** or **Dask**, these systems are inherently ephemeral and do not provide a way to reproduce results on a HPC scale. Therefore, we need to solve the following problems:

- **P1:** *The GitOps first approach with introduces a latency between the changes made and the deployment of the new workflow. This latency prevents a real-time explorative workflow and limits the usability for the interactive exploration of the data.*
- **P2:** *Existing solutions for interactive workloads are focusing on the convenience and flexibility of CC systems, but do not provide a build in way to be combined with an automatic reproducible orchestration system for HPC scale workloads.*

1.3 Research Questions

Based on these needs and problems the following questions need to be answered:

- **RQ1:** *How can the system be extended to allow for a real-time explorative workflow, allowing for an interactive exploration of HPC scale data?*
- **RQ2:** *How can such a system be designed to combine the performance of HPC with the flexibility and convenience of CC systems?*

1.4 Contributions

By way of this research the following results were achieved:

- **C1:** *A deductive analysis of the three intersecting fields of HPC, CC and Exploratory Computing (EC)*
- **C2:** *An abstraction of these fields into a conceptual solution for the problem at hand*
- **C3:** *A practical implementation of the conceptual solution showing the feasibility of the approach, where the deductive analysis and the conceptual solution are used as a guideline and are validated by the practical implementation.*

2 Methodology

This section describes the chosen methodologies for the project and the rationale for their selection. The methodologies chosen for this project are the Conceptual-Deductive Analysis (CDA) and Rapid Application Development (RAD) model, which will be described in the following sections.

2.1 Conceptual-Deductive Analysis

The CDA is a methodology especially popular within the fields of Information Systems (IS) and Information Technology (IT) research, as it has proven itself to be especially useful as a design basis for the development of complex computer systems¹⁰.

For this reason it has been chosen as the main co-methodology of many prototype based projects.¹¹ Especially in the context of projects where the requirements and possibilities are not fully known at the beginning of the project, or the previous work in the field is rapidly changing, needing complete evaluation of the current state of the art.

It is based on the following steps:

1. Presentation of basic principles
2. Mapping of the problem domain
3. Evaluation of previous work and partial solutions
4. Design of a solution hypothesis
5. Implementation of the solution
6. Evaluation of the solution hypothesis compared to the implementation

For this project the CDA methodology will be augmented with the RAD model a popular prototyping methodology, which will be described in the next section. The solution hypothesis will be the basis for the requirements of the entire prototyping process. The final evaluation of the resulting prototype serves as the test of the solution hypothesis, making this not only a synergistic combination of methodologies, but also a complete scientific method¹².

¹⁰cf. Wilde/Hess w.y.

¹¹cf. Schreiner/Hess/Benlian w.y.

¹²cf. Wilde/Hess w.y.

2.2 Prototyping

As described in the section above, the combination of Prototyping and CDA has been chosen for their ability to handle complex and uncertain problem domains. While the CDA lays the groundwork of the research, the prototyping will be used to evaluate the solution hypothesis and to develop the final product and grounds the theoretical work in practical application.

Many prototyping methodologies have been developed over the last 40 years¹³ as they break down a large and complex problem into smaller achievable steps, which can be tackled individually¹⁴.

For this project the RAD model has been chosen, as it is signified both by its ability to handle complex and uncertain problem domains, but also the reduced overhead in comparison to other prototyping methodologies¹⁵. While other methodologies like the spiral model¹⁶ would provide a more thorough risk analysis, they would also be too time-consuming for a project of this scope, especially since the utilization of the CDA already provides a risk mitigation, by providing a solid theoretical basis for the project.

The RAD model is based on the following steps:

1. An overall goal is defined which this entire prototype incrementally should approach.
2. The project is divided into smaller iterations, which are planned and executed individually.
 - Specific goal for this iteration are defined
 - Implementation is documented and deviations from the plan are noted
 - The results of the implementation and the original plan are evaluated
3. The results of the previous iteration are used to plan the next iteration
4. A final evaluation of the original assumptions, the results of the iterations and the final prototype is done

But as this project aims to integrate the CDA and the RAD model, the RAD model will be augmenting the Prototyping methodology with the CDA as described in the previous section. The solution hypothesis of the CDA will then serve as the basis for the requirements of the entire prototyping process, and the evaluation of the resulting prototype in turn serves as primary test of the solution hypothesis.

¹³cf. Gomaa 1983, pp. 17–27

¹⁴cf. Naumann/Jenkins, A. M. 1982, pp. 29–44

¹⁵cf. Martin 1991

¹⁶cf. Boehm 1988, pp. 61–72

3 Conceptual Deductive Analysis

The structure of the CDA, as previously described in the Methodology section, is as follows: The relevant concepts of the three main paradigms to be integrated will be introduced first. While going from the general to the specific and attempting to give both an overview of the broader topic as well as a detailed understanding of the specific aspects that are relevant to our research.

Based on the knowledge gained and presented thereby, the hypothetical model will be developed of how these paradigms may be integrated and what implications this may have.

3.1 Cloud Computing

Outsourcing IT infrastructure and services to third parties, rather than maintaining them in-house, has gained popularity as a cost-effective and flexible solution for businesses. It allows for reduced upfront costs and increased scalability.¹⁷

It has become especially popular in form of multi tenant data centers, which are able to provide services to many customers. This ability to share the costs burden of the infrastructure amongst their customers, making use of the economies of scale has led to the rise of the three main cloud providers, Amazon Web Services, Microsoft Azure and Google Cloud Platform¹⁸.

These multi tenant data centers need to guarantee the isolation of hardware and software resources between the different customers, as highly sensitive, valuable, and mission critical data is stored and processed in these data centers.

While this need has existed before the advent of the cloud computing business model, as user isolation is a fundamental requirement of any multi-user system, the need for isolation has become more important since number of customers and the amount of data stored and processed has increased significantly. This has led to the continual development of virtualization and containerization technologies, which are used to provide the necessary isolation between the different customers¹⁹.

The two main technologies used to provide these isolated environments are virtualization and containerization. What they are and how they work will be discussed in the following sections.

¹⁷cf. Armbrust et al. w.y.

¹⁸**InfographicAmazonMaintains2024**

¹⁹cf. Randal 2021, pp. 1–31

3.1.1 Virtualization and Isolation

The differentiating factor of Virtual Machine (VM)s is that they are at their core hardware emulators. They consist of a host Operating System (OS), a so called hypervisor and one or more guest OS, including their dedicated virtual resources like Central Processing Unit (CPU), memory, storage, and network interfaces²⁰.

This does not only provide isolation between the processes running on the different guest OSs, but also enables the users and developers to emulate hardware architectures and run different OSs than the host OS

In figure 1 the different layers of a full virtualization setup are shown. The hardware layer is the actual hardware, which is used to run the OS and the hypervisor.

The Hypervisor is a software layer which is responsible for the management of the different VMs, while it may be a separate piece of software, it can also be tightly integrated into the OS itself and is then called a type 1 hypervisor, while a hypervisor running on top of an OS is called a type 2 hypervisor²¹.

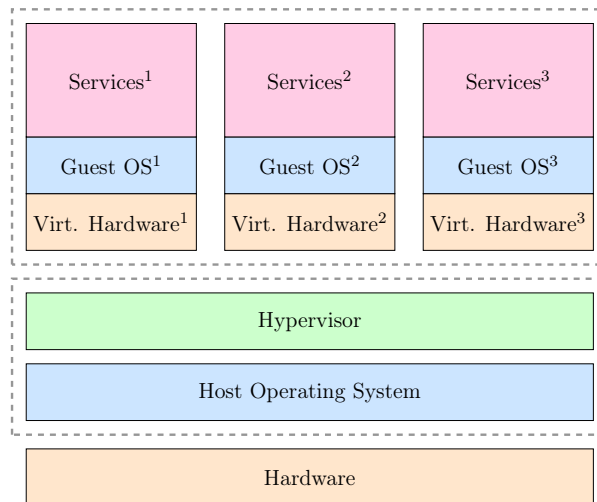


Fig. 1: Full Virtualization

While virtualization is a powerful tool, it is not without its drawbacks, as the overhead of the emulation of an entire hardware stack is a significant performance hit, which is why Kernel-based Virtual Machine (KVM) and Xen have been developed. They are able to pass through instructions directly to hosts hardware.

Features like VT-d, virtio and EPT are used to respectively pass through CPU instructions, network/storage interfaces and memory management to the hardware, if the hardware supports these features. But if the hardware does not support these features, the hypervisor will fall back

²⁰cf. Goto 2011, pp. 362–368

²¹cf. AWS 2024b

to emulate the needed hardware making it support a wider range of hardware, but at the cost of performance²².

3.1.2 Containerization

With the need for isolated environments and the economic desire for a more efficient use of resources, the next logical step was to create a solution which trades the flexibility of virtualization for the performance of executing CPU instructions native to the host hardware²³.

This gap was gradually filled by the development of containerization technologies in the Linux kernel, by way of giving the host OS the ability to “jail” processes and denying them to interact with other processes and the host OS. This enables the host OS to run multiple isolated environments, which are called containers, with close to native performance.²⁴

The downside of this approach is the strong dependency on the hosts kernel, which means that all the containers are bound to its architecture and may not use features of newer kernels, or legacy features of older kernels²⁵.

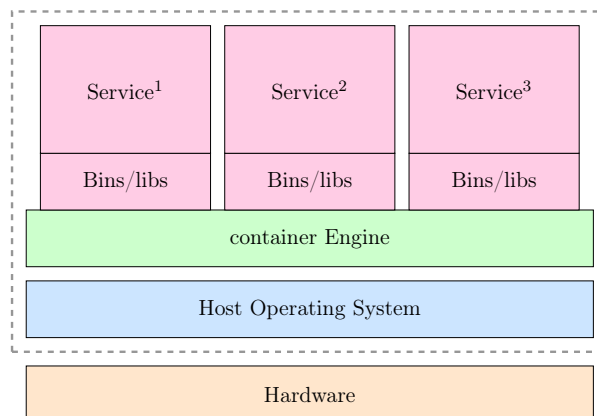


Fig. 2: Containerization

In figure 2, a containerization setup is depicted, which includes the hardware, the host OS, and the container engine. The container engine is responsible for managing the containers and the resources they use as well as providing a unified way to interact with the containers.

The emergence of containerization was more of a gradual process than a single release of a new technology, but the three most important steps in this process were the introduction of the chroot system call, the introduction of namespaces and the introduction of c-groups, which will be discussed in the following sections.

²²cf. Barham et al. 2003, pp. 164–177

²³cf. Erceg 2022

²⁴cf. Randal 2021, pp. 1–31

²⁵cf. Kovacs 2017, pp. 47–51

Chroot

The first major step towards process isolation was the introduction of the `chroot` system call in the 7th edition of Unix and later in the 4.0BSD release, which was further developed to the `chroot jail`. These jails were used as a security tool to protect the larger system in *BSD*, this sentiment was not applied when Linux adopted the `chroot` system call²⁶

While not technically being considered a security feature, as it was not designed to be inherently secure, and therefore is easy to circumvent²⁷, it does provide a way to isolate processes from the rest of the system. As changing the root directory of a process and its children to an arbitrary directory, results in them considering this directory as the root of the file system.

This prevents a given process effectively from reaching outside this boundary, as it is unable to access files outside the new root directory. Linux works under the paradigm "everything is a file", which means that everything on the system is represented within the file system, be it a process, a network interface or a block device. So if the process is unable to access the file, it is unable to access the resource. This is useful for running untested code which is not suspected to be actively malicious, or for providing a restricted environment for a process to run in.

While some consider *chroot* the first step towards containerization, some argue²⁸ that the true origin might be attributed to the "process capabilities" function, which allowed for the limitation of process resource utilization and permissions, this is a key feature of containerization, but as they fell out of favor after the advent of virtualization, we will not discuss them further.

Namespaces

The central feature of isolation in modern Linux containerization beyond the simple separation of the file system is the use of namespaces.

As the *chroot* command only provides a way to deny access sections of the file system, a process still needs some way to interact with the rest of the system, for example to access the network or to create new processes it would need access to the `/proc` file system, undercutting the isolation provided by `chroot`. Therefore, a way to provide a process with its own segregated view of the system is needed, which is where namespaces come in²⁹

Namespaces are a feature of the Linux kernel, which is used to provide isolation between the 6 different classes of kernel resources. Five of which are so-called privileged namespaces, which require root access to create and manage³⁰.

²⁶cf. Bernstein 2014, pp. 81–84

²⁷cf. Linux - manual page 2024

²⁸cf. Randal 2021, pp. 1–31

²⁹cf. Datadog 2023

³⁰cf. Sun et al. 2018, pp. 1423–1439

The five privileged namespaces are:

- **PID** — Process IDs
- **NET** — Network devices, stacks, ports, etc.
- **IPC** — System V IPC, POSIX message queues
- **MNT** — Mount points
- **UTS** — Hostname and NIS domain name

The sixth namespace, the **User** namespace, is special, because it can be used by unprivileged users to create containers, which is not possible with the other namespaces³¹.

Namespaces are applied to all Linux processes, regardless of whether they run in a container or not, every process has its own view of the system, which is demarcated by the ID of the namespace it is running in. Processes with the same namespace ID share the same view of the system, while processes' with different namespaces are totally ignorant of the other processes existence and influence on the system³².

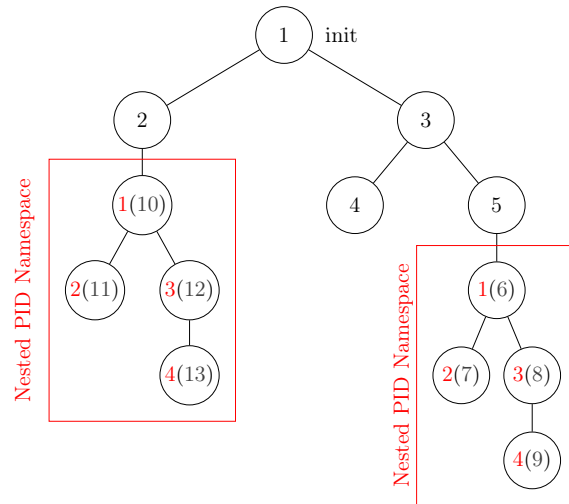


Fig. 3: Namespace isolation³³

In figure 3 a simple example of the Process Identifier (PID) namespace is shown, where the processes 1, 2, 3, 4 and 5 are running in the “init” namespace, which is the namespace the host OS is running in. The processes 6, 7, 8 and 9 are running in their own nested PID namespace, believing to be the only processes on the system. The same goes for the processes 10, 11, 12 and 13, which also believe to be processes 1, 2, 3 and 4 in their own namespaces. This results in a hierarchy of namespaces, where the parents are able to see the children, but not the other way around. So if a process in the “init” namespace were to kill process 1, it would identify it with

³¹cf. Friedhorsky/Randles 2017, pp. 1–10

³²cf. Michael Kerrisk 2013

³³Modified from Sultan/Ahmad/Dimitriou 2019

its true ID of 1 addressing it in the “init” namespace. But if it wanted to kill within the nested namespace, it can reference the process with ID 1 of the namespace by using the ID 6 as the internal PIDs of a namespace are mapped to a different PID in the parents namespace³⁴.

Control groups

While chroot isolates the file system and namespaces isolate system resources, a given service might think it is alone on the system and starts to take up all the resources it can get, filling memory, taking up all the CPU time and saturating the network. To prevent this from happening intentionally or unintentionally, control groups were introduced³⁵ for each of the namespaces, so that each process can be limited in all 6 dimensions.

The c-groups have two levels of enforcement for the limitations, on one hand there is the hard limit, which is an absolute limit, that if exceeded will result in the process being terminated immediately. On the other hand there is also the soft limit which lets the process exceed the limit for a certain amount of time. But if the system is starved for resources, the services which are overstepping the most get killed first³⁶.

Capabilities

While the namespaces limit what can be seen and interacted with, and the c-groups limit how much of the resources can be used, the *capabilities* make sure that the processes are not able to do anything they are not supposed to do.

This need comes from the binary way Linux handles permissions, as either root or not root. In the context of containers we need to differentiate between the ultimate power a process may exert within its own namespace, but the fine-grained control over what a process may modify in the host system.

An example for this would be to host an Apache server in a container, in order to expose the web server to the internet, the container needs to be able to bind to a port. Without *capabilities* feature the container would need to run as root, which would pose a security risk, as the container would be able to do anything the root user can do, which is not desirable.

This is where *capabilities* come in. They are a way to give a process a subset of the root users capabilities, which are needed to perform a certain task, without giving it the full power of the root user.

In the case of the Apache server, the process would need the capability to bind to a port, which is the `CAP_NET_BIND_SERVICE` capability.³⁷

³⁴cf. Michael Kerrisk 2013

³⁵cf. Docker 2015

³⁶cf. Linux Kernel - Docs 2024

³⁷cf. Sultan/Ahmad/Dimitriou 2019, pp. 52976–52996

3.1.3 Container Runtimes

All these major features work to enable the concept of containers, the coordination of these features is done via the container runtime which is responsible for the creation, management, and destruction of containers.

LXC

The first container runtime was Linux Containers (LXC), which was developed in 2008 by the Linux kernel community³⁸, it was the first to exclusively use the mainline kernel features to provide isolation and centered around the idea of a “system container”, a container pretending to be a full OS, which is able to run multiple services and applications³⁹.

Docker

Nowadays, one of the most popular container runtimes is Docker - a high-level container runtime⁴⁰ focusing on “application containers”, which are used to run a singular service per container. Docker extends the basis of LXC container runtime by a Advanced Multi-Layered Unification File System (AUFS) file system, as well as further kernel features providing further isolation⁴¹.

The whole Docker project aims to provide a simple and easy to use interface to the underlying containerization technologies, it administrates the containers through a unified Application Programming Interface (API), the docker daemon, to which and from where all the communication goes.

It also provides a standard way of predefining environments, by way of the docker image format. The docker consists of a set of immutable Read-Only (RO) layers, stacked onto of each other and then combined with a Copy-On-Write (COW) layer, these combine to the file system of the container runtime. While the RO layers are shared between different containers and instances, as resources. the COW layer is unique to each container and is used to store the changes made to the file system for this container⁴².

While Docker has become the de facto standard for containerization, it is not without its drawbacks, as the docker daemon mandates root access to run and operate. This has been considered as undesirable in the HPC community⁴³ and some consider it to be proving too large of an attack surface⁴⁴. Several alternatives have been developed, some as a drop-in replacement for the

³⁸cf. Jordan Webb 2024

³⁹cf. Kumaran S. 2017

⁴⁰cf. Vailshery 2024

⁴¹cf. Debab/Hidouci 2021, pp. 135–161

⁴²cf. Morabito/Kjällman/Komu 2015, pp. 386–393

⁴³cf. Mujkanovic et al. 2023, pp. 165–176

⁴⁴cf. Priedhorsky/Randles 2017, pp. 1–10

Docker runtime like Container Runtime Interface - OCI (CRI-O) and Podman, while others like Singularity and Charliecloud are designed to be used in the HPC community specifically.

3.1.4 Orchestration

For now, we have only talked about the creation and management of containers in the scope of a single host, but as the containerization effort was driven by Googles need to manage their data centers, It was always the intention to manage containers in a distributed environment⁴⁵.

Coordinating the execution, communication and scheduling of containers in a large scale environment is called orchestration, and has become a key feature for the cloud computing business model, as it allows for a more efficient use of resources than virtualization and a more secure and flexible way than traditional HPC batch systems⁴⁶ which will be discussed in the 3.2 section.

The three dominating orchestrations are Kubernetes, Docker Swarm and Mesos, with Kubernetes being the most popular by a wide margin, with 46% of companies reporting to use at least one Kubernetes based service while Swarm and Mesos are used by 11% and 9% of companies respectively⁴⁷. All these tools have a significant feature overlap, while differing in their internal approach⁴⁸. As Kubernetes is by far the most popular while having the most in common with the other two, we will be using it as a reference for the following discussion.

Kubernetes works as a distributed declarative system, that means that instead of being told what to do, it is told what should be, and the cluster does its best to make the desired state a reality.

Kubernetes Basic Concepts

At the heart of Kubernetes are the *pods*, they are the smallest deployable unit in Kubernetes and consist of one or more containers, which are always running together as a single unit.

As they share the same namespaces for network and storage, they are able to communicate with each other and share data. So even the components of Kubernetes itself are containers running as pods on the cluster. The core functionality of Kubernetes is to manage and schedule these pods on the different Nodes of the cluster and to provide them with the necessary resources to run.

The way to interface with the cluster is through providing a *manifest*, a description of a desired state of the cluster, usually in form of a **YAML!** file, which is then submitted to the *API server* of the cluster. These manifests specify in which way and under which conditions the pods should

⁴⁵cf. Verma et al. 2015, pp. 1–17

⁴⁶cf. Kenny/Knight 2021

⁴⁷cf. Garden 2024

⁴⁸cf. Truyen et al. 2019, p. 931

be running on the cluster, and the *controller manager* of the cluster is responsible for making sure that the desired state is a reality.

There are several types of controllers which are used, but the most important of which are the *Jobs*, the *Deployments*, *StatefulSets* and *DaemonSets*.

- **Jobs** are one off tasks which are run to completion, similar to a batch system job in a HPC environment.
- **Deployments** are used to declare a state of the cluster, for example that at anytime there should be 3 instances of a web server running, no matter what happens.
- **StatefulSets** as all the containers in the pods are ephemeral, they will not persist their state after they are terminated, for this reason a StatefulSet can be used to provide a stable network identity and stable storage for which an instance of a pod can be associated with.
- **DaemonSets** are pods which permanently run on the nodes of the cluster, they are used to provide additional interfaces between the clusters nodes and the containers running on them e.g. a network proxy or a monitoring agent.

High Level architecture

To ensure a controllable environment for the containers to run in and to communicate with each other, a homogeneous substrate is provided across the different nodes of the cluster, which is the responsibility of the *container runtime* and the *container network interface*. To keep all these nodes in sync and to provide a unified way to interact with the cluster, the *control plane* which is itself a containers on the cluster. The high level architecture of Kubernetes is divided into two main parts, the control plane and the nodes, The control plane is responsible for the coordination of the different nodes and the management of the different resources. Itself is divided into several microservice like components, which are responsible for the different management tasks. Just like everything else the services of the containers themselves are containers running on the cluster⁴⁹. The cluster may consist of a single node or thousands of nodes, which are responsible for the actual execution of the containers⁵⁰.

Control Plane

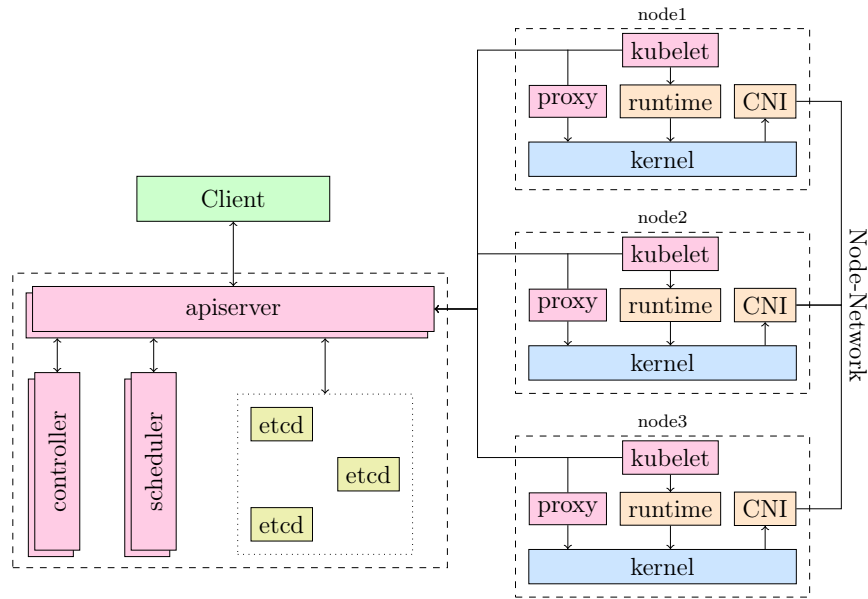
A state is declared to the system via the unified API `kube-apiserver`, this preferred state is then given to the `etcd`, a distributed key-value store, which is used as the Single source of truth for the cluster⁵².

⁴⁹cf. Kubernetes - Docs 2024h

⁵⁰cf. Kubernetes - Docs 2024j

⁵¹Modified from Zempashi 2024

⁵²cf. Nalawala et al. 2022, pp. 481–489

Fig. 4: High Level Architecture⁵¹

On a regular basis, the **kube-controller-manager** checks the actual state of the cluster against the desired state, if the actual state deviates from the desired state⁵³ the **kube-controller-manager** will wake the corresponding controller for the resource in question, which knows what to do to make the actual state match the desired state⁵⁴

This woken controller then submits a request to the **kube-scheduler** via the API, which then assigns the task to a node, based on the current scheduling policy⁵⁵.

Nodes

On the Side of the clusters nodes, the **kubelet** is responsible for interfacing with the local container runtime through the Container Runtime Interface (CRI) and handles the execution of the containers locally on behest of the larger cluster. Once the kubelet has announced itself to the **kube-apiserver** the node is ready to run containers. If the scheduler has assigned a task to the node, the **kubelet** will check if it needs to pull the container image from the registry, and then start the container⁵⁶.

The other service running on all nodes is the CNI, which is responsible for the networking of the containers. The CNI is a plugin-based system, which allows for the use of different underlying

⁵³cf. Kubernetes - Docs 2024c

⁵⁴cf. Kubernetes - Docs 2024e

⁵⁵cf. Kubernetes - Docs 2024i

⁵⁶cf. Kubernetes - Docs 2024g

networking solutions, in order to provide the container with a layer 3 network interfaces and a Internet Protocol (IP) address⁵⁷.

The `kube-proxy` is responsible for regularly checking in with the `kube-apiserver` and updating the local IP tables, to make sure that the containers are able to communicate with each other and the outside world⁵⁸.

The containers themselves are run within a `Pod`, which is the smallest unit of work in Kubernetes. While a `Pod` can consist of as little as a singular container, the idea is to provide a way to allocate a main container, in combination with additional supporting containers, which are used to provide additional services to the main container. An example for this would be a main container running a service which needs to be able to access a database.

The connection to the Database (DB) instead of hard-coding the service simply calls `localhost`, and the support container being a sidecar container, which is working as an easily replaceable proxy to the database.

All this while a third container captures the entire `stdout` and `stderr` of the main container and sends it to a log aggregator.

Networking

As already mentioned the networking of Kubernetes is quite involved in order to provide the flexibility, isolation, and performance needed for a large scale distributed system. This is because the containers need to be replaceable on a whim, accessible both from the outside and the inside of the cluster, but should also all have a unique IP address without the need for NAT. This is achieved by abstracting the networking into three different levels, the pod level, the node level and the cluster level.

Pod level At the pod level, the containers share the same network namespace and therefore also the same IP address. The communication between the different containers happens through the loop-back interface, so that the containers are able to communicate with each by simply calling on their `localhost` interface. This is made possible through the `pause` container, a container in every pod which handles the whole administration of basic tasks, like setup of the namespaces, network interfaces or the killing of zombies in the PID namespace⁶⁰.

Node level

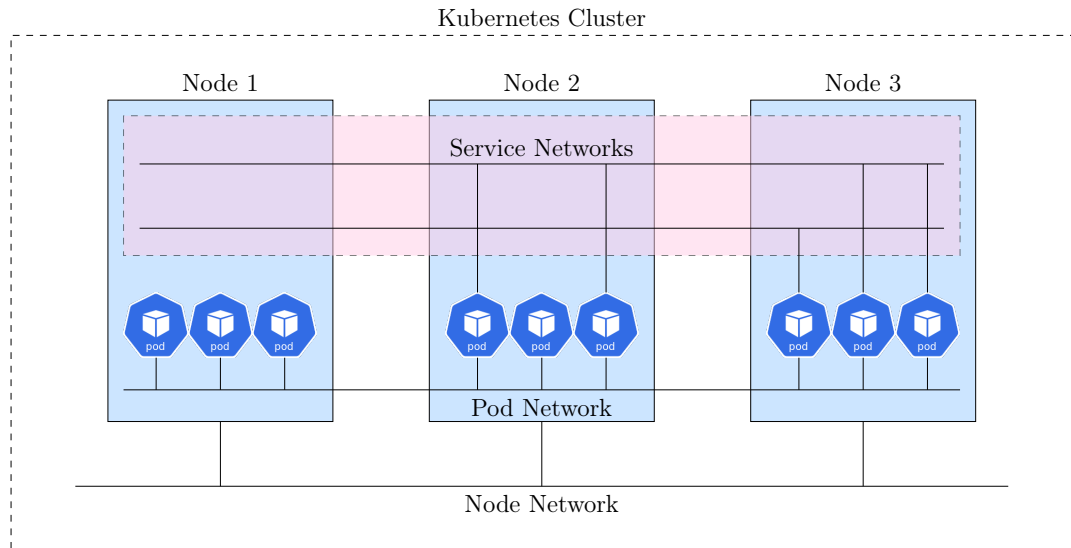
Each pod has its own network namespace and its own dedicated IP address, which is unique to the pod and is reachable from the entire cluster. If two pods need to communicate with each other, each of them patching their network interface to the root network namespace via a `Virtual`

⁵⁷cf. The Linux Foundation 2024

⁵⁸cf. Kubernetes - Docs 2024f

⁵⁹Based on: Kubernetes - Docs 2024d

⁶⁰cf. Sookocheff 2018

Fig. 5: Kubernetes Network Diagram⁵⁹

Ethernet (VETH) pair. Now the root network namespace can see the two VETH endpoints as two network interfaces, it can bridge the two interfaces, which allows the two pods to communicate with each other⁶¹.

Cluster level

If the pod needs to communicate with another pod on a different node, the bridging will fail as the Address Resolution Protocol (ARP) will not be able to reach an Ethernet interface on the other node. If this happens the bridge will forward the packet to the default gateway, which is the real outwards facing network interface of the node connected to the rest of the network. The routing of the packets is then handled by the CNI plugin, which is responsible for the networking of the clusters nodes. The CNIs job is to provide the containers with a layer 3 network interfaces and an IP address, which is unique to the pod and is reachable from the entire cluster⁶². How they do this is up to the CNI plugin itself, as network infrastructure and IP allocation can be very different depending on the underlying infrastructure. While CNI plugins like **flannel** or **calico** simply provide flat layer 3 networks, with external routing. Others like **DataDog's** custom solutions enable direct pod to pod routing without any layer or bridge in between⁶³.

Services

A special case in this situation is, if a website, database or any other external interface is supposed to be able to be accessible under a certain IP address permanently. As the containers are ephemeral so are their IP addresses and if the underlying container is replaced the IP address will change. To be able to expose a services running in pods which can be replaced at any time, Kubernetes provides the concept of the **Service**.

⁶¹cf. Sookocheff 2018

⁶²cf. Laurent Bernaille 2018

⁶³cf. Laurent Bernaille 2018

A **Service** is a stable IP address, which is reachable from the entire cluster, and is used to expose a set of pods to the outside world. It can be used to expose a single pod, a set of pods or even a set of pods running on different nodes and balance the load between them. Even when the IP address of the pods changes, the **Service** will still be reachable under the same IP address⁶⁴.

The solution to this problem is to use **Service**, which works as both a proxy and a load balancer. The **Services** have a whole network of their own, neither in the “real” network the nodes are connected to, nor in the pod network. Unlike the pod network, services don’t even have a virtual network interface associated with their IP address. This would prevent them from being routable if it weren’t for the kube-proxy. Utilizing the IP tables of the nodes, as well as the Netfilter of the Linux kernel, the kube-proxy is able to reroute packets mid-flight from the service IP to the current IP address of a pod⁶⁵. The **kube-proxies** keeps in sync with the global state of the cluster by regularly checking in with the **kube-apiserver** and updating their local routing rules accordingly.

Scheduling

As mentioned above, when Google started to work on the Kubernetes predecessor, the Borg system, the initial goal was to provide a way to have both their high availability services and their non production jobs, including large scale batch processing jobs running on the same infrastructure⁶⁶.

Therefore, a balance needed was between having a low latency and free resources for the high availability services, to be able to react to the changing demand of the users, and to be able to run the batch processing jobs which were not as time or latency sensitive as the high availability services. So not only does the scheduler need to optimize when to start which pods, but should also take into account the different priorities of the jobs as well as complex constraints like physical and logical distances of interconnected services, the availability of the required hardware, or the need to anticipate the future demand of the users⁶⁷. Therefore, the scheduler does not only start the pods, but is also able to evict, reschedule and scale the pods according to the current demand of the cluster.

Google’s solution to this problem is a highly customizable and flexible scheduling system, which bases its decisions in two steps.

- **Predicates** The predicates are a set of knockout criteria which ideally disqualify a node from being able to run a pod (no capacity, missing important hardware ...)
- **Priorities** The priorities are a set of weighted criteria which are used to rank the remaining nodes, to find the best fit for the pod.

⁶⁴cf. Velayudhan 2024

⁶⁵cf. Betz 2022

⁶⁶cf. Abhishek Verma et al. 2015

⁶⁷cf. Verma et al. 2015, pp. 1–17

Based on these two steps, the state of the cluster and the specific requests of the pod, the scheduler is able to make a decision whether and where to run the pod⁶⁸. After the decision has been made, the scheduler updates the `etcd` through the `kube-apiserver`, with the new state of the cluster. Then the `kubelet` on the node is expected to poll the `kube-apiserver` and to start the pod, once it has become aware of the new state of the cluster⁶⁹.

Native Kubernetes provide a myriad of different options to influence the schedulers' behavior⁷⁰.

- **nodeSelector** is a simple binary label matching system. If the node has the required label, the pod can be scheduled on it. This mechanism allows for straightforward restrictions on pod placement based on node labels.
- **Node Affinity** offers a more nuanced approach compared to `nodeSelector`, allowing specifications of node preferences rather than strict requirements. Pods can still be scheduled even if preferred labels are not found on any node, which makes this a more flexible option than `nodeSelector`⁷¹.
- **Inter-pod Affinity/Anti-Affinity** considers the current pod topology for scheduling decisions, aiming to place pods in proximity to or at a distance from each other based on specified criteria. This is beneficial for optimizing network traffic or ensuring high availability by distributing instances of a service across different nodes.
- **Taints and Toleration** work together to ensure that pods are not scheduled onto inappropriate nodes. Taints are applied to nodes and can repel pods that do not have a matching toleration. This is useful if certain nodes have special hardware or are reserved for certain tasks.

The Scheduler is not only for starting a pod, but also evicts pods which need to be rescheduled or are not needed anymore. There are two main reasons why a pod might be evicted, the first is that there is something more important which needs to be run on the node. This is the so-called *preemption*, which is the act of evicting a lower priority pod to make room for a higher priority pod. The priority is simply determined by a numerical value associated to a pod, which is then used to rank the pods on the node⁷².

The second reason is the so-called *eviction*, which is generally caused by the node running out of resources, this is called a *node pressure*. Kubernetes, by way of the kubelets, listens to the resource utilization of the nodes. If a critical threshold is reached, it will start to evict. The kubelet will make a decision based on the Quality of Service (QoS) class of the pod as well as its Pod Disruption Budget (PDB).

⁶⁸cf. Zivuku 2023

⁶⁹cf. Kubernetes - Docs 2024g

⁷⁰cf. Rejiba/Chamanara 2023, pp. 1–37

⁷¹cf. Kubernetes - Docs 2024b

⁷²cf. Martínez 2022

The QoS class is a simple way to categorize the pods based on their resource requirements, they are a manifestation of the soft and hard resource limits of the cgroups already mentioned in section Control groups. Kubernetes differentiates between the following three classes in order of priority⁷³:

1. **Guaranteed** pods have both a hard and a soft limit, and are guaranteed to have the resources they need.
2. **Burstable** pods have a hard limit, which is higher than the soft limit, and are allowed to burst up to the hard limit.
3. **BestEffort** pods have no limits and are allowed to use all the resources of the node, which are not reserved for the system.

If the resource needs can not be easily estimated, but a hard requirement of a certain number of pods still needs to be met the PDB can be used. The PDB is a way to specify the minimum number of pods of a certain type which need to be running on the cluster. If a node is running out of resources, the kubelet will check if the eviction of a pod would cause the number of pods of a certain type to fall below the specified number, and refuse the eviction for that pod⁷⁴.

While Kubernetes offers quite a powerful and flexible scheduling system, the user is not limited to use only this system. Especially for the HPC community, the Kubernetes scheduler is not always the best fit, as its design is geared towards the balance between high availability services and resource utilization, the former of which is not always the main concern in HPC environments⁷⁵.

Next to the direct modification of the scheduler source code, Kubernetes offers 3 different ways to introduce custom scheduling logic to the cluster⁷⁶. For one there is always the possibility to write a custom scheduler, which is the most flexible way to introduce custom scheduling logic, and has been done as by the **Kube-batch** scheduler, which is specifically designed to take into account the needs of the HPC community⁷⁷. Alternatively Kubernetes also offers less invasive ways to introduce custom scheduling logic, like the **Extender API** or the **Scheduling Framework**, which allow for the extension of the default scheduler, without the need to write a custom scheduler, while providing extensive customization options⁷⁸.

⁷³cf. Matias Nicoletti 2023

⁷⁴cf. Matias Nicoletti 2023

⁷⁵cf. Misale et al. 2022, pp. 310–326

⁷⁶cf. Rejiba/Chamanara 2023, pp. 1–37

⁷⁷cf. Kubernetes - Docs 2024a

⁷⁸cf. Kubernetes - Docs 2024l

3.2 High Performance Computing

High Performance Computing has been an integral part of scientific research and engineering for decades. While it is being used synonymously with supercomputing⁷⁹, the term HPC is used to describe the use of supercomputers and parallel processing techniques for solving complex computational problems that respond to scale, meaning that the application of more resources will result in a more valuable product.

HPC systems are designed to go beyond the capabilities a single system can provide, and therefore utilizes clusters of multiple computers that work together to solve a problem instead. These systems have enabled scientific breakthroughs, like the research on the human genome⁸⁰, the discovery of the Higgs boson⁸¹ or the first image of a black hole⁸². That said, HPC systems are by no means limited to purely scientific applications, as they are also an essential tool in various industries, such as finance, oil and gas, automotive, and many more which require complex simulations, data analysis, and modeling⁸³.

Supercomputers, which have been around for many decades, have historically focused on how to scale performance so as to produce more valuable results by applying more computing resources. For this, HPC systems sacrifice ease-of-use in favor of performance, employing bare metal systems and highly specialized homogeneous hardware. CC systems, however, have only recently begun to take on HPC-scale workloads. CC systems are tailored to maximize ease-of-use, and have historically sacrificed performance for the sake of offering increased flexibility and reliability. As AI workloads have increased in size, the task of managing HPC clusters has become more expensive and complex. With cluster sizes increasing to thousands of nodes, the two communities have been moving towards each other. That is to say, HPC customers have become increasingly interested in ease-of-use and CC customers have become increasingly interested in performance-at-scale.

3.2.1 Introduction to High Performance Computing

At its core, HPC revolves about the concepts of distribution and parallelization – the spreading of highly complex computational tasks across multiple systems to solve problems that are too large or too complex for a single computer to handle. Therefore, its main issue is the splitting of a problem into smaller tasks that can be solved in parallel, as well as the coordination of running separate nodes as to work together in a coordinated manner.

HPC differentiates between two types of compute problems:

⁷⁹cf. Sterling/Brodowicz/Anderson 2017

⁸⁰cf. Schadt et al. 2010, pp. 647–657

⁸¹cf. Belyaev et al. 2017

⁸²cf. HLRS High Performance Computing Center Stuttgart 2024

⁸³cf. Fortune business insights 2024, p. 170

First are the so-called *embarrassingly parallel* problems, which are sometimes also referred to as *pleasingly parallel* problems. Such problems have gotten their name by the ease of parallelization, as they can be split into independent tasks that can be solved in parallel. Classical examples include the Ray tracing of images, where each pixel can be calculated independently, or Monte Carlo simulations⁸⁴.

In contrast to that is the true challenge of HPC – *strongly coupled* problems, where the different tasks are dependent on each other and therefore need to be solved in a coordinated manner. Classic examples of this include weather simulations, **Finite Element Analysis** or **n-body** simulations⁸⁵. As each element of the simulation is highly dependent on all or at least a subset of the other elements, a high need for inter-node-communication arises.

Therefore the two most important parts of a HPC system are the compute power of a given node on the one hand and the interconnect on the other. This can be seen in how these systems are ranked in practice as the authority on HPC performance, the *Top500* list ranks the fastest supercomputers by their **LINPACK** performance, which measures the speed of solving a dense system of linear equations⁸⁶. As the **LINPACK** benchmark is a highly parallelize problem, the ranking is highly biased towards the raw compute power of the system, as the communication need between the nodes is negligible. In order to supplement this, the **Graph500** list was created, which ranks the systems by their ability to solve the traversal of a graph⁸⁷. This provides an insight into the performance of a system when solving a problem that is not just a sum of the individual compute power of the nodes, but is also highly dependent on the interconnects and the communication between the nodes.

3.2.2 HPC Infrastructure

The core component of a HPC system are the compute nodes, which are the individual systems that perform the calculations. These nodes may be equipped with multiple processors, including CPUs, Graphics Processing Unit (GPU)s, or FPGAs, and are connected to each other via a high-speed network. While homogeneous systems only utilizing a single type of processor across all nodes are still common, a trend towards GPU coprocessor systems can be observed⁸⁸.

In the past, the larger architecture of HPC systems used to be much more diverse, employing architectures such as constellations, Single Instruction, Multiple Data (SIMD), or even single processor systems⁸⁹, but more recently Massively Parallel Processors (MPP) and Cluster systems have become dominant.

⁸⁴cf. Gandy w.y.

⁸⁵cf. Capuzzo-Dolcetta/Spera/Punzo 2013, pp. 580–593

⁸⁶cf. Dongarra, J. J./Luszczek/Petit 2003, pp. 803–820

⁸⁷cf. Murphy et al. 2010, p. 22

⁸⁸cf. Kindratenko/Trancoso 2011, pp. 92–95

⁸⁹cf. TOP500 2024a

While clusters make up 80% of all *Top500* systems, when scaled for the total compute power per architecture, MPP systems make up 44% of the total compute power of the *Top500*⁹⁰.

There are significant differences between MPP and Cluster systems with regard to how they are connected and how they communicate. Clusters are based on a shared-nothing architecture, where each node has its own memory and communicates via messages over a network connection. In contrast, MPP systems are based on a shared-memory architecture, where all nodes share the same memory and can communicate via this shared memory⁹¹. However the size of cores per node has increased, these distinctions have become less pronounced. This is because these multi-core systems also need to communicate internally, which may also be done via shared memory communication, while communication between the nodes is still done via a network connection, creating a hybrid architecture⁹².

Memory Architecture

In a HPC system the structure of the memory is one of the most distinguishing features of the system, as it has a significant impact on the performance of the system, how it needs to be programmed and what kind of workloads it is best suited for.

A significant difference between HPC systems is how they manage the coordination between the different compute cores. The two main architectures are either a shared memory based approach, generically called Symmetric Multi-Processing (SMP) which combines its parts into a single large operating system, enabling the processors to access private, shared or common memory. The other approach is to have multiple separate systems, keeping their memory exclusively to themselves, coordinating and sharing data through the passing of messages over the underlying network or interconnect⁹³.

UMA

Its most basic implementation is a single node with multiple cores, where all cores have access to the same flat memory, through a shared bus, as can be seen in Figure 6. This is what is known as a Uniform Memory Access (UMA) system, where all memory is equally accessible to all cores. Being able to access the same memory is a significant advantage of SMP systems, it is exceptionally well suited towards tightly coupled problems, where the different cores can directly work on the same data, without the need to communicate over a network connection. In the early days of HPC this worked especially well, as CPUs were much slower than the memory itself.

⁹⁰cf. TOP500 2024b

⁹¹cf. Dongarra, Jack et al. 2005

⁹²cf. Li/Hoefler/Snir w.y.

⁹³cf. ARCHER UK National Supercomputing Service 2016

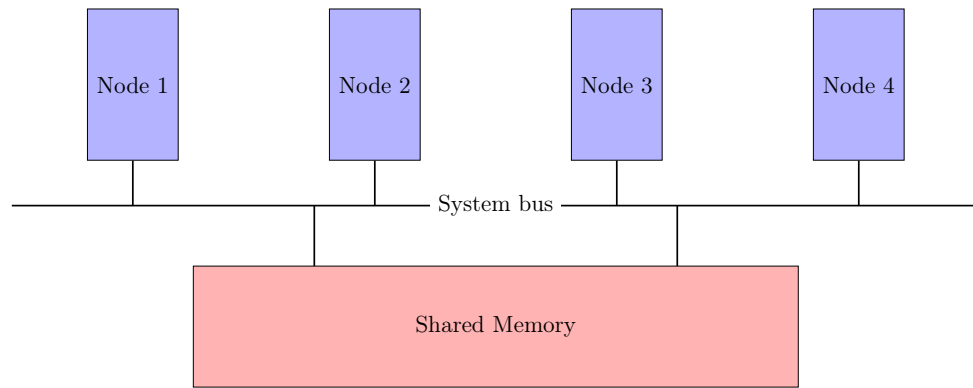
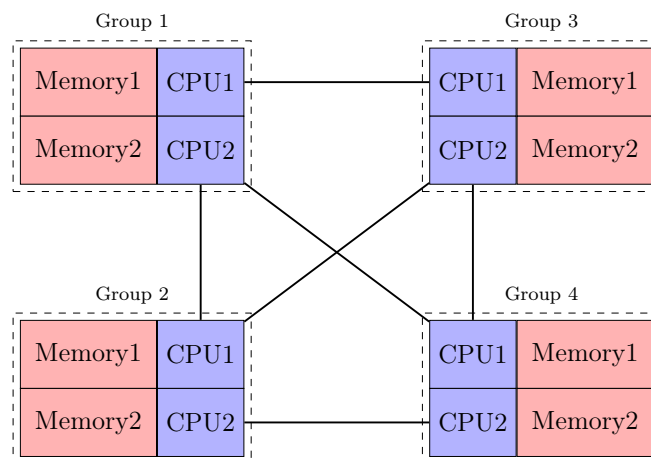


Fig. 6: Symmetric Multi-Processing (SMP) Architecture

CPU speed improvements outpacing memory technology led to the flat memory not being able to keep up with the Input/Output (I/O) demand of the CPUs; this problem has come to be commonly known as the "von Neumann bottleneck".

NUMA

To alleviate this issue of the "von Neumann bottleneck" new, more distributed memory architectures have been developed. The central idea is to create multiple smaller zones of collaboration – namely, instead of having all cores having equal access to a single large chunk of memory, allocating smaller chunks of memory to a few cores each which they can access quickly, while still allowing them to access the memory of all other cores. This prevents the issue of the "von Neumann bottleneck" by reducing the number of cores that need to access the same memory while at the same time, but still allowing all cores to access the memory of all other cores. This is what is called Non-Uniform Memory Access (NUMA) architecture, as can be seen in Figure 7.

Fig. 7: Three Tiered NUMA Architecture⁹⁴

⁹⁴Diagram adapted from Ploskas/Samaras 2016

The diagram in 7 shows a three-tiered NUMA system, two processors having access to their own memory, while also being able to access the memory of the other processor.

1. The CPUs own cache which is the fastest to access.
2. The cache of the CPUs neighbor cores on the same group.
3. The memory of the other groups.

While this lets the SMP systems scale much larger than the UMA systems, it also has to deal with the issue of keeping these distributed memory systems in sync, as well as making sure to distribute this memory in a way that is optimal for the workload.

Cache Coherence

The problem of keeping the memory in sync is called *cache coherence*, which is the process of making sure that every core has the same view of the memory⁹⁵. If an operation on the memory is performed all other cores need to be informed about this change. To do this efficiently, is a significant challenge, and many protocols have been developed to address this issue⁹⁶.

Memory Placement

A second issue is how to distribute memory across different nodes in such a way that the overall access time is minimized. As memory is spread across many nodes, which are of different accessibility, it becomes especially important where the memory is initially placed. and how it is spread across the nodes. The default behavior of *OpenMP* is to place the whole data on the first node that accesses it, which is called *first touch* policy. This is done to reduce the overhead of the communication, as the data is already on the node that needs it. But even in the very baseline case, where more than one core needs to access the same data, the data will be entirely placed on the first node that accesses it, needing to manually instructed to distribute the ownership of the data across the nodes⁹⁷.

Distributed Memory

While memory sharing systems have come a long way in scaling and performance and have a strong foothold in multi-core systems, they are still limited in their scalability and flexibility, as they are limited to a single system both physically and by operating system⁹⁸. The solution to this are distributed memory systems which take the opposite approach to the shared memory systems; instead of having the entire memory available to all cores all working on the same problem, the memory is instead private to each core. Therefore instead of having a single large memory, be it uniform or not the memory is split into separate chunks. This is especially useful for problems that are not tightly coupled, as the cores can work on their own data without the need to communicate with the other cores. The way the cores communicate is by passing

⁹⁵cf. Prof. Dr. Ben H. Juurlink 2018

⁹⁶cf. Hennessy/Patterson 2011

⁹⁷cf. Ruud Van Der Pas w.y.

⁹⁸cf. HPC Wiki 2024a

messages over the network, which is much slower than the shared memory communication, but also much more scalable, how this is done specifically is discussed in the section 3.2.3. A welcome side effect of this is its compatibility with commodity hardware, as the nodes do not need the physical ability to support shared memory, enabling the use of off the shelf hardware, which was a significant factor in the rise of HPC clusters and CC systems⁹⁹.

Hybrid Systems

Similar to how the NUMA systems are a more distributed version of the UMA systems having smaller zones of fast access, modern HPC systems take on a hybrid approach. The current trend in HPC is to combine the best of both worlds, utilizing many commodity multi-core NUMA nodes, combined via a high-speed network, to form a distributed memory system¹⁰⁰ While most of these systems are either based on extensions of the Message Passing Interface (MPI) standard in order to support local shared memory communication, alternative approaches have been developed to support this hybrid approach on a more fundamental level¹⁰¹.

One such approach is the Partitioned Global Address Space (PGAS) model, which try to utilize the best of both worlds by letting the programmer control the memory placement, while still having the ability to communicate between the nodes. The way this is done is create the perception of a single large memory, while in reality the memory is distributed across the nodes, and in order to describe the topology of the address space, as it is not uniform in accessibility, the memory is partitioned into segments¹⁰². The distance or the cost of using a remote memory access is then modeled by the specific implementing library, which either optimizes the placement and access of the memory implicitly, or lets the developer explicitly control the placement of the memory¹⁰³. One of the distinguishing factors of PGAS is the one sided communication, as foreign memory can be directly accessed via Remote Direct Memory Access (RDMA) operations without needing to involve the other core¹⁰⁴.

Networking and Interconnects

As the communication between the nodes is one of the most significant bottlenecks in HPC systems, the interconnects are a crucial part of the system. The two significant factors when it comes to interconnects is the kind of network fabric used as well as the topology of the network itself¹⁰⁵ On the chip level the communication is done via the process interconnects which communicate between the cores and memory, while on the cluster level the communication is done via a network fabric, interconnecting the nodes through a high speed network connection¹⁰⁶.

⁹⁹cf. Dongarra, J. et al. 2005, pp. 51–59

¹⁰⁰cf. Rabenseifner/Hager/Jost 2009, pp. 427–436

¹⁰¹cf. Jin et al. 2011, pp. 562–575

¹⁰²cf. De Wael et al. 2015, pp. 1–27

¹⁰³cf. Almasi 2011, pp. 1539–1545

¹⁰⁴cf. Dewan/Jenkins, L. 2020, pp. 659–666

¹⁰⁵cf. Besta et al. 2021a, pp. 943–959

¹⁰⁶cf. Dongarra, J. et al. 2005, pp. 51–59

When talking about the network fabric, the most common types of network fabrics used in HPC are Ethernet and InfiniBand, each having roughly 40% of the market share, with no significant difference even when weighed by performance¹⁰⁷.

Ethernet has been the default network connection for most devices for decades, resulting in a high availability of hardware and software support, while being highly flexible and relatively cheap. But as it was designed for general purpose Local Area Network (LAN) and Wide Area Network (WAN) communication, it, in its standard form not originally designed for high speed communication between nodes¹⁰⁸. The largest issue with baseline Ethernet is that it does not natively support RDMA operations which is a very specific requirement for HPC systems, as it enables the direct access of memory between the nodes, without the need to involve the CPU¹⁰⁹. To solve this issue the Internet Wide Area RDMA Protocol (iWARP) protocol was developed, enabling to use the entire Ethernet stack for the infrastructure, while letting special RDMA capable Network Interface Controller (NIC)s handle the communication¹¹⁰. But iWARP, being dependent on the Transmission Control Protocol/Internet Protocol (TCP/IP) stack is not as fast as the native RDMA capable InfiniBand, which is a specialized network fabric designed for high speed communication between nodes. In order to close this gap between the two, the RDMA over Converged Ethernet (RoCE) protocol was developed, providing better performance and lower latency than iWARP, while still being able to use the existing Ethernet infrastructure while being on par with InfiniBand¹¹¹.

In contrast to Ethernet, InfiniBand was designed from the ground up for high speed communication between nodes in a cluster. It is a very low latency, high bandwidth network fabric, which native supports RDMA operations, but requires specialized hardware and software support as well as being a proprietary standard, which can lead to higher costs and less flexibility¹¹².

The choice of network fabric is highly dependent on the specific use case, as Ethernet is more flexible and cheaper, while InfiniBand is faster and more specialized. But as can be seen in the HPC Top500 list, both are relatively equally represented in the top systems, showing that both are viable options¹¹³.

Network Topology

No matter which network fabric is used, the topology of the network is also a significant factor in the performance of the system, as the topology becomes more expensive and complex as the number of nodes increases. The topology of the network is the way the nodes are connected to each other, essentially being the structure of the whole supercomputer.

¹⁰⁷cf. TOP500 2024c

¹⁰⁸cf. Howard 2023

¹⁰⁹cf. RDMA Consortium 2003

¹¹⁰cf. Intel 2010

¹¹¹cf. Howard 2023

¹¹²cf. Pfister w.y.(a)

¹¹³cf. TOP500 2024c

Below are three very simplified examples of network topologies which each demonstrate the central problems of network topologies, between which the system architect has to balance: One needs to keep in mind, that while these only have 8 nodes each the actual systems can have thousands of nodes which need to be connected. The three most important factors when it comes to network topologies are the diameter, the bisection bandwidth as well as the cost of the network¹¹⁴.

First, the *Star Topology* in figure 8 shows that all nodes are connected to a central switch. This configuration ensures that each node can directly communicate with every other node with a single hop, resulting in a low diameter connection. But as the entire communication has to go through the central switch, the entire communication has to go through the central switch, the bisection bandwidth is ultimately limited by the capacity of this central hub. This presents a scalability issue; as the number of nodes increases, the central switch becomes a potential bottleneck, impacting the overall system performance. Moreover, the cost and complexity associated with upgrading or maintaining such a central switch can be prohibitively high for larger networks.

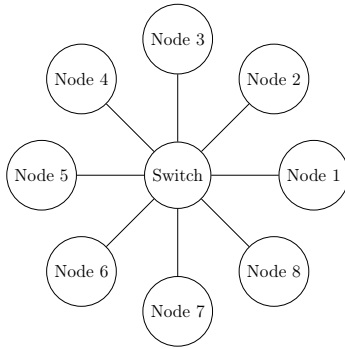


Fig. 8: Star Topology

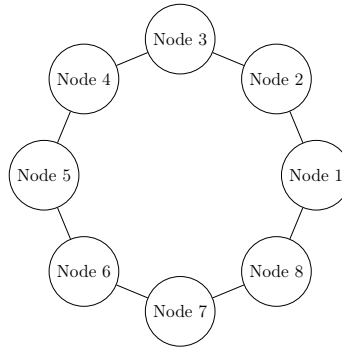


Fig. 9: Ring Topology

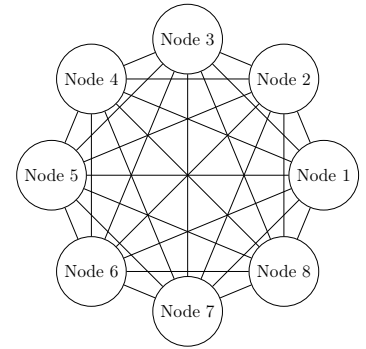


Fig. 10: Connected Graph

Next, the *Ring Topology* depicted in figure 9 is examined, where each node is connected to two other nodes, forming a loop. This topology allows for a straightforward path of communication, with data traveling around the ring to reach its destination. The diameter of this topology grows linearly with the number of nodes, making it less ideal as the network size increases. On the plus side, the ring topology is relatively inexpensive to implement and scale, as adding more nodes simply involves creating two additional connections for each new node. However, the bisection bandwidth of a ring topology is inherently limited, cutting the ring at any point to form two halves only requires severing two connections, which could easily become a bottleneck in data-intensive applications.

Finally, the *Fully Connected Network*, as shown in figure 10, offers the most direct communication paths, with each node directly connected to every other node. This topology boasts the lowest possible diameter of 1, as any two nodes can communicate directly without intermediary nodes. The bisection bandwidth in this topology is maximized, given the sheer number of connections

¹¹⁴cf. Besta et al. 2021b, pp. 943–959

that would need to be severed to bisect the network. However, the cost and complexity associated with a fully connected network are its most significant drawbacks. The number of required connections grows quadratically with the addition of nodes, making this topology impractical for large-scale applications due to the prohibitive cost of implementing and maintaining such an extensive network.

In practice, most HPC systems utilize a combination of these topologies, such as the Fat Tree Topology (11) or Hypercube Topology(12), which strike a balance between performance, scalability, and cost¹¹⁵.

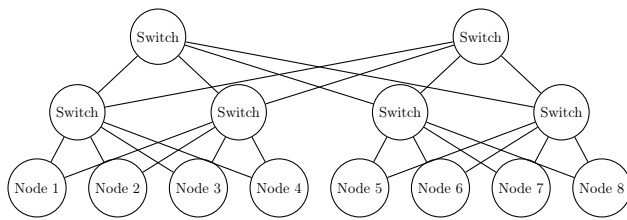


Fig. 11: Fat Tree Topology

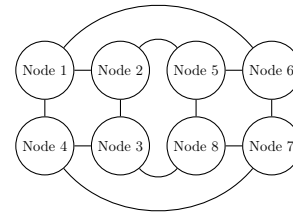


Fig. 12: Hypercube Topology

3.2.3 HPC Software Stack

Now that we have discussed the hardware components and their configurations, we will dive into the software stack that powers HPC systems. While this is the only part we can directly interact with as developers, many of the decision made in the hardware stack have a direct impact on the software we will be working with on the cluster.

Operating Systems

While the operating system is the most fundamental part of modern computers, in the context of HPC systems there exists no competition to the Linux operating system, which has had a 100% market share in the HPC Top500 list since 2017¹¹⁶. Before its rise to dominance, Linux's competitors were its predecessors – other, proprietary, UNIX-based operating systems that were the default operating systems for most HPC systems. Linux has come to dominate the HPC community for a number of reasons that include a desire to avoid vendor lock-in and licensing costs, as well as an appreciation for the customizability and flexibility of Linux, as its Kernel, Software and Libraries are all open source, which enables the community to optimize the system for their specific use case¹¹⁷.

¹¹⁵cf. Besta et al. 2021b, pp. 943–959

¹¹⁶cf. TOP500 2024d

¹¹⁷cf. Rohit Gupta 2024

Programming Models

As already mentioned above, the way the hardware works in HPC systems is very closely tied into the way the software needs to be written. This is especially true for this section, as the way the memory is distributed across the nodes and how the cores communicate with each other, needs to be taken into account when writing the software, or at least assumed by the framework that is used.

OpenMP

OpenMP is a shared memory parallel programming model, it is the software counterpart of the SMP hardware architecture, and works both with UMA and NUMA systems. It is a set of compiler directives, library routines, and environment variables that enable the parallelization of code on shared memory systems¹¹⁸. So it is designed to work on a single node of a given cluster, but able to utilize and combine the power of the cores and memory of the whole node, but cannot go beyond the boundaries of the node and communicate with other nodes¹¹⁹. Compared to other parallel programming models, OpenMP presents a relatively high level of abstraction, being able to parallelize code with a few lines of code. Its main advantage is its simplicity. OpenMP is based on compiler directives, which are added to the code and interpreted by the compiler to generate the parallelized code. This makes it very easy to use, as the programmer does not need to worry about the specifics of the hardware as the compiler will take care of it¹²⁰. While this is a great advantage for the programmer and lowers the barrier of entry for subject-matter experts of a specific field, it also has its drawbacks. As the compiler makes relatively uninformed decisions about the parallelization, the performance of the parallelized code can be suboptimal¹²¹.

It implements parallelization using a multithreaded fork-join model, where a single thread starts the program and then forks additional threads to run the parallelized code as can be seen in figure 13 below. So once a parallel region is defined by the programmer the process will fork the specified number of threads, which will run concurrently until the end of the region, where they join back together to continue the sequential execution of the code¹²². While these threads are running concurrently, they have access to a shared memory space containing the previous linear state of the program, but also have their own private memory space, which is used to store the variables that are only relevant to the specific thread¹²³.

For example, a loop that would normally be executed sequentially can be parallelized with OpenMP by adding the `#pragma omp parallel for` directive in front of the loop. This will tell the compiler to parallelize the loop, and the compiler will then generate the code to run the loop in parallel¹²⁴. On runtime instead of going through each iteration of the loop one by one, all the

¹¹⁸cf. Hermanns w.y.

¹¹⁹cf. MIT Plasma Science and Fusion Center 2022

¹²⁰cf. Niklas Wittmer 2018

¹²¹cf. Ruud van der Pas 2014

¹²²cf. Laboratory L. L. N. 2024

¹²³cf. MIT Plasma Science and Fusion Center 2022

¹²⁴cf. MIT Plasma Science and Fusion Center 2022

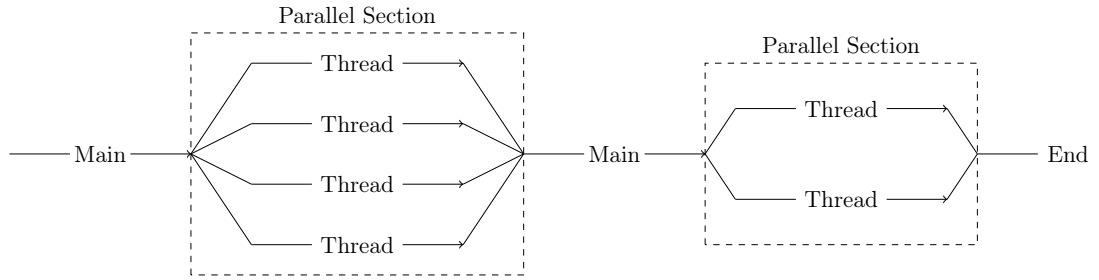


Fig. 13: Fork-Join Model

iterations will be divided between the threads, which will then run the iterations concurrently. The program then waits for all threads to finish before continuing.

Message Passing

If OpenMP is the standard for shared memory systems, then the MPI is its equivalent for distributed memory systems. It is a standardized and portable message-passing system designed to work on a wide variety of computers, and is language independent, supporting C, C++, and Fortran¹²⁵. In practical terms, MPI serves as an API that enables the inter-process communication across nodes and the network, allowing the programmer to send and receive messages between the nodes, and to synchronize the processes¹²⁶.

While some implementations enable the use of shared memory within a node, which have become a valuable tool in hybrid systems, MPI itself is primarily designed around the actively sending and receiving messages during runtime instead of accessing the memory directly.

MPI is based on the Single Program, Multiple Data (SPMD) model, where each process runs the same program, but with different data, and can communicate with each other via messages. This is done via the so-called *rank* of the process, which is a unique Identifier (ID) assigned to each process, serving as an address for the process, used to send and receive the messages, without needing to concern itself with the specifics of the underlying ISO/OSI model¹²⁷.

Messages in MPI are sent and received either via point-to-point communication where the *rank* of the sender and receiver are specified, or via a collective communication, where information is sent or received from all or a subset of the processes¹²⁸. These collective operations can be split into two dimensions:

1. **Data flow direction:** As a process can either send or request the reception of a message, the flow of communication can be either towards the initiating process, so a process might send a subtask to other processes (e.g. *BROADCAST*) or request the results of a subtasks in order to continue (e.g. *GATHER*)¹²⁹.

¹²⁵cf. Gropp/Lusk/Skjellum 1999

¹²⁶cf. Vishu Vaishnav 2023

¹²⁷cf. Open MPI 2014

¹²⁸cf. Eickhoff w.y.

¹²⁹cf. Dartmouth College 2024

2. **Data distribution:** The data a process sends/receives can either be a single value being replicated across all processes (e.g. *BROADCAST*) or a number of values being spread across the processes (eg. *SCATTER*)¹³⁰.

Chapel/GASNet

If OpenMP is the standard for shared memory systems, and MPI is the standard for distributed memory systems, then Chapel is the standard for PGAS systems. Chapel is a high-level parallel programming language designed for productive parallel computing across a wide range of scales, from multi-core desktops to the supercomputers¹³¹. Being based on the Global Address Space Networking (GASNet), Chapel is able to work on both shared, distributed memory systems and hybrid systems, or even emulating multiple nodes on a SMP system for development and testing¹³². Making use of RDMA in order to present a part of the memory as a large combined memory space, while in reality the memory is distributed across the nodes. Being able to directly accessing the memory across the network, avoiding the overhead of the CPU and the OS in order to send and receive messages, also has the advantage of being inherently asynchronous, as the memory can be directly accessed without needing to wait for the other process to be ready¹³³.

While some MPI implementations such as *MVAPICH* or *OpenMPI* have started to implement RDMA capabilities, and GASNet also supports network stacks which are not directly RDMA capable, each of these implementations was written with a specific use case in mind are oriented towards a specific use case. An example for this is GASNets ability to access memory directly is not just limited to the CPU memory, but can also directly connect to auxiliary compute devices such as GPUs in order to offload the computations to the corresponding coprocessor¹³⁴.

Workload Management Systems

With an understanding of how the hardware works on the nodes, how the cores can communicate with each other, even across the network, and how these networks can be structured, the next step is to discuss how when and where the code is being executed on the cluster. This is where the workload management systems come into play, also known as schedulers or Batch systems, as most of these systems main feature is the scheduler and primarily rely on a batch based distribution of jobs¹³⁵.

The workload manager distributed jobs across the compute cluster. Working with a list of computation tasks which are submitted by the users along with the amount of resources they need, the workload manager is responsible for the distribution of the jobs across the cluster. In order to optimize the utility of the cluster, the workload manager reduces the idle time of

¹³⁰cf. Dartmouth College 2024

¹³¹cf. chapel-lang.org 2024b

¹³²cf. chapel-lang.org 2024c

¹³³cf. Dewan/Jenkins, L. 2020, pp. 659–666

¹³⁴cf. Chamberlain w.y.

¹³⁵cf. Hovestadt et al. 2003, pp. 1–20

the nodes, by fitting in as many jobs of different users as possible, trading off between quick execution and fairness¹³⁶.

One of the most common workload management systems is Simple Linux Utility for Resource Management (SLURM), which is an open-source workload manager targeted towards HPC systems¹³⁷. From the perspective of the user, SLURM is a command line tool, which is used to submit jobs to the cluster and from the perspective of the node it is a simple daemon not to different from a remote shell¹³⁸. But under the hood it is a highly complex system, checking the availability of the resources, managing permissions and most importantly deciding the order of the jobs to be executed. In its most basic form SLURM is a First In, First Out (FIFO) queue, where the jobs are executed in the order they are submitted, which would lead to a low utilization of the cluster. Therefore, it does not strictly adhere to the FIFO principle, but slots in a lower priority job if it is able to run in a slot between regularly scheduled ones, as long as it does not delay any job of higher priority¹³⁹.

Additionally, SLURM can be provided with information about the topology of the network, supporting both torus and fat tree topologies, which can be used to optimize the placement of the jobs on the cluster, by using an n-dimensional space filling curve in the former and a tree based algorithm in the latter in order to get the jobs as close to each other as possible¹⁴⁰.

¹³⁶cf. HPC Wiki 2024b

¹³⁷cf. Yoo/Jette/Grondona 2003, pp. 44–60

¹³⁸cf. SCHEDMD 2024a

¹³⁹cf. SCHEDMD 2024b

¹⁴⁰cf. SCHEDMD 2024c

3.3 Exploratory Computing

With the advent of Big Data, the increasing complexity and increase of easily available compute resources, the field of data analysis has been under an ever-increasing pressure to "make sense" of the abundant data. This need has given rise to the methodology of Exploratory Computing (EC), which is an iterative process of data analysis which maintains a close to real-time dialogue between the analyst and the data¹⁴¹.

Unlike conventional data analysis methods which might follow a straighter path towards an answer to a specific question, EC aims to be a more open-ended process, where the eventual goal is not just to find a numerical answer to a question but rather to gain a deeper understanding of the dataset. Therefore, the tools in this field require a high degree of interactivity in order to become a more natural extension of the analysts thought process¹⁴².

Computational Notebooks

The defacto standard for scientific EC is the *Computational Notebook*, allowing for the direct integration of narrative text, code, and it's output like visualizations, tables and other data structures. While in some domains other notebooks systems like *Wolfram Mathematica* and *Matlab* are used, the most common notebook especially in the field of data science is the *Jupyter Notebook*¹⁴³.

Jupyter Notebooks is a Free and Open Source Software (FOSS) server-(web)client application that exposes an interactive editor. User can write code, documentation and visualizations into so-called cells. These cells are then executed on the server, which can be local or a remote compute resource, and the output is displayed in the notebook. As the computational environment is preserved across cells, with each cell being able to access the variables and functions defined in previous cells. This lends itself to the Read-Eval-Print Loop (REPL) style of programming, without the need to re-run the whole script¹⁴⁴. One of the key features of *Jupyter Notebooks* is the ability to run code in multiple languages, as the server on the backend is build on a modular architecture that allows for the integration of *kernels*. These kernels are responsible for executing the code in the cells and returning the output to the notebook, but are only limited by the API spec of the notebook server¹⁴⁵. This has lead to a wide variety of kernels being available, with the most common ones being for *Python*, *R* and *Julia*, but also supporting languages which are not traditionally associated with data analysis like *Postscript*, *PowerShell* and *Brainfuck*¹⁴⁶.

¹⁴¹cf. Di Blas et al. 2014, pp. 577–580

¹⁴²cf. Granger/Perez 2021, pp. 7–14

¹⁴³cf. Perkel 2018, pp. 145–146

¹⁴⁴cf. Jupyter - Docs 2024c

¹⁴⁵cf. Jupyter - Docs 2024a

¹⁴⁶cf. Jupyter - Docs 2024b

3.4 Deductive Design of Conceptual Architecture

Based on the deep knowledge gained by the previous sections, it is now possible to create a combined conceptual approach to address the issues described in the Introduction and the Problem Statement sections. This approach will be used as a guideline for the prototype implementation in the chapter Artifact Development.

In contrast to the sections above which went from the basic principles towards the higher levels of abstraction, this chapter will start at the high level goals, their implications and what will be needed on the lower levels in order to accomplish these goals.

3.4.1 Goals

Based on the Problem Statement, the goals of the system can be summarized as follows:

The designed system should provide the user with an easy-to-use interface, capable of interactively exploring HPC scale data. The system should be able to run on cloud infrastructure, be able to allocate the necessary resources, as well as scheduling and executing the computation within a reasonable time frame.

3.4.2 Implications

In order to achieve the goals, the system must be able to manage the following aspects, in order of abstraction:

- **Programming Language Support:** in order to enable HPC scale computation for the user, the programming language must support parallel computation natively, or at least provide a library which does. As programming languages such as Python are not natively able to support parallel computation paradigms such as described in the Programming Models section. The language must not just support parallel computation, but also be user-friendly in order to let the user focus on exploring the data and not spend their time on the intricacies of parallel computation.
- **Notebook Environment:** The system should support a Notebook environment, allowing users to facilitate the communication between the client and the compute cluster, requiring an endpoint being able to submit the code to the cluster and receive the results.
- **Compute Cluster:** The system must be able to schedule and execute the computation within a reasonable time frame. Requiring a system capable of managing the resources and scheduling the computation.
- **Platform Agnostic:** As the system is supposed to run on cloud infrastructure, whether it is on-premise or in the cloud, the system must be able to run cloud natively.

3.4.3 Suggested Architecture

Based on the requirements outlined above a hypothetical software can be suggested to fulfill the requirements. While the implementation of this in the following chapter will additionally constrain by the available resources and time frame here we can outline a system which is not limited by these circumstances.

Service Layer Architecture

This section will cover the highest level of abstraction, the service layer architecture, which will be the most important part of the system for the user.

Client

With the usage of a Compute Notebook being an essential part of exploratory data analysis, and as discussed in the Exploratory Computing section, Jupyter Notebook is the most popular choice and will bring a familiar interface to scientists and domain experts, who had little experience with HPC before.

In this case we will make use of the Jupyter Server Client architecture, having the user only run the client locally while the server will be running on the cloud infrastructure.

Language

The language of choice is *Arkouda*, a highly parallel framework for python, specifically build for enabling data scientists to perform parallel computation on terrascale HPC systems on a high level of abstraction and interactive speeds¹⁴⁷.

It is build on top of the *Chapel* programming language, which itself is a parallel programming language specifically designed on HPC systems, and instead of being a set of libraries on top of a general purpose language it is build from the ground up to work on PGAS systems, which is a perfect fit for the use case of this system as will be discussed later on in the Hardware Level Architecture section.

Another option could be MPI, as it is almost as user-friendly as Python, but as Python is a more popular language and is more user-friendly, it would be a better fit for the system.

Cluster

While the development of containers was primarily driven by CC needs and those two subjects how become tightly linked, CC is much more than just containers. Complete CC systems such as *OpenStack* don't limit themselves to just containers, but also provide virtual machines as well as bare metal provisioning to provide compute resources¹⁴⁸.

¹⁴⁷cf. Enthougt 2020

¹⁴⁸cf. Jackson/Bunch 2013

For the reasons discussed in the Cloud Computing section, virtualization is not a good fit, as in the best case we would still emulate a full operating system, which would be a huge overhead.

Bare metal, which is the practice of simply foregoing any kind of virtualization or containerization and just running the code on the hardware itself, at the cost of not being able to subdivide the resources of a node¹⁴⁹. This is done by creating a custom operating system image which is installed via a network boot. This means that provisioning a new node is extremely slow in comparison to starting an isolated container, but also ensures 100% of the resources are available to the user and no overhead is introduced by the virtualization layer. The specific use case this could fit is if the entire nodes compute resources would be taken up by a single service anyhow but most importantly the resource demand is very stable and predictable.

While the former is true and would make a compelling case for the compute-nodes, but especially in the context of exploratory computation the resource demands are highly dependent on the user and can change rapidly, which would make the latter a bad fit for the system.

The best fit for the use case is a container based system, as spawning containers is as fast as starting a new process, while bringing isolation, resource subdivision and a rich ecosystem of tools and libraries to the table.

This cluster will be hosting the entire system, from the compute services doing our HPC computation, to the exposed services such as the Jupyter Server, which will be able to communicate with the client.

Jupyter Server

The Jupyter Server web application will be running on the cluster being the developers access point to the system. It will be able to communicate with the client, submit the code to the compute cluster and request the necessary resources.

While there is also a multi-user Jupyter Server called JupyterHub, which allows multiple users to share the same server, we will still be using the single user instance¹⁵⁰. If more than one user wants to use the system at a given time, instead of sharing the same environment they each will get their very own complete environment, which will be spawned on demand and destroyed after the user is done with it. This will ensure not just a clean environment but also prevent any kind of interference between the users.

At least one Jupyter Server will always be running on the cluster, ready to accept new connections from the client.

Compute Services

Now that the user interface and its functionality have been discussed, the next step is to examine the compute services. These services are responsible for executing the code submitted by the

¹⁴⁹cf. Canonical 2024b

¹⁵⁰cf. Jupyterhub - Docs 2024

user on the compute nodes. As we have already discussed, the language of choice is Arkouda. It is specifically designed for enabling interactive programming and parallel computation. Arkouda splits the computation from the developer's machine to the compute nodes, allowing for efficient and scalable processing. Its infrastructure is split into three parts¹⁵¹:

- **Client:** This can be any python interpreter running on the users machine, which will be able to communicate with the server.
- **Server:** This is the bootstrap server, it is the entry point for the client and bootstraps the multi-locale cluster.
- **Locales:** These are the workers, at least one of them will be running on each node that contributes to the computation.

For the system to be build, both the server and an arbitrary amount of locales will be running on the cluster, with the server establishing the initial connection to the locales through Secure Shell (SSH), setting up the communication in order to set up the PGAS space.

The server will be on standby until a connection request is made by a client and will then submit the processing request to the locales through the interconnect.

Resource Provisioning / Scaling

While in traditional HPC systems the resources are statically allocated and the user has to request the resources beforehand in order to be scheduled, the system we are building is supposed to be more dynamic and able to scale the resources on demand. Similarly to the time google initially developed the Borg system in order to handle both their services as well as their batch jobs, as described in the Orchestration section, it is to be expected that this system might simultaneously handle other services as well as interactive computation, in order to make the best use of the hardware invest.

While an ideal solution would be able to scale the available resources for each single submission of a code cell, the way the Arkouda system is designed, the size of available nodes needs to be known beforehand. This is due to the fact that it relies on the PGAS model (see Programming Models) which requires the memory to be shared between the nodes, if we now started to add or remove nodes during the computation the memory space would need to be resized each time which would come with a large overhead. A possible solution to this problem could be the use of a system such as Fabric Attached Memory (FAM) which is able to provide an external memory space which is able to survive program termination and is made as a shared memory space between the nodes¹⁵².

An alternative approach could be to work on the scale of a session instead of a single code cell, where the user would be provided with a fixed amount of resources for the entire session,

¹⁵¹cf. Arkouda - docs 2024b

¹⁵²cf. Keeton/Singhal/Raymond 2019, pp. 70–89

depending on the utilization of the system, the user could be provided with more or less resources. This would be a more stable approach, as the user would not be interrupted by the system scaling up or down, but would also not be able to make use of the full potential of the system.

Depending on circumstances, another factor in this case could be Kubernetes ability to over provision resources. Over provisioning in this case means that while one node is technically bound up in a session, it could still be used by a different session, while the cores are sitting idle. In classical HPC systems this would not be possible as the compute request has been scheduled long before, but as exploitive computing is inherently burst, sitting idle while the user is thinking or writing code, these nodes could be used by other users. But this system is inherently a trade-off between the user experience and the utilization of the system and depends entirely on the specific use case.

Cluster Level Architecture

This section will cover the lower level of the system architecture, the sections which are not directly visible to the user but are essential for the system to work.

Operating System

Using containerization as the main method of resources isolation makes the choice of Linux as a base an imperative. As discussed in the section Containerization containers are build around features specifically created in the Linux kernel. While other operating systems do technically support some form of containerization, either through creating a Linux virtual machine or creating a custom containerization system, these are not as mature as the Linux based solutions¹⁵³. Aside from specific feature support regarding containers, Linux has become the de facto standard for HPC systems (3.2.3) as well as cloud infrastructure¹⁵⁴, due to its open source nature and the ability to customize it to the specific needs of the industry.

While the choice of operating system is clear, the choice of which distribution is much less so. Essentially there are four kinds of distributions which could be used, while they will have little functional difference for the system build on top of them, as containerization abstracts the underlying system away, they will have a large impact on the maintainability of the system, the availability of support and the ease of development of the system itself.

- **Cloud Managed Distributions:** The large hyperscalers such as AWS¹⁵⁵, Google Cloud¹⁵⁶ or Azure¹⁵⁷ all provide their own distributions within their cloud environments, which are specifically tailored to their needs. These are specifically geared towards the cloud environment and are not meant to be used outside it.

¹⁵³cf. McGrath/Brenner 2017, pp. 405–410

¹⁵⁴cf. Steven Vaughan-Nichols 2024

¹⁵⁵cf. AWS 2024a

¹⁵⁶cf. Google 2024

¹⁵⁷cf. Burt 2024

- **Enterprise Distributions:** These are distributions which are specifically tailored to the needs of the enterprise customers, running their own data centers. They provide a fully featured out-of-the-box solution, with professional support and a large ecosystem of tools. Examples of this are *Red Hat OpenShift*¹⁵⁸ or *VM Ware Tanzu Kubernetes Grid*¹⁵⁹. These systems are costly but provide a high level of support and are specifically tailored towards a specific way of administration.
- **Specific FOSS Distributions:** These are distributions which are build around the idea of being open source and free to use and are extendable by the user. Examples of this are *Suses Rancher*¹⁶⁰ or the community edition of *OpenShift* called *okd*¹⁶¹. While some offer a professional support option, they can be used and extended by the user as they see fit.
- **Generic FOSS Distributions:** These are distributions which are not specifically tailored towards any specific use case, but as the features of kubernetes are build around the kernel which is shared by all distributions, they can be used as well. But will require a manual installation of the necessary software and will not have any kind of professional support. This includes distributions such as *Debian*¹⁶² or *Ubuntu*¹⁶³.

As the system we will be building is going to be highly specific and will be running software which is out of the ordinary, from a cloud infrastructure perspective, the utility of outside support might be limited and the flexibility of the system might be more important. Therefore, the choice between a generic FOSS distribution or a specific FOSS distribution depends on factors such as the need for ready-made integrations and GUIs, as well as the administrators' familiarity with different systems. In this case, a Linux-based operating system would be the most suitable choice.

Container Runtime

The choice of container runtime is a crucial one, as the container runtime is the software doing the "containerization" – the isolation of the resources, the bootstrapping of the container and the execution of the code as described in the Containerization section. The Cloud Native Computing Foundation (CNCF) recognizes 19 different container runtimes¹⁶⁴, the most relevant in our case are *containerd*, *CRI-O* and *singularity*. *containerd* is the runtime used by docker, which has a high market share and is well known, due to it being the one of the first container runtime to be widely adopted both of the others are improvements or specializations of it.

CRI-O is a lightweight container runtime specifically designed for Kubernetes and is build around the Open Container Initiative (OCI) standard, being more streamlined towards large scale CC

¹⁵⁸cf. RedHat 2024b

¹⁵⁹cf. VMware 2020

¹⁶⁰cf. Rancher 2024

¹⁶¹cf. Innes 2024

¹⁶²cf. debian 2024

¹⁶³cf. Canonical 2024a

¹⁶⁴cf. CNCF [Cloud Native Computing Foundation] 2024

systems¹⁶⁵ but for the application of this system would work mostly interchangeably with *containerd*.

Singularity on the other hand is a container runtime specifically designed for HPC systems¹⁶⁶, which are trying to run as close to the metal as possible, and to support the specific needs of HPC systems such as direct access to external accelerators such as GPUs and Field-Programmable Gate Array (FPGA)s, support for HPC specific interconnects¹⁶⁷ (see 3.2.2) and the ability to run in rootless mode¹⁶⁸.

Container Image

The container image will be an extension of the base image which can be found in the Arkouda repo¹⁶⁹, which itself is based upon the chapel container image¹⁷⁰. It makes chapel and Arkouda available to be used in a containerized environment.

Network Abstraction

The network abstraction layer creates a homogenous network between the different containers even if they are running on different nodes. This makes the code running in the containers much easier to scale, manage and write. This can become quite complex involving multiple layers of abstraction and nested protocols such as described in the Networking section.

But while these systems make the network much easier to manage, they are a trade-off between usability and performance, as the layers of abstraction such as Virtual Extensible LAN (VXLAN) encapsulation or IP in IP tunneling come with a performance overhead. While this overhead is negligible for most use cases, in HPC systems where the network is the bottleneck it can be a significant factor.

While the most common network overlays in Kubernetes such as *Flannel* and *Calico*, support both VXLAN and IP in IP tunneling¹⁷¹, as this is needed to communicate across subnets in a cloud environment, but for performance reasons they also support foregoing a virtual network and using the physical network directly¹⁷².

But as Arkouda and many other HPC systems are built around the RDMA as an available feature, we also need an available network which supports RDMA as well. This can be done by creating an additional pod level network dedicated to the RDMA communication between the nodes, using *Multus*¹⁷³ and then the specific network plugin for the RDMA capable network card¹⁷⁴.

¹⁶⁵cf. crio 2024

¹⁶⁶cf. Singularity - Docs 2024c

¹⁶⁷cf. Uni.lu 2024

¹⁶⁸cf. Singularity - Docs 2024a

¹⁶⁹cf. Arkouda - repo 2024b

¹⁷⁰cf. Chapel 2024a

¹⁷¹cf. Tigera 2024

¹⁷²cf. Flannel - Repo 2024

¹⁷³cf. Multus 2024b

¹⁷⁴cf. NVIDIA 2024

Hardware Level Architecture

Compute Nodes Hardware

Regarding the hardware of the nodes making up the cluster, many rules which are true for HPC systems described in the HPC Infrastructure section are also true for this system. For this system, it is important to consider factors such as the need for low latency and high bandwidth in order to facilitate efficient communication in HPC systems. Additionally, the computational power should be as high as reasonably possible to handle large-scale computations effectively.

In order to handle smaller scale computations which fit into a single node, a High core count CPU would be a good fit as well as a large amount of memory, which will also come in handy for the larger scale computations, as part of the PGAS.

One advantage of Kubernetes is the ability to use more heterogeneous hardware than in traditional HPC systems. This allows for the use of nodes of different sizes and capabilities, optimizing the system for the specific use case while still using the same software stack and administration tools.

Overall this part of the system is mostly governed by the budget available, as more powerful hardware will be able to handle larger computations faster. And if the use case requires specific hardware, such as GPUs or FPGAs, the system should be able to accommodate these as well.

Network Interconnect

For the underlying interconnect, it is crucial to use a network that supports RDMA. However, in cloud environments, the most common network interconnects are simple Ethernet networks, which do not have built-in support for RDMA. While Arkouda is able to use PGAS which can use any kind of network, by wrapping the communication in a higher level protocol, this is primarily for proof of concept and small scale computations, as resolving the communication needs to be done in the software stack, instead of being able to rely on the network hardware to do it for you.

For this reason it would be best to use a network which supports RDMA natively, such as InfiniBand or SlingShot, which are able to provide the necessary low latency and high bandwidth needed for HPC systems¹⁷⁵.

Topologically the network would differ from traditional cloud network topologies, which are usually less interconnect than HPC systems, as a lot of the communication comes from the outside of the cluster and not between the nodes themselves. Our network would instead be ideally a full bisectional bandwidth network, where every node is connected to every other node, in order to provide the lowest latency possible. But more realistically a fat tree network topology would be used, just as discussed in the Networking and Interconnects section.

¹⁷⁵cf. De Sensi et al. 2020, pp. 1–14

4 Artifact Development

With the theoretical design of the system complete, the next step is to translate the design into a working prototype. Therefore, this chapter will cover the development of the same, including the iterative process and shining a light the difference between the initial assumptions we made during the design phase and the reality of the implementation.

For this reason the chapter is divided into three sections.

First we need to discuss the constraints on the prototyping process, this includes the existing infrastructure and the available hardware, as this project as discussed in the Introduction is the extension and repurposing of an existing system, build in a previous project.

Then the changes and improvements will be covered by the different steps of the iterative prototyping, each step will be discussed in detail, the changes made and the reasoning behind them as well as the issues encountered during the development process.

Lastly this chapter will conclude with a summary of the prototyping process, the final state of the prototype and most importantly a reflection upon the original design which has been deducted from the theoretical analysis in contrast to the practical implementation, which changes where made out of incorrect assumptions and which where made out of necessity.

4.1 Previous Work

As mentioned before, the goal of this project, besides to gather new knowledge about the conjunction of HPC and CC is to extend the result of a previous project by enabling it to become an Interactive Supercomputing Environment.

But as we will be take this previous system as a basis for our prototyping, we will need to understand the design and architecture of the existing system, and then decide which parts will be extended, which will be replaced, which will be removed and which will be kept as they are.

The goal of the previous project was also a converged HPC system based on the *Arkouda* library, but instead of focusing on an interactive environment, it is based around a GitOps workflow, where instead of the user interacting with the system directly, the user submits code to a central version-control-system, from where it gets cloned and packaged into container automatically. If certain predefined conditions are met the workflow-system will then start the container on the cluster and execute the code.

4.2 Existing Infrastructure

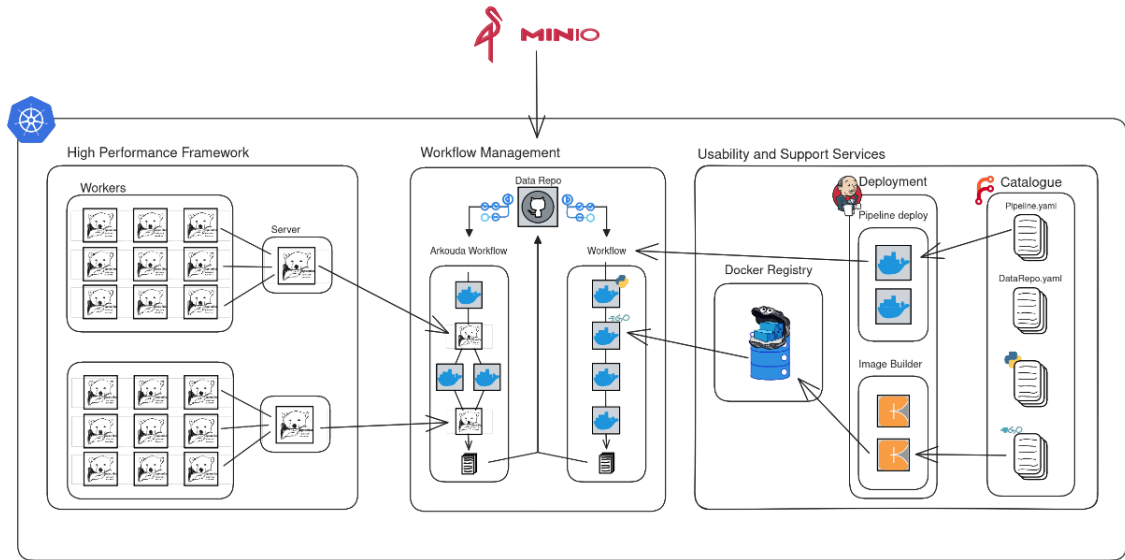


Fig. 14: High level architecture of Pachykouda¹⁷⁶

In the figure 14 we can see the high level architecture of the existing system, it is split up in three relevant sections:

- **Usability and Support Services:** On the most tightern side we have the supporting infrastructure which also runs on the Kubernetes cluster, this includes the versioning system (Forgejo), the agent which builds the containers and workflows (Jenkins) and the storage for the containers images (Registry V2).
- **Workflow Management:** In the middle we have the workflow system(Pachyderm), which is responsible for the orchestration of the workflow, it starts the containers, manages their input and output and ensures the versioning of the results.
- **High Performance Framework:** On the left we have the Arkouda workers and the Arkouda server, which are the actual HPC part of the prototype.

Beneath the service layer we have the Kubernetes cluster, which is responsible for the orchestration of the containers and managing the resources, for both the computation as well as the all the other services, the only exception being the storage of the datasets versioned by workflow system, which stores them in an external storage system, running on the a node of the same cluster, but not managed by Kubernetes.

On a hardware level we have a cluster of 20 nodes, connected by a 10Gbit/s network, in a star topology with a central switch. Each node has a *Intel Xeon E5-2680 v3 (48) @ 3.300GHz* and

¹⁷⁶Taken from Eckerth 2023

270GB of RAM according to the information given in the system stats. Also five of the nodes actually have an Infiniband capable network card, which could be used to test the functionality of RDMA capabilities of the Arkouda library, or jobs only requiring few cores but a lot of bandwidth. On the Network abstraction layer the cluster is using the *Flannel* network overlay, which provides a simple layer 3 network between the nodes¹⁷⁷.

4.3 Implication for the Prototype

In the section Suggested Architecture we created a suggestion for an architecture based on our deductive reasoning we have done during the deductive analysis, as we now have discussed the environment and existing infrastructure, we can now assess how our theoretical architecture can fit in a real world scenario.

The largest section of the previous system is the workflow management and its supporting infrastructure, while this is also able to access the compute resources provided by the HPC part of the system, from a structural standpoint it will not impair the function of the system we are planning to building. And the already running container registry will also be useful for our prototype, as we will be able to use it to store the images of the containers we will be using. Instead it demonstrates the high flexibility of kubernetes, being able to run many different services on the same cluster.

The HPC part of the system we will be using as a basis for our prototype. It is already able to run the Arkouda library, even though it is as of right now only accessible through the workflow system. while there is potential for optimization and improvement, it will provide a solid foundation for our work.

On the other hand, the cluster is mostly connected through a non RDMA capable network, which will limit the performance of the Arkouda library as discussed before. While we do have two nodes with Infiniband network cards, we will only be able to use them for testing purposes, but not in a full scale production like environment. While there is little we can do about the available hardware, Another consideration is the CNI currently in use on the cluster, as we might want to use a multi network setup, in order to provide a separate network for the RDMA capable nodes, we would need to install *Multus* or a similar CNI on the cluster, which we then combine with and RDMA capable CNI like *OVN4NFV* or *SR-IOV*.

While not a total necessity, it might be beneficial to have a container runtime available, which is closer aligned with the HPC part of the system, such as *Singularity*, as it brings additional features such as native GPU support and MPI support, which might be useful for the prototype.

¹⁷⁷cf. Flannel 2024

4.4 Iteration 1: Interactive Environment

This is the first iteration of the prototyping phase. The main goal is to get a Minimum Viable Prototype (MVP) running, what exactly this entails, how it was achieved, how this relates to our Suggested Architecture and what the results of this iteration are, will be discussed in the following sections.

4.4.1 Milestones and Goals

The main goal of this iteration is to have an Arkouda environment running on the cluster, which is accessible through a Jupyter client running on a local machine.

The milestones to be reached in order to achieve this goal are as follows:

1. **Verification** While there is an existing installation running on the cluster, it has been running unattended for roughly a year and was originally intended for a different purpose. Therefore the first step is to verify that the installation is still operational and that the environment is still functional.
2. **Deployment of Jupyter** The next step is to deploy a Jupyter server on the cluster, which is able to access the Arkouda environment. In order to do this a custom container image will need to be created, to run alongside the existing Arkouda environment.
3. **Access from Local Machine** The final step is to make the service as low barrier as possible, by making it accessible from a local device.

4.4.2 Implementation

Verification

Considering that the original installation was done as an experiment and has been running unattended for a year, and has survived a hardware change and a resizing of the cluster, it is still operational. One issue that was discovered during later steps was that during a final reformatting of the repository, the name variable of an Role-Based Access Control (RBAC) role was changed, which caused subsequent redeployment to fail, but which was quickly resolved after the issue was identified¹⁷⁸.

¹⁷⁸cf. Jon Eckerth 2024i

Deployment of Jupyter

While there exist many ready made images for Jupyter, the way the Arkouda client needs to be installed and configured requires a custom image to be created. This is due to the fact that the Arkouda client library is not available through any standard package manager, but needs to be installed from source¹⁷⁹, the same goes for the *Chapel* compiler Arkouda is built on¹⁸⁰. Making the users do this manually would be a waste of time and compute resources. Additionally in order to make the connection to the Arkouda environment, *IPython Kernel* needs to have access to current connection information at initiation and runtime.

Image:

The first step was to create this custom image, the foundational image for this is the image provided by the *Chapel* project¹⁸¹, as it already contains a whole precompiled version of Chapel. This is important as the Arkouda client library needs to use the binaries provided by Chapel, in order to create the necessary shared objects for the Python bindings¹⁸². One thing which was not clear was, whether the Chapel binaries were still needed after the shared objects were created, as our version only needs to act as a client. After the chapel binaries and the Arkouda client library were installed, the single user notebook server was installed and the *IPython* kernel was extended to load the required modules and establish the connection to the Arkouda environment as well as to terminate the connection when the kernel is shut down.

Deployment:

In order to get this custom image to work on the cluster, a deployment script was created which has access to the internal network of the Arkouda Namespace and the connection information needed to establish the connection to the Arkouda endpoint. The container was then pushed to the clusters registry and deployed to the cluster.

Testing:

To see whether the whole integration works the setup was tested with scripts provided by the Arkouda project for demonstration purposes¹⁸³. Unfortunately the system did not work as expected, the program terminating with the indication that the requests of the client were not being answered by the server. Due to no error messages being produced by the server, the assumption being that the connection, going through all layers of the Kubernetes network stack, was suddenly being dropped silently. After thorough investigation it was discovered that the issue was instead caused by a bug in the Arkouda *argsort* function causing a silent crash of the process, when the function was called on a string array¹⁸⁴. For the initial prototype it was worked around by Monkey Patching the function to use the *NumPy* implementation instead

¹⁷⁹cf. Arkouda - repo 2024c

¹⁸⁰cf. chapel-lang.org 2024a

¹⁸¹cf. Chapel 2024b

¹⁸²cf. Arkouda - repo 2024c

¹⁸³cf. Arkouda - repo 2024d

¹⁸⁴cf. hokiegeek2 2024

when the input is a string array. This is not an ideal solution, as the *NumPy* implementation is not distributed and therefore not as performant as the Arkouda implementation. But this will get address in the next iteration, where the performance of the environment will be improved. With this workaround in place the scripts provided by the Arkouda project were able to run successfully, and the connection to the Arkouda environment was established and stable for the duration of the test. More thorough testing in form of a benchmark for each of the operations will be done in the next iteration at section 4.5.

Access from Local Machine

The final step is to make the service accessible from a local device, with the knowledge gained from the previous research, this was a simple task. The Jupyter container was exposed to the outside world (the corporate intranet) through a *NodePort* service, which is now accessible from any device on the network. In order to prevent unauthorized access, as the service is not secured by default, additionally a password was set for the Jupyter server.

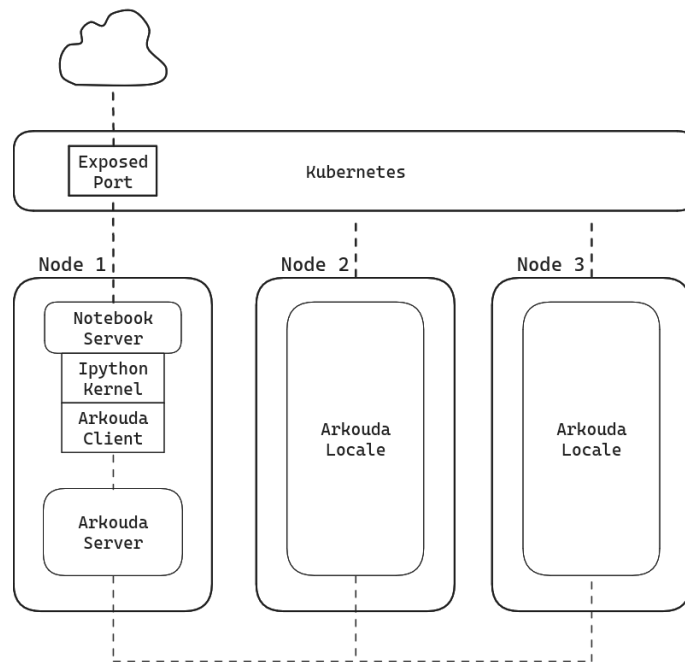


Fig. 15: High Level Overview of the Jupyter Setup

In diagram 15 above is shown how the setup is structured, most importantly the Jupyter container is the only container which is exposed to the outside world, the Arkouda container is only accessible from the Jupyter container. Which itself connects to the *Arkouda* Server only through a node internal network connection. Each of the Ipython kernels has its own independent connection to the Arkouda server, which is established at the start of the kernel and terminated at the end of the kernel.

Currently this system is completely ephemeral, as neither the Jupyter container nor the *Arkouda* GASNet domain are being persisted.

4.4.3 Evaluation of Results

The Goal of this iteration was the creation of a MVP of an interactive Arkouda environment running on a multi node compute cluster, accessible through a Jupyter client. While some issues where encountered during the implementation, the goal was successfully achieved. The Arkouda environment is now accessible from any device on the corporate intranet, and the connection to the Arkouda environment is stable and easily usable.

During the implementation some original assumptions made during the research and the ‘Suggested Architecture’ on page 35 but showed to be infeasible or incorrect and had to be adjusted.

While using Chapel as the base for the high performance computing has shown to be a good choice, the choice of using Arkouda to increase the ease of use for python developers has shown that it is still quite young in comparison to long established libraries like *NumPy*, and sometimes lacks Quality of Life (QoL) features, like a simple pip based installation or a more verbose error handling. But as it is unrivaled in the area of python based distributed computing on large scale data¹⁸⁵, it is a promising library to invest more time into.

While there exists a container image to be used as an Arkouda client, which was intended to be used as a base layer for the Jupyter container, is build on the basis of an SMP Chapel compilation. If the Chapel binaries need to be replaced anyhow, there is no reason not to use the right Chapel image from the start, and building everything on top of that our self.

Conclusion:

The goal of this iteration was to create a MVP of an interactive Arkouda environment, which is accessible from a local device. This goal was successfully achieved. The environment is now accessible from any device on the corporate intranet, and the connection to the Arkouda environment is stable and easily usable.

The initial assumptions made during the research phase and the Suggested Architecture, where addressed and corrected during the implementation phase. And during the implementation some new issues where discovered, such as an issue with a function in the Arkouda library which will need to be addressed. The next iteration will focus on improving the performance of the environment, as during the testing the opportunity for improvement was discovered.

¹⁸⁵cf. Ronaghan 2020, pp. 677–677

4.5 Iteration No 2: Performance Improvements

In the previous section the prototype was successfully deployed and tested. The next step is to move this from a MVP to a more stable and performant environment, this includes improving deployment and execution times, as well as properly addressing previous workarounds.

4.5.1 Milestones and Goals

1. **Deploy time:** As the image contains the entire Chapel compiler, its source code as well as the Arkouda client library and its dependencies, the image is quite large. This slows down the deployment process, as the image needs to be transferred to the cluster and then unpacked. The goal is to reduce the size of the image to speed up the deployment process.
2. **Container Runtime:** The current container CRI is *Containerd* which, while being lightweight, is not the fastest CRI available. The goal is to switch to a faster CRI more suited to HPC workloads, such as *Singularity*.
3. **Investigating Arkouda Crash:** During the previous iteration, the Arkouda environment crashed when running the *argsort* function on a string array. This was fixed by Monkey Patching in the *NumPy* implementation. The goal is to investigate the cause of the crash and fix it properly, to gain back the performance lost by using the *NumPy* implementation.
4. **Performance Testing:** In order to even quantify the performance in the first place, a way to measure the performance needs to be established. This will be done by running a set of benchmarks before and after the changes are made.

4.5.2 Implementation

Deploy time

The time it takes a service from being requested and being available is crucial for the flexibility of a Kubernetes cluster, as this determines how fast the cluster can adjust to changing workloads, increasing demand or failures. The current size of the image as reported by *podman* is 3 GB, which is quite large, for a simple client service. This is due to the fact that the entire Chapel project as well as all build tools are included in the image. Having such a large image slows down the deployment process, as the image needs to be transferred and unpacked to the node. One solution to this is prefetching the image on the nodes, is called *image pre-pulling*. Taking the time it takes to pull the image at an earlier time and storing a local copy on the node, this way the image is already available when the service is requested. This is feasible for small clusters with a low diversity of images, but the overhead of keeping local copies of images locally, scales quadratically with the number of images and nodes, as each image needs to be stored on each

node. Therefore, we want to investigate by how much we can reduce the size of the image, to speed up the deployment process.

For this reason a script was written to measure the time it takes to deploy an image. More about how this testing works can be found in the Performance Testing section of this iteration.

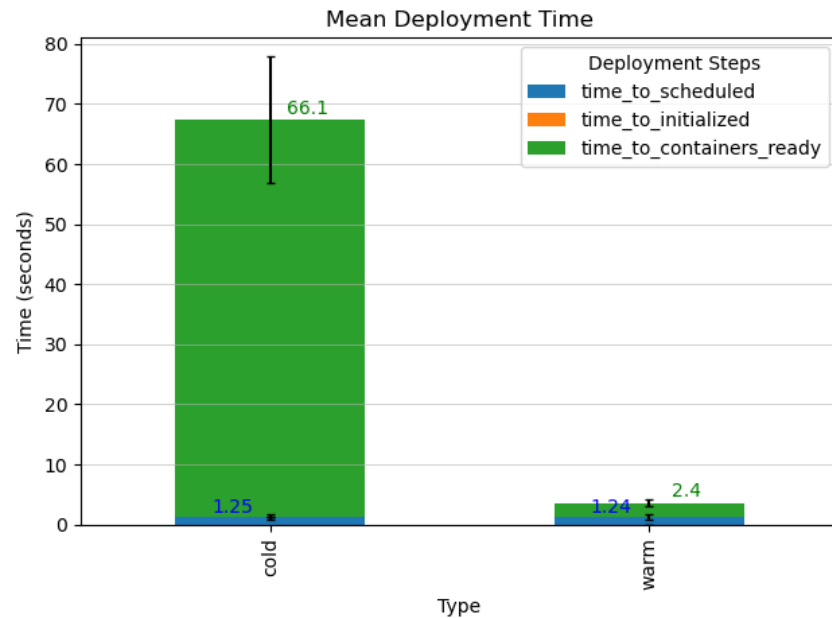


Fig. 16: Deployment time of a 2.86GB image¹⁸⁶

Figure 16 above shows the time the different steps of the deployment process take in seconds. In blue is the time it takes from requesting the deployment of a service until Kubernetes assigns the pod to a node. In the orange section we would see the time it takes the initialization containers of the pod to start. The init-containers are needed to set up the namespace and network for the pod; they pull and start the actual container image (see ‘Containerization’ on page 7). However, as Kubernetes can only measure events with a resolution of seconds, and these containers are ready in milliseconds, they are too short to be measured reliably by Kubernetes. Going from requested to ready in between the polling makes it seem like they are ready instantly. The green section is the time it takes from the initialization containers being started to the whole pod being able to accept requests. This includes the time it takes to pull the image from the registry, unpack it, the start of the container and the time for the software inside the container to be ready to accept requests¹⁸⁷.

With all else being equal we can clearly see the large difference between fetching the image from the registry, and having the image already available on the node. As container images are built out of COW layers (see ‘Docker’ on page 11), one can specifically target the layers which are the largest and try to reduce their size. One thing to note is, that due to every command creating a

¹⁸⁶Data can be found in the project repo at Jon Eckerth 2024d

¹⁸⁷cf. Kubernetes - Docs 2024k

new immutable layer, one should try to chain commands together, to reduce the number of layers created. So instead of creating a layer for the installation of a package, another layer for the usage and a third layer for the removal of the package, one should chain these commands together in one layer to result in a single layer containing only the artifact needed for the package. The most effective method to reduce the size of an image besides, choosing smaller base images, and being mindful of the layers created, is to use multi-stage builds. This is a feature of the Dockerfile format that allows you to create multiple images from a single Dockerfile. Its main use case is to build the project in one image and then copy the build artifacts to a second image. This is useful as the first image can contain all the tools and dependencies needed to build the project, while the second image can be a much leaner distribution, only copying the build artifacts from the first stage over, reducing the overall size of the image significantly, while making the build process easier.

These practices were applied to our initial image, reducing the size from 2.86 GB down to 710 MB, a reduction which is a largely due to the use of multi-stage builds, replacing the Chapel image after the Chapel build was done. Additional size reduction could be achieved by isolating the python modules after the Chapel build process and moving them using a Virtual Environment (VENV) to the final image.

Overall the size reduction of the image had a significant impact on the deployment time as can be seen in figure 17 below., reducing the median deployment time of a non-pre-pulled image from 63.47 to 19.7 seconds, a reduction of 69%, which is nice for a simple change in the Dockerfile.

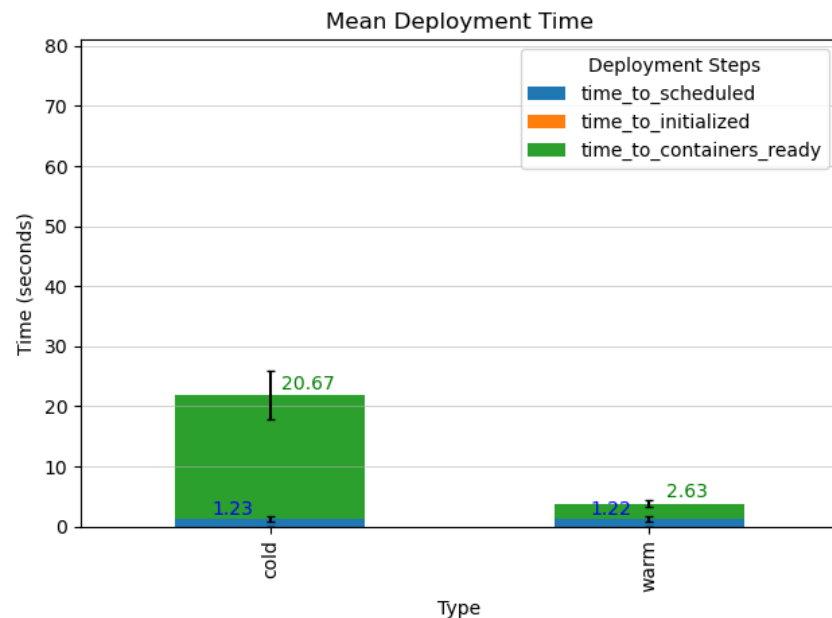


Fig. 17: Deployment time of a 710 MB image¹⁸⁸

¹⁸⁸Data can be found in the project repo at Jon Eckerth 2024e

But if we compare the difference the build makes on the prepulled image, we can barely see any difference, both being around 2 seconds. Being so close to a resolution of 1 second we can not see much of an effect.

As the images of the HPC are taken from the Chapel and Arkouda projects, they can also be given the same treatment, as the images are quite large as well. While the Arkouda server and locale environment are even larger than the Jupyter image, they are also much more complex, and rely on more than just the python bindings to work. Multi-stage builds are especially important here since the tooling needed to compile the chapel project alone already make up more than a gigabyte of the image. It is also built on top of the original Chapel image, which is already quite large, containing the precompiled Chapel binaries, saving the time of compiling Chapel during the build process. But compiling both Chapel and Arkouda from source, using multi-stage builds, and VENVs to isolate the python modules, we were able to reduce the size of the image significantly. The size of the Arkouda image was reduced from 3.88 GB to 1.08 GB, a reduction of 72% an effect which is even more important than the reduction of the Jupyter image, as the Arkouda images will be running on each and every node in the cluster, being scaled up and down as needed, making the flexibility of them quite important.

Container Runtime

As can be seen from the diagrams above, the significant effect changing the size of the image has on the deployment time, only effects the time it takes pulling and unpacking the image, but not the startup time or the time it takes for the software to be ready. The setup time for a prepared container is mostly determined by the software inside the container as well as the overhead of container, which is determined by the CRI used. Rewriting Jupyter notebook or a HPC scale parallel processing framework like Chapel to be faster is certainly out of scope for this thesis, however on the other hand changing the CRI is a much more manageable task, which will also might prove beneficial to later iterations as discussed in the ‘Suggested Architecture’ on page 35.

Replacing the CRI on a running Kubernetes cluster is akin to changing the engine of a car while it is still running, as the CRI is responsible for the runtime of the containers, and is responsible for starting and stopping the containers, as well as managing the container’s lifecycle. Completely replacing an all-purpose CRI like *Containerd* with a HPC optimized CRI like *Singularity* might have unforeseen consequences. Therefore, having both available on the same nodes, and utilizing whichever is more suited for the task at hand, would be ideal.

Unfortunately, this step proved to be more difficult than anticipated, as the *Singularity* CRI documentation claims to be able to run on a Kubernetes cluster, which is not untrue; But due to the singularity becoming a part of the open software foundation in 2015, the project has been split into a community edition being maintained by volunteers, and a completely rebranded version called *Apptainer* which has moved away from the HPC space completely focusing now

on subjects such as *Sigstore* and Continuous Delivery. The original version of *Singularity* is still available on the *Sylabs* website, but has not been updated in the last 5 years, a fact which is not mentioned when looking at the documentation¹⁸⁹, only visible when looking at the actual code of the project¹⁹⁰. The version of *Singularity* that is being maintained by the community, and more up to date, is much smaller in scope and decided to focus their efforts to core projects, abandoning the CRI implementation¹⁹¹. While it technically still possible to run a cluster with *Singularity* running as a CRI, it is not possible to do so for any version of Kubernetes beyond 1.26. This is due to the fact that the *Singularity* CRI is based on the now deprecated version *v1alpha2* which was removed in version 1.26¹⁹².

It is possible to run a *Singularity* container without the CRI on a Kubernetes cluster, as it supports the OCI standard, which means that one can convert a *Singularity* image to any other OCI compliant image, and then run it on the generic CNI of Kubernetes. The features of *Singularity* that made it originally so attractive for HPC workloads, such as its native support for GPUs and *Infiniband* would need to be implemented the hard way, in Kubernetes itself, which then would defeat the purpose of using a different CRI in the first place.

4.5.3 Investigating Arkouda Crash

As we touched on in the previous iteration (4.4.2) we had a problem with the Arkouda environment crashing when a certain function is called on a string array. At that point in time, tracking down the error proved time-consuming, and once enough information was gathered a workaround was found by Monkey Patching the function to use the *NumPy* implementation instead(which can be found in the project repo¹⁹³), while leaving the proper fix for the next iteration, as the primary goal was to get the system up and running. After further investigation the hypothesis was that the error was not actually a bug in the code, but a mismatch in the communication protocol between the Jupyter server and the Arkouda server. While both were using the same version of the Arkouda library the Arkouda was originally build when the system was set up during the "Pachykouda" project; the Jupyter server being build during the first iteration of this project, there might have been some changes within the containers since then.

As we have been relying on these containers provided by the Chapel and Arkouda projects, we have little control over the versions and the variables used to compile the binaries. Additionally, these images are quite large, slowing down the deployment as discussed in the first iteration. To reduce the size of the images and to gain more control over the environment, the images were rebuilt from scratch. So a common base image was created, containing all the necessary files and binaries needed for the client, server and locale images. From this base image, the corresponding images were created, having an identical common environment but including only the necessary

¹⁸⁹cf. Singularity - Docs 2024b

¹⁹⁰cf. Singularity - Repo 2024

¹⁹¹cf. Singularity - Repo 2024

¹⁹²cf. Kubernetes 1.26 Release Team 2022

¹⁹³cf. Jon Eckerth 2024g

files for the specific image to be created. Thereby ensuring that the environment is the same for all images and have the same version of the binaries and libraries at heart.

After the images were successfully brought to a state where they could be used as a drop-in replacement for the original images, the client was tested again, and the error was gone.

4.5.4 Performance Testing

Originally it was planned to test both the effect of the image size on the deployment time, as well as the effect of the CRI on the startup time and the execution time of the software. Due to the fact that *Singularity* only supports OCI based execution on Kubernetes, there would be no relevant difference as both get converted into the same container format¹⁹⁴, making the effort of converting the compute nodes to *Singularity* inefficient in the context of this project.

But testing the startup time was still possible and with the conversion of the Arkouda images and the reduction of the size of the Jupyter image, having a good sense of how quickly the system is able to respond to changes in the workload is still important. The testing was done by utilizing Kubernetes "Pod Conditions" which are a set of states a pod transitions through in its lifecycle, which can be used to monitor the state of the pod¹⁹⁵. Being able to query this information with the Kubernetes API, we can measure the time it takes for a pod to go from start of the request to the pod being scheduled on a node, the time it takes for the pod to be initialized and the time it takes for the pod to be ready to accept requests.

So in order to compare hot and cold starts, it was needed to make sure that during the first run, all of the images were not pre-pulled, and during the second run, all the images were pre-pulled, the former needing to be wiped from the corresponding nodes each time. The results of the testing can be seen in the diagrams above, and show a significant difference between the two, with the pre-pulled images being ready in a fraction of the time of the non pre-pulled images. The tests were done with a sample size of 400 with the images being deployed on the whole cluster, and the results were then aggregated to show the median time it takes for the images to be ready. The code¹⁹⁶, the notebook of the analysis¹⁹⁷ and the raw data¹⁹⁸ can be found in the project repo.

But as we are planning to compare the performance improvements of future iterations, it is important to establish a baseline anyhow, for this reason a testing environment was set up and benchmarks were run on the cluster. In order to do this, the Jupyter client was modified to run the micro benchmarks provided by the Arkouda project¹⁹⁹. The micro benchmarks test the performance of each of the major functions, such as *argsort*, *sum* and *aggregate*, on all the

¹⁹⁴cf. Open Container Initiative 2024

¹⁹⁵cf. Kubernetes - Docs 2024k

¹⁹⁶cf. Jon Eckerth 2024f

¹⁹⁷cf. Jon Eckerth 2024b

¹⁹⁸cf. Jon Eckerth 2024c

¹⁹⁹cf. Arkouda - repo 2024a

supported data types. Getting a very fine-grained view of the performance of the Arkouda environment, in each and every aspect. As the cluster has four nodes which are theoretically InfiniBand (IB) capable, and will be enabled in the next iteration, the benchmarks were limited to four nodes as well in order to have a fair comparison. The tests were run with a problem size from 10^6 to 10^{11} items per array, in order to show how the performance behaves with increasing problem sizes.

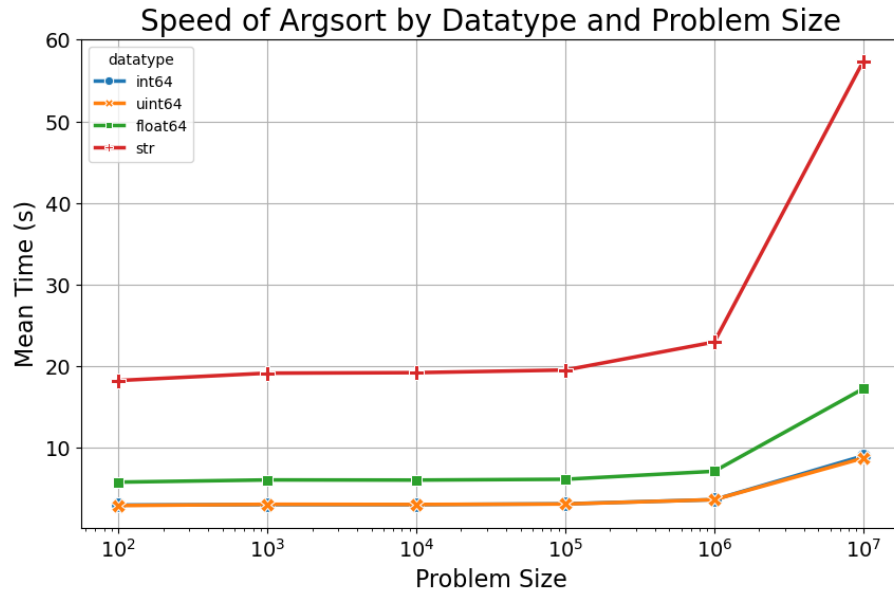


Fig. 18: Argsort by Problem Size

The figure 18 above shows the how the performance of a tightly coupled problem such as the argsort function. Here it can be seen how the time to solve the problem starts to increase exponentially, corresponding to the problem size increasing by orders of magnitude for each test.

Another interesting aspect is the performance of the different data types, as sorting a string array of the same length takes much longer than any other data type. This is due to the variable length of strings in Arkouda, making the addressing of the elements by index much more computationally expensive²⁰⁰

While argsort implementing the sorting algorithm *least-significant-digit radix sort* demonstrates one of the worst cases for the performance an distributed system, as it is a tightly coupled problem with a complexity of $O(w \cdot n)$, where w is the number of bits in the key and n is the number of elements in the array. The *aggregate* function, shown in the diagram 19 below, shows a much more diverse set of problems and how the performance of the system behaves with them.

¹⁹⁹Data available at: Jon Eckerth 2024a

²⁰⁰cf. Arkouda - repo 2024e

²⁰⁰Data available at: Jon Eckerth 2024a

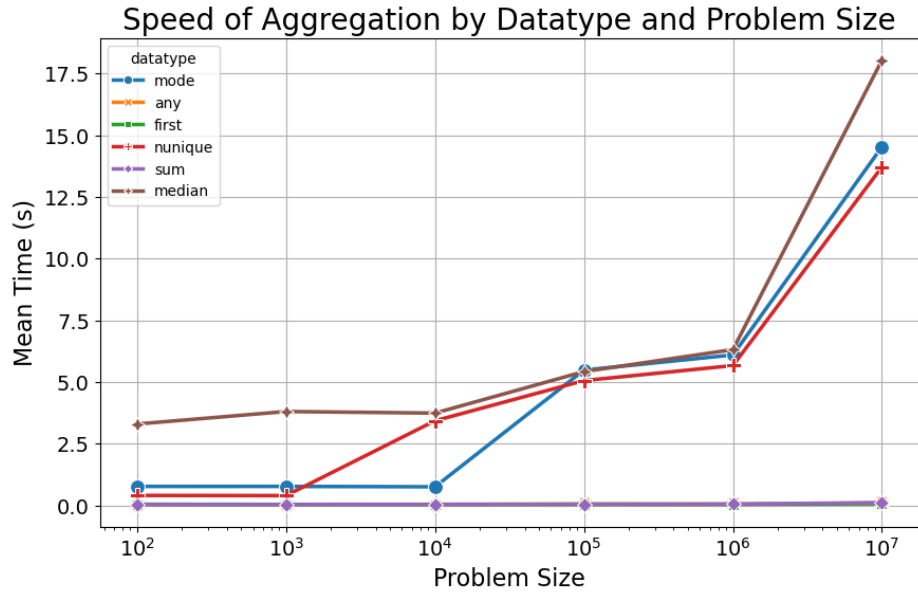


Fig. 19: Aggregated Speed by Data Type

The figure 19 above shows clearly the wide gap in performance between *embarrassingly parallel* problems like *sum*, *any* or *first*, which are being split up into smaller problems and can be solved independently on each of the nodes, without the need for any communication between them. While less computationally expensive than *argsort*, functions such as *median* or *mort* become time expensive, as they can not be split up into smaller problems as easily.

So far the performance of the system has only been measured by the time it takes to solve a problem of a certain length, but as the size of the elements themselves are not created equal, or might even vary such as in the case of strings one should also take into account the actual amount of data being processed.

Diagram 20 is especially interesting as it shows the clusters most extreme conditions. On one side the functions with the highest processing rate and then the three functions with the slowest rate of processing.

The speed of scanning the distributed array is essentially only limited by the speed of the underlying network, as the data can be read in parallel from all nodes, and then aggregated on the client. While the *scan* function is very close the theoretical ideal of the network speed, showing that the overhead of the whole network stack can be as low as 4.4% under certain conditions; The streaming function is not too far from the ideal processing speed of 30% of the network speed. As the streaming function sends the *axpy* operation to all nodes, calculating $y \leftarrow \alpha x + y$. Adding two arrays together while multiplying one of them by a scalar, this is a very simple and fast operation, as each of the nodes can calculate the operation independently and then send

²⁰⁰Data available at: Jon Eckerth 2024a

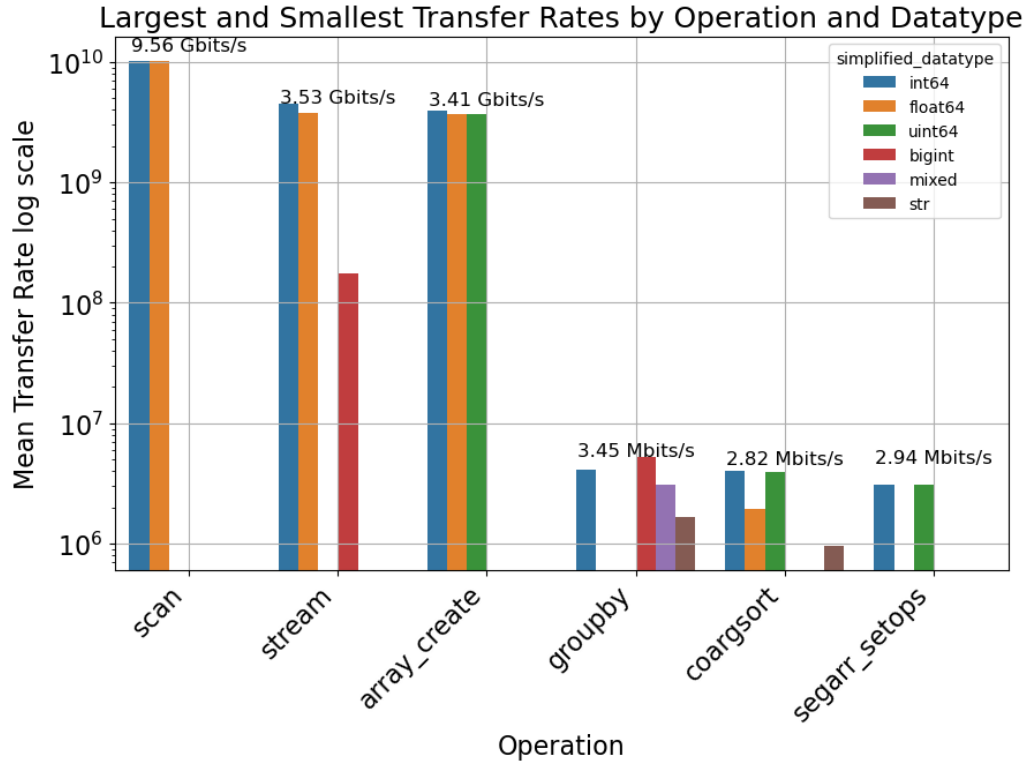


Fig. 20: Operation Data Transfer

the result back to the client. The only delay being the time it takes to send the two arrays and receive the resulting array back.

The three arkouda functions which take the longest to process the data are `coargsort`, `groupby` and `segarr_setops`. While `groupby` directly calls the `argsort` function and then performs additional aggregations, `coargsort` provides an index array just like `argsort`, but instead of ordering it numerically it orders it by the order of another array. As these functions just as `argsort` are tightly coupled problems, and rely on a lot of communication between the nodes, they are the slowest functions in the benchmark. But the outlier in this case is the `segarr_setops` function, which is a set operation on segmented arrays. `Segarr_setops` is even slower than complex sorting algorithm, which seems counterintuitive at first, but the reason for this is that the function has not been implemented for the distributed array yet²⁰¹, and is being executed on the client negating the benefits of the distributed array implementation.

4.5.5 Evaluation of Results

The goal if this iteration was to move the MVP to a more stable and performant environment, reducing load times, fixing bugs and improving compute performance. From this the need for a

²⁰¹cf. Arkouda - docs 2024a

way to measure the performance in the first place was established.

The first goal, reducing the deploy-time by optimizing the image size of all parts of the system was a complete success, with reductions in the size of the images around 70% and the time to deploy reduced significantly in the case of a non pre-pulled image. While the original suggestion only intended to extend the images depending on our needs (see ‘Cluster Level Architecture’ on page 40), during the prototyping phase the potential of large improvements were discovered, and the decision was made to rebuild the images from scratch.

The second major goal, replacing the CRI with *Singularity* was not accomplished, and the decision was made to not pursue this further, as this would require updating a CRI implementation which has been unsupported for the last 5 years. While the performance increase would have had an impact on the container overhead, a more significant impact would have been the native support for *Infiniband* and GPUs, the former being especially important for HPC workloads (see 3.2.2) in but even more so with the current GASNet implementation of Arkouda (see 3.2.3). This gap in potential will be addressed again in the next iteration.

During the effort to rewrite the container images from scratch, the communication error was fixed, and the Arkouda environment was running without any issues, passing all the unit tests and benchmarks. The current theory being, that the issue was most likely due to a mismatch in the communication protocol, between the jupyter server and the Arkouda server, which was fixed standardizing the versions across all images.

The last goal, establishing a baseline for the performance of the system was originally intended to be the basis for the comparison between the CRIs, but even though this was not possible, the benchmarks were still run, in order to have a baseline for the performance of the system.

4.5.6 Conclusion

While some of the goals were successfully accomplished, the goal of replacing the CRI with *Singularity* was not. The performance of the image distribution was improved significantly, and a baseline for the performance of the system was established. The Arkouda environment was fixed and is now running without any issues.

4.6 Iteration No 3: Infrastructure and Interconnect

4.6.1 Milestones and Goals

As has been discussed in the section ‘HPC Infrastructure’ on page 21 the second most important aspect of a HPC system, after the computation is the communication. As the networking was the glass ceiling of the previous iteration, both in maximal parallel performance as well as slowing down the communication needed for tightly coupled applications, the goal of this iteration is to improve the networking of the cluster.

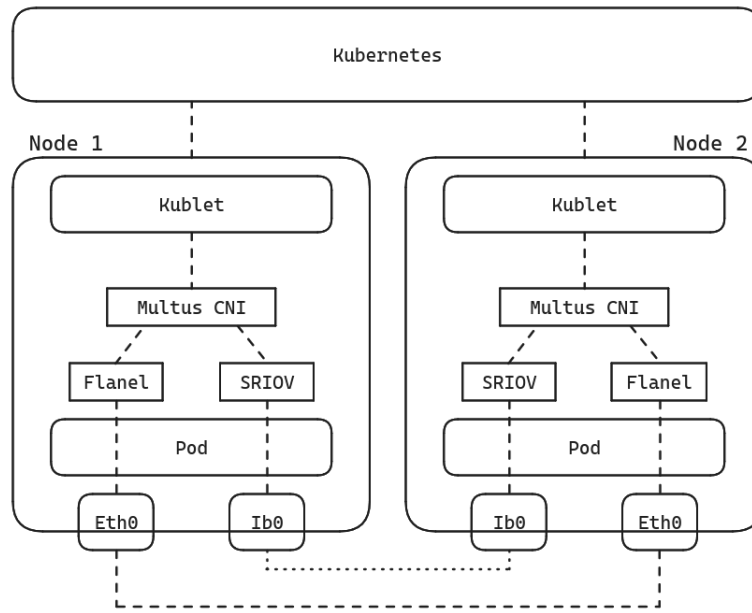
1. **Multi CNI Support** The current networking setup of the cluster is based on a simple *Flannel* overlay network. In order to support more complex networking setups, the CNI needs to be able to support more than one network interface at a time. This will allow the cluster to be able to support multiple types of networking interfaces, such as IB and Ethernet.
2. **Utilizing IB** The current cluster includes five nodes that are equipped with InfiniBand (IB) networking, enabling faster node-to-node communication, which is crucial for HPC as detailed in sections 3.4.3 and 3.2.2. This allows the IB-enabled nodes to communicate with each other using IB while maintaining the capability to interact with the rest of the cluster over standard networking protocols, ensuring seamless integration within the existing infrastructure.

4.6.2 Implementation

Multi CNI Support

The first step in achieving multi CNI support was to identify a CNI that supports multiple interfaces. As described in the section ‘Networking’ on page 15, the way Kubernetes handles networking is by creating a VETH for each pod, which gets patched into the CNI plugin. Therefore, if a pod is to have multiple network interfaces, this needs to be handled by the CNI plugin. One such CNI plugin is *Multus*, which acts as a meta-plugin, allowing multiple other CNI plugins to plug into a single pod, providing multiple network interfaces. This is achieved by acting as an intermediary between the host and the container, providing a single interface to the kubelet/kubelet, while plugging into multiple CNI other plugins to provide the necessary networking interfaces to the container²⁰².

²⁰²cf. Multus 2024a

Fig. 21: Multus CNI with SRIOV and Flannel²⁰³

In diagram 21, above one can see the end goal of this whole iteration, as this diagram shows how *Multus* slots itself between the *kublet* and the default CNI plugin, in our case *Flannel*, and then plugs in another CNI plugin, in this case *SRIOV* being used to provide the direct access to the IB network.

To implement this, the nodes were first cordoned and drained to ensure that no pods were running on them. The binaries for *Multus* were compiled and built into a container image, which was run as a daemonsets on the cluster. The container placed accessing the filesystem of the hosts, copying over the binaries, and most importantly replace the config files of the previous CNI plugin and not deleting but deprioritizing them²⁰⁴. This is done so the *Multus* plugin will be the first to be called by the *kubelet*, when creating a pod; if nothing else is defined the *Multus* plugin will then simply call the default CNI plugin, in this case *Flannel*.

Generally, installing another CNI plugin would not be much more complicated as the *Multus* plugin. Adding the CNI binary to all hosts and adding a generic configuration file to the *kubelet* would be enough. However, as the goal is to connect an IB network to the pods, the process is a bit more complicated and will be discussed in the next section.

²⁰³Based on: RedHat 2024a

²⁰⁴cf. Multus 2024a

4.6.3 Evaluation of Results

Installing the *Multus* plugin was successful, and the plugin was able to run as expected. Injecting itself automatically between the existing *Flannel* plugin and the *kubelet* worked exactly as expected.

Utilizing IB

As described in the chapters ‘HPC Infrastructure’ on page 21 and in the design of this prototype ‘Hardware Level Architecture’ on page 41, the Interconnect is a crucial part of a HPC system. Especially so in in GASNet systems relying on the low cost RDMA based injections to achieve its global address space (further discussed in 3.2.3).

But Infiniband is by far not as deeply integrated in the Kubernetes ecosystem as the generic TCP/IP stack. As such the process of connecting the IB network to the pods is a bit more complicated than just installing a CNI plugin.

First the IB network needs to be configured and available on the hosts. The right kernel modules need to be loaded, the right firmware installed and the right Unified Extensible Firmware Interface (UEFI) settings need to be set. If all the hardware is set up correctly, and the network has a subnet manager running, the IB network should be available on the hosts.

Unlike the TCP/IP stack on Kubernetes the IB network is not made out emulated interfaces within the Pod which get routed by the containerized infrastructure until it funnels out a single interface on the host (as described in 3.1.4). Instead, the IB network is a physical network that which needs to execute these operations on the hardware level in order to maintain the extremely low latency and high bandwidth that IB networks are known for²⁰⁵.

But in order to provide more than one container at a time with isolated access to the IB network, the IB network needs to somehow be split up into multiple virtual interfaces for the pods to use. As this has already been a topic of interest to virtualization technologies, wanting to take advantage of the low latency and high bandwidth of IB networks similar to the pass through solutions described in the section ‘Virtualization and Isolation’ on page 6.

SRIOV:

One such technology is Single Root I/O Virtualization (SRIOV), which splits a single physical Peripheral Component Interconnect (PCI) into multiple virtual PCI interfaces, which can then be access by different entities on the host, acting as if they were physical interfaces. These Virtual Function (VF)s can be configured by directly accessing the original Physical Function (PF) to create the VFs as needed.

²⁰⁵cf. Pfister w.y.(b)

Both of these still reside in the hosts' kernel space and are not yet available to the pods. So if a VF has been created manually the Pod needs a way to access this VF directly. This is where the *ib-sriov-cni* comes in, as it is a simple CNI plugin that allows a pod to connect to an existing VF on the host²⁰⁶. While this plugin offers access to a VF it needs to be provided with all necessary network information to establish the connection. This includes everything, from the PCI address of the VF to access to the IP Address Management (IPAM) data of the network the VF should connect to. This requires setting up and writing the configuration for each pod on each node manually. The *ib-sriov-cni* plugin itself simply resides as a binary in the *CNI* directory of the host, being called upon if *Multus* is instructed to use it for a pod.

SRIOV-Device-Plugin:

In order to avoid to manually configuring the network data for each pod, the *SRIOV-Device-Plugin* can be used. This Kubernetes plugin scans the nodes via a Daemonset for PCI devices which match a given definition and index them. This creates a resource pool of SRIOV devices available to connect to, which can then be used by the *ib-sriov-cni* plugin to connect to the VFs. Reducing the need to manually declare the data of the PCI device manually for each pod. This needs to be configured with the parameters of the PCI devices to be matched and then run as a Daemonset on the cluster.

Sriov-Network-Operator:

Now that the pod can automatically discover VFs on the host, the network data configuration for the pods need to be set up, so that a pod can actually receive the configuration of which network to connect to. The *sriov-network-operator* is a Kubernetes operator that allows a user to define a custom resource that describes the desired state of the network configuration of the cluster. Requesting a specific network devices to provide the necessary VFs to the pods.

Once the *sriov-network-operator* pod has been deployed to the head node of the cluster, it will start a daemon on each node which keeps track of the state of each node and the network devices ready to be used by the *ib-sriov-cni* plugin. It also sets up endpoints for the KubernetesAPI to interact with the *sriov-device-plugin*.

Once a user creates an instance of the custom resource called *SriovNetworkNodePolicy*, the *sriov-network-operator* will create a *NetworkAttachmentDefinition* object which will can be referenced in the pod configuration as an annotation. While in the background the *sriov-device-plugin* on the nodes register the difference between desired and actual state of the network devices, and signals the KubernetesAPI to update the state of the Node. The Api then calls on the *sriov-network-operator* to update the configuration.

Once the VFs are available a pod can be created with only the name of the network to connect to, the once a pod with the right keyword is requested, the resource injector matches the network name with the existing *NetworkAttachmentDefinition* and injects the necessary information into

²⁰⁶cf. IB-SRIOV-CNI - Repo 2024

the pod configuration. Using this configuration the *Multus* plugin then knows to call the *ib-sriov-cni* plugin providing the injected network information to the pod.

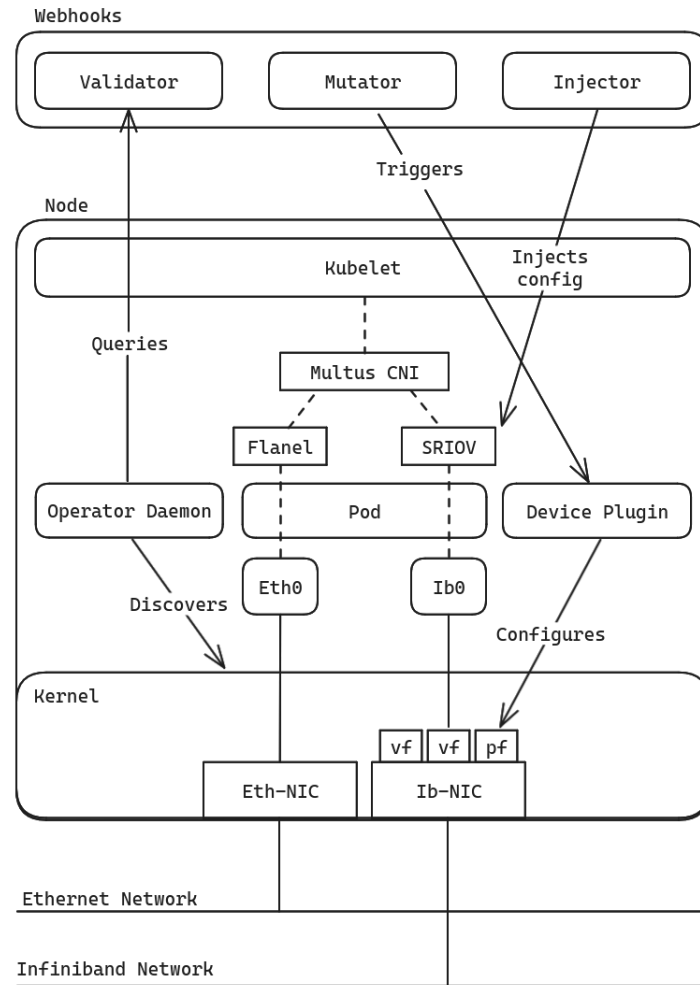


Fig. 22: Simplified architecture of a SRIOV enabled Kubernetes Node

In diagram 22 above, a simplified version of the interactions within a single node of the cluster is shown. Not pictured are the daemons running on the head node or, calls of the Kubernetes API, or the mutating config maps which are being mutated by the system to declare the current and desired states of the network devices.

Implementation process

During the implementation of this iteration many problems arose in orchestrating these many interdependent components. As the use case for IB enabled HPC on Kubernetes has not been widely explored, many of the components are originally intended for other use cases, unmaintained or proprietary. A large part of the existing documentation was addressed at fellow developers, describing the necessary steps built the components, but do not provide information about how to deploy or configure them, as this is assumed to be known by the developer of

the component in the first place. This made the process of setting up the IB network on the cluster a very time-consuming and painstaking process, involving a lot of reverse engineering and trial and error, sometimes discovering known caveats documented only as a side note in the source code of the component. The most helpful resource was *Redhats* documentation²⁰⁷ for their *OpenShift* platform, their custom Kubernetes based platform. While useful and detailed, their documentation is only targeted at their platform and the translation to vanilla Kubernetes is not always straightforward.

Amongst the issues encountered was a communication problem, between the Kubernetes API and the sriov-operator-webhook-config *mutatingwebhookconfigurations* and its *validatingwebhookconfigurations*. While the webhook was configured according to the documentation, the underlying pod was able to receive and log requests and the provided certificates were valid, signed and concurrent between the webhook and the container the API was getting a "500 Internal Server Error - Bad Gateway" when trying to reach the webhook. This behavior could not be reproduced in any other way during debugging. While the issue was worked around at first by overwriting the webhook, it was later coincidentally discovered that The system requires the filename of the certificate file to be identical to the secret containing the file. While a banal issue, it was only documented in form as a comment in the code of the webhook itself²⁰⁸.

The most Significant hurdle was the in transparent configuration of the IB device and the components directly interacting with it. As there is little community effort behind these projects and numerous proprietary components involved, the configuration of the IB device was the part of the prototyping process which cased the project to hit the deadline for the prototyping process, requiring the project to be continued after release of this paper.

Current State

While the above described infrastructure has been set up and brought to a working state, completely enabling the configuration of the cluster by declaring the required configuration of the nodes and declaring a network. Even the mounting of the IB network to the pods has been achieved. And while the VF functions are accessible and intractable by the pods, the pods are not able to able to send or receive data over the IB network. This is due to the interface requiring the RDMA module requiring to be set to the *"exclusive mode"*. While setting the device to exclusive mode is a simple command, this in turn disables the VFs completely. While this is most likely a trivial configuration issue, the deadline of the prototyping process has been reached and this paper is to be handed in. The project will be continued after the release of this paper. The current state of the project can be found in the corresponding repository and will be updated as the project progresses²⁰⁹.

²⁰⁷cf. RedHat - Openshift 2024

²⁰⁸cf. Sriov-Network-Operator - Repo 2024

²⁰⁹cf. Jon Eckerth 2024h

4.6.4 Conclusion

While the setup of the infrastructure as a whole was successful within on the Kubernetes Level, the many small issues resulted in the project running out of time, without being able to fully set up the hardware and kernel level configuration of the IB network. The original goal of this iteration was to enable the IB network to be used by the pods, in order to enable a high speed low latency network for the HPC applications. This goal has not been achieved, as the pods are not able to send or receive data over the IB network, without the correct configuration of the IB device. While this iteration was not finished, this is not due to any insurmountable technical reason, but rather due to the time constraints and the complexity of the set up the infrastructure.

Comparing this state to the our original design, the set up has come very close to the original intention, as it was already suspected that a multi network infrastructure might be necessary. While it was originally planned to use the Singularity CNI in order to provide the native IB support, its abandonment by the community required the use of a manual integration of the necessary components into the cluster. But even in the case a dual CNI setup would have been needed to provide the necessary network abstraction for the standard Kubernetes functionality to function with the *Singularity* containers.

While there will be no additional iterations in this paper, the project will be continued further, both to achieve the original design goal, as well as to follow up on the opportunities described in the section ‘Future Work’ on page 79.

Therefore the next undocumented iteration will first focus on configuring the IB network to be accessible by the pods and compare the performance of the IB network to the standard established during the second iteration.

4.7 Conclusion of the Prototyping Process

As the goal of this chapter was not just to create a usable artifact for its own right and explore the process of building a kubernetes-based HPC tool, but also to stand as the validation step of the system designed in the ‘Suggested Architecture’ on page 35.

While each iteration of the prototyping process is split into goal, implementation, and evaluation sections, this section serves to do this for the process as a whole and conclude not just the prototyping process, but the conceptual design of the system as a whole, by reevaluating the original design with the insights gained from the prototyping process.

4.7.1 Summary of the Prototyping Process

Before the three iterations of the prototyping process are set up, executed and evaluated, the existing infrastructure, previous work, and the environment of the prototyping process are described. This is done to set the stage for the prototyping process, as well as to highlight the first differences between the suggested architecture and the prototype to be. This is important as the architecture is designed from a deductive standpoint, being a system that is supposed to be generic and not influenced by previous decisions. Confronting this design with the first practical steps is the first step in the validation process. From this the changes extensions and removals of the previous system are derived.

The first iteration (4.4) of the prototyping process is to create the most basic version of the system, possible, to test the feasibility of the system and the environment. This is done by verifying the operative state of the system and the environment, fixing issues with the existing infrastructure. For the basic functionality of the system, a custom container image is created, containing the necessary tools and libraries to host a Notebook server. Using a modified *Ipython* kernel, the Arkouda and Chapel libraries are integrated into the container which automatically connect to the compute nodes when activated. To make the system able to be used on the cluster a deployment and a service are created, to make the system accessible from the outside. The system is then tested by running a simple example, to verify the functionality of the system, some issues are then identified and workarounds are implemented to create the first working version of the system.

As the first iteration has successfully implemented a MVP of the system, second iteration (4.5) aims to improve the system as the previous version did not regard speed and stability as a priority. During this iteration the previously created notebook container is rebuilt in order to optimize the performance of the system. The infrastructure from the previous iteration is then given the same treatment, completely rebuilding the entire container images from scratch in order to reduce their size and to remove version mismatches in the underlying libraries. To measure these improvements the deployment times of the system are benchmarked. Additionally it was attempted to replace the CRI with a faster HPC oriented one, which was not successful due to

kubernetes dropping support for the API specification which the CRI was based on. Even though the CRI could not be replaced, the system was still benchmarked to measure the compute node's performance and to identify bottlenecks in the system. For this purpose a simple benchmarking framework was converted to a deployment capable of running in a kubernetes cluster and to interact with the compute nodes to measure the performance of the system.

The goal of the third iteration (4.6) was to move the system from a simple proof of concept to a system which could not just technically solve HPC problems, but one that is not constrained by consumer level hardware, so its performance can be compared to an equivalent section of a real system. While the previous section had the goal of implementing *Singularity* to make use of its near native performance, it was also hoped to make use of its native support for IB to make the third iteration much easier to implement. But as *Singularity* was not supported the IB was implemented manually using a multitude of different tools and configurations in order to properly provision and integrate the device into the Multi CNI setup, installed to enable the usage. During the integration of this step the prototyping was cut short due to the time constraints of the thesis, leaving the IB integration for future work. But as the flannel CNI is still completely in place the system is still fully functional and can be used to run HPC workloads using the TCP/IP protocol.

4.7.2 Evaluating against the Deductively Designed System

As the prototyping process is now concluded and summarized, it is time to complete the validation of the deductively designed system and to evaluate the prototyping process against the original design; contrasting the theory with practical implementations, which changes where made out of incorrect assumptions and which where made out of necessity, and which are improvements of the general design.

The following section will go through the different aspects defined in the 'Suggested Architecture' on page 35 and evaluate them.

Client:

In the design it was assumed that the client of the user would be a local interface of Jupyter notebook, which would be connected to the server via a web interface. This assumption has been implemented in the prototype and even used during the data analysis of the prototyping process. Beyond having a local interface, such as a local Jupyter notebook or the *VSCode* extension, the system can just as easily be accessed from a web browser, making it highly accessible and flexible in its use. The original design has been validated by the prototyping process.

Language:

In the original design it was decided to make this project Arkouda first, as it is a modern and easy to use way of interacting with the HPC system.

While Arkouda itself is very user friendly and easy to use, the installation, needing to have a precompiled Chapel version, and then manually compiling and importing the Arkouda library, is less so.

While this made the creation of custom containers more complicated, from the perspective of the user, the system is now as easy to use as native python on any jupyter notebook server, making the system very user friendly.

From the perspective of compute performance, the system was as efficient as the underlying infrastructure permitted, being bottlenecked by the networking (see ‘Performance Testing’ on page 54).

There is no reason to discount the original decision to use Arkouda as the primary language of the system.

Cluster:

The original design assumed that the system should be run on a Kubernetes cluster, while this is an integral part of the ability to scale the system, the prototyping has shown that Kubernetes is not build for connecting easily to hardware devices which are standard in HPC environments.

Integrating these devices is a major challenge and when deciding on building a cloud native HPC system, it should be strongly considered wether a Kubernetes cluster is the right choice for the system, or whether the other opportunity discussed in this section should be used. This being the bare the orchestration of bare metal systems.

So while the design inherently relies on the use of a Kubernetes cluster, and therefore it needs to be included in the system, this is a decision that should only be made if the high flexibility and scalability of containers is needed.

Jupyter Server:

The original design discusses the decision between using a simple Jupyter server or a whole JupyterHub setup, the prototyping process has shown that the setup of a single Jupyter server is sufficient for the system, and quite uncomplicated to set up.

Orchestrating a single or a few JupyterHub in oder to manage multiple users would increase the complexity of the system, while the use of a single Jupyter server per user is a much simpler solution, which is sufficient for the current goals of the system.

This decision is validated by the prototyping process.

Compute Services:

As Arkouda was already selected the choice of compute service was obvious, as the Arkouda Client is build to interface with a Chapel backend, through the Arkouda server witch wraps the Chapel library.

While the same limitations exist for the Arkouda server as for the Arkouda client, needing to compile Chapel and Arkouda manually, making an iterative development process more complicated, the experience during runtime is user-friendly and and relatively quick, considering the networking limitations of the system.

The original design has been implemented as described and is a valid choice for the system. But further optimization of the system could be considered to ease and speed up the development process, as many steps of the initial setup are still required to be done manually. A more automated and consolidated setup would be beneficial for both the scalability and the maintainability of the system.

Over all, the original design is validated by the prototyping process, and further potential has been identified.

Resource Provisioning / Scaling:

Original goal of the system in the design was to dynamically scale the system, either on a job by job basis or by scaling the based on sessions.

While the system can be rescaled via a single variable in between sessions, this feature has not yet been implemented deeply enough in the system. It's implementation during later iterations was a significant factor in the decisions made during the earlier stages of the prototyping process, such as setting up the self configuring implementation of IB, the optimization of the container images, and the one notebook per user setup.

While this feature has been deemed important for the system the decision to make all parts scalable from the beginning should be reconsidered. Creating a system that is at first not scalable, but completely functioning with all the relevant features and then optimizing the necessary parts for scalability. Would have resulted in more insights early on in the prototyping process, but would have created more work over all. Under the light of the time constraints of the thesis, the decision to prioritize the scalability in each step component of the system has lead to not reaching the goal of a fully scalable system, in the time frame of the thesis.

Therefore, while the original design is valid, it should have been differently prioritized.

Operating System:

The original design assumed that the system would be run on a Linux based system, which is a necessary and valid assumption, for both Kubernetes and HPC systems. This part is validated by the prototyping process.

A second consideration was whether to use a generic Linux distribution or a specialized one, while the choice of a generic distribution has been sufficient for the prototyping process, choosing a licensed distribution specialized in this sector of Kubernetes would have enabled the access to more specialized support and documentation, which would have speed up the prototyping process significantly. On the other hand the use of a non generic distribution entails the reliance on licensed software, which would have made the system less accessible to the general public. And setting up the entire cluster from a blank system offers a much deeper understanding of the system as a whole, which is invaluable for the development of the system.

Therefore both decisions are equally valid and the choice between them should be made depending on the goals of the developer, for a more flexible and public system a generic distribution would be the better choice, while for a licensed and specialized system will provide benefits for production systems.

This decision is validated by the prototyping process.

Container Runtime:

During the design phase three different container runtimes were picked out as possible candidates on which the system could be run.

On the baseline either CRI-O or *Containerd* were considered with CRI-O being the preferred choice due to its closer accordance with the OCI specification. As an additional option *Singularity* was considered as a runtime for the compute pods to use its performance benefits.

As the previous system was already running on *Containerd* and the benefits of CRI-O were not significant enough to warrant the change, the decision was made to keep the system on *Containerd*. Not invalidating the original proposal but diverting from it out of practicality.

The decision to include *Singularity* as a secondary runtime, on the other hand was inherently misinformed, as is described in the section ‘Container Runtime’ on page 52. Going down this path is not recommended, as either an unsupported version of Kubernetes would have to be used, or the *Singularity* CRI would need to be patched.

In conclusion the original design is validated in regards to the base container runtime, but the decision to include *Singularity* has been completely invalidated.

Container Image:

As touched upon in the compute services section, the container images used were based on the ones provided in the contributions repository of the Arkouda project.

While they have been an excellent starting point, the prototyping process has shown that the images were able to be significantly reduced in size and optimized for the system, which has improved the performance of the system.

One significant change needed to be made in the future is to create a set of IB capable images, as this needs to be set up during compilation time and can not be changed during runtime.

The original design has been validated by the prototyping process and has been improved upon, and still has potential for further improvements.

Network Abstraction:

During the original design the tradeoff common network abstraction layers make between performance and flexibility was highlighted and it was concluded that a multi CNI setup would be an essential part of the system.

Creating two separate network, one for the orchestrating communication, and one to provide the high-speed RDMA capable communication the HPC components require.

It was already recognized that for this setup to work a meta CNI such as *Multus* would be necessary to enable the use of both networks.

While the need for this system has been thoroughly shown during the prototyping process, and the implementation of a dual CNI setup has been proven as the only valid solution; Other solutions such as keeping the original *Flannel* CNI and facilitating the RDMA communication on the container runtime level where considered but have been proven to be infeasible.

While the original proposal on the network abstraction level has been implemented, via the use of a dual CNI setup, the ramifications for the configurations of the underlying hardware were severely underestimated, which will be discussed in more detail in the paragraph "Network Interconnect" of this section.

Overall using a dual CNI setup is a valid solution if the requirements of the system demand a high-speed RDMA network, a containerized environment and a flexible communication infrastructure.

If this is not the case, a bare metal ethernet network, running the workload directly on the system or a fixed network configuration should be considered instead.

Compute Node Hardware:

In the effort to create a generic design, only generic recommendations were made during the design phase, especially as the system, using containerization is largely hardware agnostic.

As the network revealed itself to be the main bottleneck of the system, the compute and memory specification of the system did not impact the performance in a significant negative way. This is demonstrated by the fact that during the performance testing PGAS memory occupation did not exceed 30% of the available space when the 10Gbit network was at its maximum capacity.

But nevertheless, the memory speed and capacity, as well as the CPU performance and NUMA topology of the two socket nodes would most likely be of significant importance after the implementation of a dedicated IB network.

The other aspect discussed, being the capability of respecting non homogeneous systems was important during the building and testing of the network abstraction. Using a hardware scanning daemonsets, to identify available hardware made it easy to segment the deployments to either the IB capable or in capable nodes, by simply stating the need in the configuration.

While not proven during the prototyping process, the assumption that a more compute capable system would be able to handle more workloads still stands. On the other hand it was shown that the use of heterogeneous systems is possible, as special need hardware can be reserved for special workloads if needed, but is fully capable of running pods with on all nodes, independent of the underlying hardware.

Validating the latter and not invalidating the former, part of the original design.

Network Interconnect:

As has previously been referenced, one of the major bottlenecks of the prototype was the network. This was already correctly anticipated during the design phase and the decision to utilize an IB network interconnect was made.

During the design phase it was assumed that the installation of the IB network, would be of equal complexity as any other CNI addon; not trivial but manageable.

But this notion was shown to be incorrect, as the setup of the IB network and the necessary multi layered infrastructure to configure it through Kubernetes was a major challenge took up a significant amount of time during the prototyping process.

While the Kubernetes side of the setup has now been brought to a working state, the configuration of the necessary kernel modules has been cut short due to time constraints of the thesis.

While the reason for the IB interconnect is valid; bringing the performance from acceptable for medium sized workloads, to good performance for large scale HPC workloads. The complexity of this setup was severely misjudged and resulted in a major disruption of the time schedule of the prototyping process, resulting in features not being implemented and the original design not being able to be fully validated.

One of these features is the dynamic scaling of the system, which would have lead to the investigation of the need for topological awareness, which has also been discussed in this section of the design.

While the need for a high-speed network interconnect has manifested as expected, the applied solution was not feasible in the time frame of the thesis. Instead it is recommended to either circumvent the need for a IB integration, or to allot this step the necessary resources to be implemented correctly. This could be done by making use of enterprise support, hiring a specialist or allotting more time to the prototyping process. Regarding the network topology, no evaluation can be made, as neither a state in which the need could be evaluated nor a test setup has been reached.

Conclusion

Circling back to the original goal of the Conceptual Deductive Design, as defined in ‘Goals’ on page 34:

“The designed system should provide the user with a easy to use interface, capable of interactively exploring HPC scale data. The system should be able to run on cloud infrastructure, be able to allocate the necessary resources, as well as scheduling and executing the computation within a reasonable time frame.”

This prototyping process has shown that this goal is technically feasible, and has largely been achieved. While two of the design goals where based on incorrect assumptions, most have either been validated or even expanded upon during the prototyping process.

These divergences between a theoretical design and the realities of an actual implementation, have now been either been corrected, put in a more appropriate context or have been proven to be based on incorrect assumptions.

Overall the design as well as the prototyping process has been a success, as the central functionality of an interactive system was successfully implemented, but most importantly valuable insights have been gained about the prototyping process as well as the cavitates of a deductively designed system. Also a useful system has been created which was used to run the data analysis. While there is still work to be done and much long term potential for the project as a whole, what has been done builds a solid foundation and many paths forward.

5 Summary and Outlook

With each section of this thesis now complete it is time to take a step back and consider the work from a broader perspective, reflecting on what was achieved, how it was achieved, and what this means for the future.

First the work done in this thesis will be summarized on a high level. Then the questions raised in the introduction will be answered and the contributions of this thesis will be outlined. Afterwards the work done, and the choices made during this project will be reflect on. Finally, the outlook will be given, discussing the steps following the completion of this thesis and the future work to be done.

5.1 Summary

Introduction: This research project was inspired by the need for a user-friendly way to interact with HPC systems, which themselves should be easy and quick to create. The need for an interactive environment being driven directly by the shortcomings of an earlier project. This being a pipeline driven GitOps system which automatically schedules HPC workloads on a Kubernetes cluster. As the jobs are scheduled by the pipeline, which is being build by committing to a git repository, this system lacked the interactivity needed to develop the pipeline code. For this reason the goal for this project was to create a user-friendly way, to enable the user to interact with the HPC system in a more direct Based on this problem the problem statement, research questions and the contributions of this thesis were defined.

Methodology: To solve this problem which exists in the intersection the HPC and CC worlds, which are both highly complex and specialized fields, a combination of methodologies was chosen which on one hand allows for a deep understanding of the theoretical background and on the other hand contrasts this with a practical implementation.

For this reason a conceptual deductive Analysis was chosen, in order to derive a design from the theoretical background and the requirements from the problem statement. This is done by starting as close to the first principles of the technologies involved as possible and then working towards the higher levels required for the implementation.

To prove or invalidate this hypothesis of a system design it was decided to implement the as a prototype in a series of iterations and then evaluate the results of these iterations against the design and the theoretical background.

Theoretical Work - Cloud: The theoretical work started with the foundations of the CC world. Beginning with the basics of process and systems isolation and how they are achieved in practice. Starting at KVM and their ability to access some of the hardware directly, did not only provide insight into the predecessor of containers but also into the features which became important later during the use of SRIOV devices. The inner workings of containers and their implementation in the Linux kernel were addressed next, as well as how container runtime and their images work. Bringing valuable insight especially useful during the second stage of the prototype.

Expanding from containerization to the concepts of Kubernetes, which first were explained on an architectural level, and then on a networking level. Information which served as the basis for each of the iterations. The final section, explaining the scheduling and topology considerations of Kubernetes were intended as basis for the fourth iteration and included in the design of the system, but were not implemented due to time constraints.

Theoretical Work - HPC: Having established the theoretical background of the CC world, the background of the HPC world was addressed next. Once again starting with the very basics, the distributed processing and relevant hardware infrastructure, in order to establish the context for the relevance of Memory architectures. As the problems of HPC are much closer to the hardware than the typical CC applications, this knowledge is essential to understand the way these systems allocate and access memory as this builds the foundation of more complex memory architectures like the PGAS model relying on a NUMA like architecture of over the network memory access. Leading into the explanation of network interconnects and fabrics and the place they hold through the RDMA and MPI protocols. From there the complex network topologies, common in the HPC world to reduce cost and increase performance were explained. Finally these concepts are tied together with an overview of the most relevant software stack components from the OS to *Workload Management Systems* are presented.

Theoretical Work - Exploratory Computing: Last but not least some of the theoretical background of the current state of the art in exploratory computing was presented. A field which is highly influenced by the computational notebook paradigm, which is a way to combine code, documentation and data visualization in a single document, which has become very popular in the data science community and many other fields where exploratory computing is done.

Experimental Work - Design: With the theoretical background established the design of the system could be derived. Starting with the problem statement from the introduction the goals for the design were derived. These goals were then used to inform the requirements for the system and a design was constructed based on these requirements. The design itself was broken into each of the segments discussed in the theoretical chapters and at each step a design decision was made based on the information gathered in the theoretical chapters.

Experimental Work - Artifact Creation: Before this design was implemented as a prototype, the theoretically designed system which was targeted to be a general design with as little influence from previous work as possible, was mapped to the practicalities of the environment the prototype

was to be implemented in, discussing the previous work. the existing infrastructure and how this translates to the design of the system to be implemented.

Experimental Work - Iterative Development: With the actual environment established the iterative development of the prototype was started. Each iteration setting a goal based on the design and the previous work, describing the implementation of these goals, mentioning mayor problems and solutions found during the implementation and evaluating the results of each step. The fist iteration started of by implementing a MVP of the system, integrating the all the software components needed for the system to work. During the second iteration the all containers of the system where rebuild from scratch to create a more performant and responsive system, and converting the benchmarking system of Arkouda so it could be used to evaluate the performance of Kubernetes cluster. Additionally it was attempted to integrate the CRI of Singularity, which only then revealed itself as not feasible. The third iteration aimed to replace the networking layer with a fully automated multi CNI setup, enabling the worker nodes to communicate more efficiently with the IB network, in order to eliminate the bottleneck discovered in the second iteration. This last iteration, while in the final stages could not be completed due to time constraints. The topics of dynamics autoscaling and cluster topography, while in the original design, where not implemented due the same time constraints.

Experimental Work - Evaluation: With the development process concluded, the final step of the deductive design process was to contrast the deductive design with the discoveries made during the experimental work. This was done by going through each of the design decisions made in the theoretical design and evaluating them separately against the results of the experimental work. Validating, correcting or invalidating each of the design decisions based on the new information gathered during the prototyping process and determining which changes made during the prototyping process would be necessary to an unrelated implementation and which where only necessary due to the specific constraints of the prototype.

5.1.1 Contributions

With the work on this thesis now concluded, the contribution of this thesis can be summarized and the research questions answered.

Research Questions

First of lets summarize the answer this thesis has given to the research questions:

- **RQ1:** *How can the system be extended to allow for a real-time explorative workflow, allowing for an interactive exploration of HPC scale data?*
- **Answer:** Within this thesis an interactive interface for an Arkouda based Compute Cluster was designed and created as an artifact. By way of confirming and correcting the original deductive design with the knowledge gained during an experimental implementation, a new design was created which describes how such a system can be implemented in detail. The main components being a computational notebook interfacing with a worker fleet of Arkouda workers running on a Kubernetes cluster, communicating over PGAS and if possible utilizing RDMA protocols.
- **RQ2:** *How can such a system be designed to combine the performance of HPC with the flexibility and convenience of CC systems?*
- **Answer:** During the theoretical and experimental work the set of design decisions and tradeoffs needed to move towards such a system where described. The most significant factors being that while containerized systems have near native performance in computation, they suffer form a highly complex stack. In order to reach the performance of HPC systems the network stack needs to be replaced with a more performant one, creating a highly complex system in order to be able to use the HPC interconnect in a flexible and convenient way. The main tradeoffs of such a system being the complexity of the system and the performance of it. For more details on the design decisions and tradeoffs necessary see the evaluation section.

Artifacts

The direct contributions to the larger field of HPC and CC are as follows:

- **C1:** *A deductive analysis of the three intersecting fields of HPC, CC and EC has been created and experimentally validated.*
- **C2:** *A design for a user friendly and scalable HPC environment on kubernetes has been created and experimentally validated.*

- **C3:** *A tangible artifact in form of an integration of Arkouda, Jupyter and Kubernetes has been created and made available as a software project.*

The work done in this thesis is a step towards a user friendly and scalable HPC environment over all. As this system integrates with the Pachykouda project, a significant progression towards an easy to use and scale HPC environment has been made. Allowing the a user friendly development of pipelines, directly developed on the same system as they will be run on, and can be directly committed to the pipeline repository local to the system, also using the same data source and storage as used by the pachyderm platform.

Some of the optimization of the system, like the optimization of the images will also be useful for the Pachykouda project but might also be useful upstream in the Arkouda project.

5.2 Reflection

5.2.1 Methodology

The dual methodology of combining the CDA with RAD has proven to be a good choice for this project. While the CDA provided a deep understanding of the theoretical background and an academic approach to the problem, the RAD on the other hand allowed for a fast and engineering based approach to the problem, of creating an experimental system. Both in conjunction provided a very useful synergy between them, making the project both deep and practical at the same time. But the practical challenges encountered, particularly in integrating InfiniBand within Kubernetes, also show that creating and estimating the complexity of such a system is not early done from literature alone. The need to crate the theoretical underpinning before the practicalities of the system can be understood, makes it easy to research and describe topics which will only be tangentially relevant to system eventually created.

5.2.2 Execution

The execution of the project was done by creating a theoretical foundation, a design and then implementing this design as a prototype. During the theoretical work a large focus where put on the theoretical underpinnings of the highly complex distribute systems and the EC where reduced to a short description of the computational notebook paradigm. The same can be said for the practical implementation, where the focus was on the integration of HPC paradigms into a CC system, while low hanging fruit of the user facing aspects of the system where not explored in depth. Integrations which could have provided important usability features, like a direct integration with the version control system or the available block storage of the existing system in order to move away from a totally ephemeral environment where not explored, even though the infrastructure was already in place to do so.

5.2.3 Results

In the end the crated artifact is fairly complete and fully usable. The system is able to run Arkouda code on a Kubernetes cluster, with a Jupyter notebook interface, and has been used personally to do data analysis on small data analysis tasks, by the author out of convenience. So while the sudden end of the project due to time constraints is unfortunate especially as the last iteration was only one or two days away from completion, the results of the project are still a success.

5.3 Outlook

As the project itself has shown much potential and a solid foundation has been laid, there are many ways to continue the work done in this thesis. The most immediate step would be to offer the improvements of the components created in this thesis to the Arkouda community.

5.3.1 Future Work

The most immediate steps which still needs to be worked on is to complete the last iteration of the project. As the work is mostly done and the integration of the InfiniBand network stack has been of intersecting to the Arkouda community before. Based on this system the performance comparison between the baseline established during the second iteration and the new system can be made, to see if the performance of the system has improved as expected.

In regards to iterations not started yet, the dynamic autoscaling of the system would be a logical and interesting next step. Both as a good way to reduce cost and increase the peak performance of the system, but also as a way to further explore the Kubernetes system and its capabilities out of personal interest.

Of a high academic as well as practical and personal interest would be the use of an externalized global memory bank for the Arkouda workers to use as a collaborative memory space. A solution like a FAM system has the potential to increase the performance beyond a fully node based PGAS system.

Additionally this project will be continued in the context of the Hewlett Packard Labs internships, where aspects such as an optimized memory allocation strategy of the Arkouda workers will be explored by fellow students furthering the research and impact of this project.

5.3.2 Conclusion

In conclusion there while the results of this project where a success, there is still much that can be explored and investigated in the field of HPC and CC systems.

Statement of Integrity

Hereby I solemnly declare:

1. that this Bachelor Thesis titled "*Interactive Converged HPC: An Arkouda-Driven Approach for Exploratory Computing in Cloud Native Environments* " is entirely the product of my own scholarly work, unless otherwise indicated in the text or references, or acknowledged below;
2. I have indicated the thoughts adopted directly or indirectly from other sources at the appropriate places within the document;
3. this Bachelor Thesis has not been submitted either in whole or part, for a degree at this or any other university or institution;
4. I have not published this Bachelor Thesis in the past;
5. the printed version is equivalent to the submitted electronic one.

I am aware that a dishonest declaration will entail legal consequences.

Stuttgart, May 7, 2024



Jon Eckerth

List of terms

- argsort** Argsort is a function that returns the indices that would sort an array. It is a way to get the sorted indices of an array without sorting the array itself.
55, 56
- bare metal** Bare metal refers to a computer system that is running directly on the hardware with as few layers of abstraction as possible, trying to get a maximum performance out of the system by avoiding the overhead.
20
- batch system** A batch system is a type of job scheduler that manages and schedules on off batch jobs. It is used to run large-scale, long-running, and resource-intensive jobs.
12, 13
- bridge** A bridge is a network device that connects two or more network segments together on an ISO/OSI layer 2. It is used to forward traffic between the segments.
16
- bus** A bus is a communication system that transfers data between components inside a computer or between computers. It is a collection of wires that carry data, address, and control signals.
22
- cordoned** In the context of Kubernetes, cordoned is a command that marks a node as unschedulable. This means that no new pods will be scheduled on the node.
60
- daemon** A daemon is a background process that runs on a system. It is a long-running process that performs a specific task.
11, 31
- distribution** A distribution is a version of the Linux operating system that is based on the Linux kernel and includes a collection of software applications. There are many different distributions of Linux, each with its own set of features and characteristics.
38
- drained** In the context of Kubernetes, drained is a command that evicts all the pods from a node. This is done to prepare the node for maintenance or to remove it from the cluster.
60

EPT Extended Page Tables is a feature of the Intel VT-x virtualization technology. It allows the virtual machine to use the physical memory directly.

6

FPGA A FPGA is an integrated circuit device or programmable platform that can be programmed in the field after being manufactured, providing electronic product manufacturers with additional design flexibility. Unlike application-specific chips, FPGAs allow engineers to make changes very late in the design cycle and even upgrade products with new functionality after manufacture.

21

GitOps Git Operations is a way to do Continuous Deployment. Its core idea is having a Git repository that always contains declarative descriptions of the infrastructure currently desired in the production environment.

1, 2, 42, 74

guest In the context of virtualization, a guest is anything that is any operating system that is running in a virtual machine. on a host system.

6

High Performance Computing High Performance Computing is the practice of going beyond the performance a single system can provide, by coordinating multiple systems to work together in parallel. The term HPC is used to describe the use of supercomputers and parallel processing techniques for solving complex computational problems. The goal of HPC is to provide the necessary computational power to solve problems that are too large or too complex for a single computer to handle. HPC is used in a wide range of applications, including scientific research, engineering, weather forecasting, and financial modeling.

20

host In the context of virtualization, a host is the physical machine that runs the virtualization software.

6, 7, 9, 10, 12, 82

ISO/OSI model The International Organization for Standardization/Open Systems Interconnection model is a conceptual model that standardizes the functions of a telecommunication or computing system into seven abstraction layers.

30

key-value store A key-value store is a type of NoSQL database that uses a simple key-value method to store data. It is a simple way to store and retrieve data.

13

microservice A microservice is a small, independent, and loosely coupled service that is part of a larger application. It is designed to be scalable, resilient, and easy to maintain. They follow the single responsibility principle and if one service fails irrecoverably, it does not affect the entire application. 13

Monkey Patching Monkey patching is a technique used to modify or extend the behavior of a program at runtime. It is a way to change the behavior of a program without changing its source code.
46, 49, 53

NAT Network Address Translation is a method of remapping one IP address space into another by modifying network address information in the IP header of packets while they are in transit across a traffic routing device. This is used to use a single IP address for multiple devices.
15

Node In the context of Clusters a node is a single machine in a cluster. It can be a physical machine or a virtual machine.
12

patching In the context of networking, patching is the process of connecting two or more network segments together on an ISO/OSI layer 1 or 2.
16

registry In the context of containerization, a registry is a storage and content delivery system that holds container images. 44, 46

root The **root directory** is the top-level directory of the file system hierarchy. It is the first directory in the file system and contains all other directories and files. To access the root file system, the user must have root privileges.
8, 83

root The **root user** is the administrative user in a Linux system also known as the superuser or just root. The root user has full control over the system and can perform any action on the system. It is named after the root file system.
8, 10, 11, 83

rootless Rootless is a security feature that allows a user to run containers without root privileges. It is a way to run containers in a more secure way by restricting the capabilities of the container.
40

Single source of truth Single source of truth is a concept that states that every piece of data should have a single, unambiguous, authoritative representation within a system.

13

virtio A virtualization standard for network and disk device drivers. It is a paravirtualization framework that allows the virtual machine to communicate with the host.

6

VT-d Intel's virtualization technology for directed I/O. It allows the virtual machine to access the physical hardware directly.

6

zombies In the context of operating systems, a zombie is a process that has completed execution but still has an entry in the process table. It is a process that has finished its execution but has not been removed from the system.

15

List of literature

- Abhishek Verma/Luis Pedrosa/Madhukar Korupolu/John Wilkes/Eric Tune/David Oppenheimer (2015)**: ‘Large-Scale Cluster Management at Google with Borg’. <https://www.youtube.com/watch?v=7MwxA4Fj2l4> [retrieval: 19.3.2024].
- Almasi, G. (2011)**: ‘PGAS (Partitioned Global Address Space) Languages’. In: *Encyclopedia of Parallel Computing*. Ed. by David Padua. Boston, MA: Springer US, pp. 1539–1545. ISBN: 978-0-387-09766-4. 10.1007/978-0-387-09766-4_210. https://doi.org/10.1007/978-0-387-09766-4_210 [retrieval: 30.3.2024].
- ARCHER UK National Supercomputing Service, director (2016)**: *Lecture 7: HPC Architectures*. <https://www.youtube.com/watch?v=NIqrWds0cy0> [retrieval: 26.3.2024].
- Arkouda - docs (2024a)**: SegArrays in Arkouda - Arkouda Documentation. <https://bears-r-us.github.io/arkouda/usage/segarray.html#performance> [retrieval: 2.5.2024].
- **(2024b)**: Startup - Arkouda Documentation. <https://bears-r-us.github.io/arkouda/usage/startup.html> [retrieval: 10.4.2024].
- Arkouda - repo (2024a)**: Arkouda Repo - Benchmarks. GitHub. https://github.com/Bears-R-Us/arkouda/blob/master/benchmarks/graph_infra/emitted-code-size.perfkeys [retrieval: 25.4.2024].
- **(2024b)**: Arkouda-Contrib/Arkouda-Docker at Main · Bears-R-Us/Arkouda-Contrib. GitHub. <https://github.com/Bears-R-Us/arkouda-contrib/tree/main/arkouda-docker> [retrieval: 14.4.2024].
- **(2024c)**: *Arkouda/Pydoc/Setup/LINUX_INSTALL.Md at Master · Bears-R-Us/Arkouda*. https://github.com/Bears-R-Us/arkouda/blob/master/pydoc/setup/LINUX_INSTALL.md [retrieval: 18.4.2024].
- **(2024d)**: *Bears-R-Us/ArkoudaNotebooks: Place for Notebooks and Example Uses of the Arkouda Software Package*. <https://github.com/Bears-R-Us/ArkoudaNotebooks> [retrieval: 19.4.2024].
- **(2024e)**: Strings in Arkouda - Arkouda Documentation. <https://bears-r-us.github.io/arkouda/usage/strings.html> [retrieval: 2.5.2024].
- Armbrust, M./Fox, A./Griffith, R./Joseph, A. D./Katz, R./Konwinski, A./Lee, G./Patterson, D./Rabkin, A./Stoica, I./Zaharia, M. (w.y.)**: Above the Clouds: A Berkeley View of Cloud Computing. In.
- AWS (2024a)**: Open-Source-Kubernetes-Verteilung – Amazon-EKS-Verteilung – Amazon Web Services. Amazon Web Services, Inc. <https://aws.amazon.com/de/eks/eks-distro/> [retrieval: 12.4.2024].
- **(2024b)**: Type 1 vs Type 2 Hypervisors - Difference Between Hypervisor Types - AWS. Amazon Web Services, Inc. <https://aws.amazon.com/compare/the-difference-between-type-1-and-type-2-hypervisors/> [retrieval: 13.2.2024].
- Barham, P./Dragovic, B./Fraser, K./Hand, S./Harris, T./Ho, A./Neugebauer, R./Pratt, I./Warfield, A. (2003)**: Xen and the Art of Virtualization. In: *SIGOPS Oper. Syst. Rev.* 37.5, pp. 164–177. ISSN: 0163-5980. 10.1145/1165389.945462. <https://dl.acm.org/doi/10.1145/1165389.945462> [retrieval: 13.2.2024].

- Belyaev, N./Krasnopevtsev, D./Konoplich, R./Velikhov, V./Klimentov, A. (2017):** High Performance Computing System in the Framework of the Higgs Boson Studies. ATL-COM-SOFT-2017-089. <https://cds.cern.ch/record/2293317> [retrieval: 21.3.2024].
- Bernstein, D. (2014):** Containers and Cloud: From LXC to Docker to Kubernetes. In: *IEEE Cloud Comput.* 1.3, pp. 81–84. ISSN: 2325-6095, 2372-2568. 10.1109/MCC.2014.51. <https://ieeexplore.ieee.org/document/7036275/> [retrieval: 14.2.2024].
- Besta, M./Domke, J./Schneider, M./Konieczny, M./Girolamo, S. D./Schneider, T./Singla, A./Hoefler, T. (2021a):** High-Performance Routing With Multipathing and Path Diversity in Ethernet and HPC Networks. In: *IEEE Trans. Parallel Distrib. Syst.* 32.4, pp. 943–959. ISSN: 1045-9219, 1558-2183, 2161-9883. 10.1109/TPDS.2020.3035761. <https://ieeexplore.ieee.org/document/9248644/> [retrieval: 31.3.2024].
- **(2021b):** High-Performance Routing With Multipathing and Path Diversity in Ethernet and HPC Networks. In: *IEEE Transactions on Parallel and Distributed Systems* 32.4, pp. 943–959. ISSN: 1558-2183. 10.1109/TPDS.2020.3035761. <https://ieeexplore.ieee.org/abstract/document/9248644> [retrieval: 2.4.2024].
- Betz, M. (2022):** Understanding Kubernetes Networking: Services. Google Cloud - Community. <https://medium.com/google-cloud/understanding-kubernetes-networking-services-f0cb48e4cc82> [retrieval: 19.3.2024].
- Boehm, B. W. (1988):** A Spiral Model of Software Development and Enhancement. In: *Computer* 21.5, pp. 61–72. ISSN: 0018-9162. 10.1109/2.59. <http://ieeexplore.ieee.org/document/59/> [retrieval: 18.10.2023].
- Burt, J. (2024):** Microsoft’s Azure Linux Distro Is Now Generally Available. https://www.the-register.com/2023/05/26/microsoft_azure_linux_container/ [retrieval: 12.4.2024].
- Canonical (2024a):** Get Ubuntu Server | Download. Ubuntu. <https://ubuntu.com/download/server> [retrieval: 12.4.2024].
- **(2024b):** Metal as a Service. MAAS. <https://maas.io/> [retrieval: 10.4.2024].
- Capuzzo-Dolcetta, R./Spera, M./Punzo, D. (2013):** A Fully Parallel, High Precision, N -Body Code Running on Hybrid Computing Platforms. In: *Journal of Computational Physics* 236, pp. 580–593. ISSN: 0021-9991. 10.1016/j.jcp.2012.11.013. <https://www.sciencedirect.com/science/article/pii/S0021999112006900> [retrieval: 21.3.2024].
- Chamberlain, B. (w.y.):** A PROGRAMMING LANGUAGE FOR PRODUCTIVE PARALLEL COMPUTING FROM LAPTOPS TO SUPERCOMPUTERS. In.
- Chapel (2024a):** Chapel’s Profile | Docker Hub. <https://hub.docker.com/u/chapel> [retrieval: 14.4.2024].
- **(2024b):** Chapel’s Profile | Docker Hub. <https://hub.docker.com/u/chapel> [retrieval: 18.4.2024].
- chapel-lang.org (2024a):** Building Chapel — Chapel Documentation 2.0. <https://chapel-lang.org/docs/usingchapel/building.html> [retrieval: 18.4.2024].
- **(2024b):** Chapel: Overview. <https://chapel-lang.org/overview.html> [retrieval: 6.4.2024].
- **(2024c):** Multilocale Chapel Execution — Chapel Documentation 2.0. <https://chapel-lang.org/docs/usingchapel/multilocale.html#what-is-this-real-program> [retrieval: 6.4.2024].

- CNCF [Cloud Native Computing Foundation] (2024)**: CNCF Landscape. <https://landscape.cncf.io/?view-mode=grid&group=projects-and-products&category=Runtime> [retrieval: 12.4.2024].
- crio (2024)**: *Cri-o/Cri-o*. CRI-O. <https://github.com/cri-o/cri-o> [retrieval: 12.4.2024].
- Dartmouth College (2024)**: MPI Collective Communication Modes. https://www.dartmouth.edu/~rc/classes/intro_mpi/mpi_comm_modes2.html#top [retrieval: 6.4.2024].
- Datadog, director (2023)**: *Container Security Fundamentals - Linux Namespaces (Part 1): The Mount Namespace*. <https://www.youtube.com/watch?v=BwI89OnYm-4> [retrieval: 16.2.2024].
- De Sensi, D./Di Girolamo, S./McMahon, K. H./Roweth, D./Hoeffler, T. (2020)**: An In-Depth Analysis of the Slingshot Interconnect. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14. 10.1109/SC41405.2020.00039. <https://ieeexplore.ieee.org/abstract/document/9355230> [retrieval: 16.4.2024].
- De Wael, M./Marr, S./De Fraine, B./Van Cutsem, T./De Meuter, W. (2015)**: Partitioned Global Address Space Languages. In: *ACM Comput. Surv.* 47.4, pp. 1–27. ISSN: 0360-0300, 1557-7341. 10.1145/2716320. <https://dl.acm.org/doi/10.1145/2716320> [retrieval: 31.3.2024].
- Debab, R./Hidouci, W. K. (2021)**: Containers Runtimes War: A Comparative Study. In: *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 2*. Ed. by Kohei Arai/Supriya Kapoor/Rahul Bhatia. Advances in Intelligent Systems and Computing. Cham: Springer International Publishing, pp. 135–161. ISBN: 978-3-030-63089-8. 10.1007/978-3-030-63089-8_9.
- debian (2024)**: Debian – The Universal Operating System. <https://www.debian.org/> [retrieval: 12.4.2024].
- Dewan, G./Jenkins, L. (2020)**: Paving the Way for Distributed Non-Blocking Algorithms and Data Structures in the Partitioned Global Address Space. In: *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 659–666. 10.1109/IPDPSW50202.2020.00111. arXiv: 2002.03068 [cs]. <http://arxiv.org/abs/2002.03068> [retrieval: 31.3.2024].
- Di Blas, N./Mazuran, M./Paolini, P./Quintarelli, E./Tanca, L. (2014)**: Exploratory Computing: A Draft Manifesto. In: *2014 International Conference on Data Science and Advanced Analytics (DSAA)*. 2014 International Conference on Data Science and Advanced Analytics (DSAA). Shanghai, China: IEEE, pp. 577–580. ISBN: 978-1-4799-6991-3. 10.1109/DSAA.2014.7058129. <http://ieeexplore.ieee.org/document/7058129/> [retrieval: 8.4.2024].
- Docker, director (2015)**: *Cgroups, Namespaces, and beyond: What Are Containers Made From?* <https://www.youtube.com/watch?v=sK5i-N34im8> [retrieval: 16.2.2024].
- Dongarra, J./Sterling, T./Simon, H./Strohmaier, E. (2005)**: High-Performance Computing: Clusters, Constellations, MPPs, and Future Directions. In: *Computing in Science & Engineering* 7.2, pp. 51–59. ISSN: 1558-366X. 10.1109/MCSE.2005.34. <https://ieeexplore.ieee.org/abstract/document/1401802> [retrieval: 25.3.2024].

- Dongarra, Jack/Sterling, T./Simon, H./Strohmaier, E. (2005):** HIGH-PERFORMANCE COMPUTING: CLUSTERS, CONSTELLATIONS, MPPS, AND FUTURE DIRECTIONS. In.
- Dongarra, J. J./Luszczek, P./Petitet, A. (2003):** The LINPACK Benchmark: Past, Present and Future. In: *Concurrency and Computation: Practice and Experience* 15.9, pp. 803–820. ISSN: 1532-0634. 10.1002/cpe.728. <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.728> [retrieval: 21.3.2024].
- Eckerth, J. (2023):** "Development and Demonstration of a Proof-of-Concept for the Integration of Programming Frameworks for High Performance Computing into a Container-Based Workflow Orchestrator." In.
- Eickhoff, L. (w.y.):** A Topology-Aware Workload Analyzer for High-Performance Computing Applications.
- Enthought (2020):** ‘Arkouda: Terascale Data Science at Interactive Rates |SciPy 2020| Ben Albrecht’. <https://www.youtube.com/watch?v=Rqak62zNOxE> [retrieval: 9.4.2024].
- Erceg, B. (2022):** Yet Another Brief History of Container(d). Medium. <https://erzeghi.medium.com/yet-another-brief-history-of-container-d-2962eac9679e> [retrieval: 14.2.2024].
- Flannel (2024):** *Flannel-Io/Flannel*. flannel-io. <https://github.com/flannel-io/flannel> [retrieval: 16.4.2024].
- Flannel - Repo (2024):** *Flannel/Documentation/Backends.Md at Master · Flannel-Io/Flannel*. <https://github.com/flannel-io/flannel/blob/master/Documentation/backends.md> [retrieval: 16.4.2024].
- Fortune business insights (2024):** High Performance Computing Market Size & Trends [2030], p. 170. <https://www.fortunebusinessinsights.com/industry-reports/high-performance-computing-hpc-and-high-performance-data-analytics-hpda-market-100636> [retrieval: 21.3.2024].
- Gandy, A. (w.y.):** Advanced Computational Methods in Statistics: Lecture 1 Monte Carlo Simulation & Parallel Computing. In.
- Garden (2024):** Container Orchestrator Usage Europe and U.S. 2021. Statista. <https://www.statista.com/statistics/1238202/kubernetes-container-orchestrator-europe-us/> [retrieval: 18.2.2024].
- Gomaa, H. (1983):** The Impact of Rapid Prototyping on Specifying User Requirements. In: *SIGSOFT Softw. Eng. Notes* 8.2, pp. 17–27. ISSN: 0163-5948. 10.1145/1005959.1005964. <https://doi.org/10.1145/1005959.1005964> [retrieval: 3.11.2023].
- Google (2024):** Node Images | Google Kubernetes Engine (GKE). Google Cloud. <https://cloud.google.com/kubernetes-engine/docs/concepts/node-images> [retrieval: 12.4.2024].
- Goto, Y. (2011):** Kernel-Based Virtual Machine Technology. In: *Fujitsu Scientific and Technical Journal* 47.3, pp. 362–368. <https://www.fujitsu.com/global/documents/about/resources/publications/fstj/archives/vol47-3/paper18.pdf> [retrieval: 13.2.2024].
- Granger, B. E./Perez, F. (2021):** Jupyter: Thinking and Storytelling With Code and Data. In: *Comput. Sci. Eng.* 23.2, pp. 7–14. ISSN: 1521-9615, 1558-366X. 10.1109/MCSE.2021.3059263. <https://ieeexplore.ieee.org/document/9387490/> [retrieval: 8.4.2024].

- Gropp, W./Lusk, E./Skjellum, A. (1999):** Using MPI: Portable Programming with the Message Passing Interface. In: 1.
- Hennessy, J. L./Patterson, D. A. (2011):** Computer Architecture: A Quantitative Approach. Elsevier. 857 pp. ISBN: 978-0-12-383873-5.
- Hermanns, M. (w.y.):** Parallel Programming in Fortran 95 Using OpenMP. In.
- HLRS High Performance Computing Center Stuttgart (2024):** HLRS High Performance Computing Center Stuttgart: Hawk Helps Create First Image of Black Hole at Center of Milky Way. <https://www.hlrs.de/news/detail/hawk-helps-create-first-image-of-black-hole-at-center-of-milky-way> [retrieval: 21.3.2024].
- hokiegeek2 (2024):** Commits · Bears-R-Us/Arkouda. <https://github.com/Bears-R-Us/arkouda/issues/3089> [retrieval: 19.4.2024].
- Hovestadt, M./Kao, O./Keller, A./Streit, A. (2003):** ‘Scheduling in HPC Resource Management Systems: Queuing vs. Planning’. In: *Job Scheduling Strategies for Parallel Processing*. Ed. by Dror Feitelson/Larry Rudolph/Uwe Schwiegelshohn. Red. by Gerhard Goos/Juris Hartmanis/Jan Van Leeuwen. Vol. 2862. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 1–20. ISBN: 978-3-540-20405-3 978-3-540-39727-4. 10.1007/10968987_1. http://link.springer.com/10.1007/10968987_1 [retrieval: 7.4.2024].
- Howard (2023):** InfiniBand vs. Ethernet: What Are They? Knowledge. <https://community.fs.com/article/infiniband-vs-ethernet-which-is-right-for-your-data-center-network.html> [retrieval: 1.4.2024].
- HPC Wiki (2024a):** Parallel Programming. https://hpc-wiki.info/hpc/Parallel_Programming [retrieval: 28.3.2024].
- **(2024b):** Schedulers or "How-To-Run-Applications-on-a-supercomputer". https://hpc-wiki.info/hpc/Getting_Started#Schedulers_or_.22How-To-Run-Applications-on-a-supercomputer.22 [retrieval: 7.4.2024].
- Innes, B. (2024):** OKD.Io. <https://openshift-cs.github.io/okd.io/index.html/> [retrieval: 12.4.2024].
- Intel (2010):** Understanding iWARP: Delivering Low Latency to Ethernet.
- Jackson, K./Bunch, C. (2013):** OpenStack Cloud Computing Cookbook: Over 100 Recipes to Successfully Set up Manage Your OpenStack Cloud Environments with Complete Coverage of Nova, Swift, Keystone, Glance, Horizon, Neutron, and Cinder. 2. Aufl. Quick Answers to Common Problems. Birmingham: Packt Publ. 377 pp. ISBN: 978-1-78216-758-7.
- Jin, H./Jespersen, D./Mehrotra, P./Biswas, R./Huang, L./Chapman, B. (2011):** High Performance Computing Using MPI and OpenMP on Multi-Core Parallel Systems. In: *Parallel Computing*. Emerging Programming Paradigms for Large-Scale Scientific Computing 37.9, pp. 562–575. ISSN: 0167-8191. 10.1016/j.parco.2011.02.002. <https://www.sciencedirect.com/science/article/pii/S0167819111000159> [retrieval: 31.3.2024].
- Jon Eckerth (2024a):** Bachelor Thesis Project Repo - Benchmark Data. <https://github.com/Jo-Eck/BachelorsThesis/tree/main/Project/data/comutebenchmark> [retrieval: 2.5.2024].
- **(2024b):** Bachelor Thesis Project Repo - Data Analysis. GitHub. <https://github.com/Jo-Eck/BachelorsThesis/tree/main/Project/data/pod-startup-time> [retrieval: 24.4.2024].

- Jon Eckerth (2024c):** Bachelor Thesis Project Repo - Dataset. <https://github.com/Jo-Eck/BachelorsThesis/tree/main/Project/data/pod-startup-time> [retrieval: 24.4.2024].
- **(2024d):** Bachelor Thesis Project Repo - Dataset Large-Image. <https://github.com/Jo-Eck/BachelorsThesis/blob/main/Project/data/pod-startup-time/large-image-1713798943.json> [retrieval: 24.4.2024].
 - **(2024e):** Bachelor Thesis Project Repo - Dataset Small-Image. <https://github.com/Jo-Eck/BachelorsThesis/blob/main/Project/data/pod-startup-time/small-image-1713744112.json> [retrieval: 24.4.2024].
 - **(2024f):** Bachelor Thesis Project Repo - Deployspeedtest. GitHub. <https://github.com/Jo-Eck/BachelorsThesis/blob/main/Project/cluster/testing/deployspeed/test.sh> [retrieval: 24.4.2024].
 - **(2024g):** *Bachelor Thesis Project Repo - Workaround - Argsort*. <https://github.com/Jo-Eck/BachelorsThesis/blob/main/Project/cluster/notebook-server/jupyter-autorun/10-workaround-argsort.py> [retrieval: 24.4.2024].
 - **(2024h):** BachelorsThesis/Project at Main · Jo-Eck/BachelorsThesis. <https://github.com/Jo-Eck/BachelorsThesis/tree/main/Project> [retrieval: 3.5.2024].
 - **(2024i):** Fixing Erroneous RBAC Name Variable · Jo-Eck/ProjectPaper-2@164f91f. <https://github.com/Jo-Eck/ProjectPaper-2/commit/164f91f0386906d4cf426d1502ed9a4e96dd8f1e> [retrieval: 17.4.2024].
- Jordan Webb (2024):** LXC and LXD: A Different Container Story [LWN.Net]. LWN. <https://lwn.net/Articles/907613/> [retrieval: 17.2.2024].
- Jupyter - Docs (2024a):** Architecture — Jupyter Documentation 4.1.1 Alpha Documentation. <https://docs.jupyter.org/en/latest/projects/architecture/content-architecture.html> [retrieval: 8.4.2024].
- **(2024b):** Jupyter Kernels. <https://github.com/jupyter/jupyter/wiki/Jupyter-kernels> [retrieval: 8.4.2024].
 - **(2024c):** Project Jupyter Documentation — Jupyter Documentation 4.1.1 Alpha Documentation. <https://docs.jupyter.org/en/latest/index.html#project-jupyter-documentation> [retrieval: 8.4.2024].
- Jupyterhub - Docs (2024):** JupyterHub. JupyterHub. <https://jupyterhub.readthedocs.io/en/latest/index.html> [retrieval: 10.4.2024].
- Keeton, K./Singhal, S./Raymond, M. (2019):** The OpenFAM API: A Programming Model for Disaggregated Persistent Memory. In: *OpenSHMEM and Related Technologies. OpenSHMEM in the Era of Extreme Heterogeneity*. Ed. by Swaroop Pophale/Neena Imam/Ferrol Aderholdt/Manjunath Gorentla Venkata. Cham: Springer International Publishing, pp. 70–89. ISBN: 978-3-030-04918-8. 10.1007/978-3-030-04918-8_5.
- Kenny, J./Knight, S. (2021):** Kubernetes for HPC Administration. SAND2021-11507C. Sandia National Lab. (SNL-NM), Albuquerque, NM (United States); Sandia National Laboratories, SNL California. <https://www.osti.gov/biblio/1887730> [retrieval: 29.6.2023].

- Kindratenko, V./Trancoso, P. (2011):** Trends in High-Performance Computing. In: *Comput. Sci. Eng.* 13.3, pp. 92–95. ISSN: 1521-9615. 10.1109/MCSE.2011.52. <http://ieeexplore.ieee.org/document/5756280/> [retrieval: 25.3.2024].
- Kovacs, A. (2017):** Comparison of Different Linux Containers. In: *2017 40th International Conference on Telecommunications and Signal Processing (TSP)*. 2017 40th International Conference on Telecommunications and Signal Processing (TSP). Barcelona, Spain: IEEE, pp. 47–51. ISBN: 978-1-5090-3982-1. 10.1109/TSP.2017.8075934. <http://ieeexplore.ieee.org/document/8075934/> [retrieval: 13.2.2024].
- Kubernetes - Docs (2024a):** *Kubernetes-Retired/Kube-Batch*. Kubernetes Retired. <https://github.com/kubernetes-retired/kube-batch> [retrieval: 20.3.2024].
- **(2024b):** Assigning Pods to Nodes. <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/> [retrieval: 20.3.2024].
 - **(2024c):** Cloud Controller Manager. <https://kubernetes.io/docs/concepts/architecture/cloud-controller/> [retrieval: 18.2.2024].
 - **(2024d):** Cluster Networking. <https://kubernetes.io/docs/concepts/cluster-administration/networking/> [retrieval: 5.5.2024].
 - **(2024e):** Controllers. <https://kubernetes.io/docs/concepts/architecture/controller/> [retrieval: 18.2.2024].
 - **(2024f):** Kube-Proxy. <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/> [retrieval: 18.2.2024].
 - **(2024g):** Kubelet. <https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/> [retrieval: 18.2.2024].
 - **(2024h):** Kubernetes Components. <https://kubernetes.io/docs/concepts/overview/components/> [retrieval: 18.2.2024].
 - **(2024i):** Kubernetes Scheduler. <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/> [retrieval: 18.2.2024].
 - **(2024j):** Nodes. <https://kubernetes.io/docs/concepts/architecture/nodes/> [retrieval: 18.2.2024].
 - **(2024k):** Pod Lifecycle. <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/> [retrieval: 21.4.2024].
 - **(2024l):** Scheduling Framework. <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/> [retrieval: 20.3.2024].
- Kubernetes 1.26 Release Team (2022):** Kubernetes v1.26: Electrifying. <https://kubernetes.io/blog/2022/12/09/kubernetes-v1-26-release/> [retrieval: 23.4.2024].
- Kumaran S., S. (2017):** Practical LXC and LXD. Berkeley, CA: Apress. ISBN: 978-1-4842-3023-7 978-1-4842-3024-4. 10.1007/978-1-4842-3024-4. <http://link.springer.com/10.1007/978-1-4842-3024-4> [retrieval: 17.2.2024].
- Laboratory L. L. N., B. B. (2024):** OpenMP Programming Model. LLNL HPC Tutorials. https://hpc-tutorials.llnl.gov/openmp/programming_model/ [retrieval: 4.4.2024].

- Laurent Bernaille (2018)**: ‘KubeCon + CloudNativeCon North America 2018: Lightning Talk: Optimizing Kubernetes Ne...’ Cloud Native Conference (Seattle, WA). <https://kccna18.sched.com/event/GrVD> [retrieval: 20.2.2024].
- Li, S./Hoefer, T./Snir, M. (w.y.)**: NUMA-aware Shared-Memory Collective Communication for MPI. In.
- Linux - manual page (2024)**: *Chroot(2) - Linux Manual Page*. <https://man7.org/linux/man-pages/man2/chroot.2.html> [retrieval: 14.2.2024].
- Linux Kernel - Docs (2024)**: *Memory Resource Controller — The Linux Kernel Documentation*. <https://docs.kernel.org/admin-guide/cgroup-v1/memory.html> [retrieval: 20.3.2024].
- Martin, J. (1991)**: Rapid Application Development. New York : Macmillan Pub. Co. ; Toronto : Collier Macmillan Canada ; New York : Maxwell Macmillan International. 826 pp. ISBN: 978-0-02-376775-3. <http://archive.org/details/rapidapplication00mart> [retrieval: 3.11.2023].
- Martínez, J. (2022)**: Understanding Kubernetes Evicted Pods. Sysdig. <https://sysdig.com/blog/kubernetes-pod-evicted/> [retrieval: 20.3.2024].
- Matias Nicoletti (2023)**: Understanding Kubernetes Pod Quality of Service (QoS) Classes. Linkedin. <https://www.linkedin.com/pulse/understanding-kubernetes-pod-quality-service-qos-matias-nicoletti> [retrieval: 20.3.2024].
- McGrath, G./Brenner, P. R. (2017)**: Serverless Computing: Design, Implementation, and Performance. In: *2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW)*. 2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW), pp. 405–410. 10.1109/ICDCSW.2017.36. <https://ieeexplore.ieee.org/abstract/document/7979855> [retrieval: 12.4.2024].
- Michael Kerrisk (2013)**: Namespaces in Operation, Part 1: Namespaces Overview [LWN.Net]. In: *Linux Weekly News*. <https://lwn.net/Articles/531114/> [retrieval: 14.2.2024].
- Misale, C./Milroy, D. J./Gutierrez, C. E. A./Drocco, M./Herbein, S./Ahn, D. H./Kaiser, Z./Park, Y. (2022)**: Towards Standard Kubernetes Scheduling Interfaces for Converged Computing. In: *Driving Scientific and Engineering Discoveries Through the Integration of Experiment, Big Data, and Modeling and Simulation*. Ed. by Jeffrey Nichols/Arthur ‘Barney’ Maccabe/James Nutaro/Swaroop Pophale/Pravallika Devineni/Theresa Ahearn/Becky Verastegui. Cham: Springer International Publishing, pp. 310–326. ISBN: 978-3-030-96498-6. 10.1007/978-3-030-96498-6_18.
- MIT Plasma Science and Fusion Center, director (2022)**: *Intro to Parallel Programming with OpenMP (Part 1)*. <https://www.youtube.com/watch?v=f1aVTx5g43o> [retrieval: 4.4.2024].
- Morabito, R./Kjällman, J./Komu, M. (2015)**: Hypervisors vs. Lightweight Virtualization: A Performance Comparison. In: *2015 IEEE International Conference on Cloud Engineering*. 2015 IEEE International Conference on Cloud Engineering, pp. 386–393. 10.1109/IC2E.2015.74. <https://ieeexplore.ieee.org/abstract/document/7092949> [retrieval: 17.2.2024].
- Mujkanovic, N./Durillo, J. J./Hammer, N./Müller, T. (2023)**: Survey of Adaptive Containerization Architectures for HPC. In: *Proceedings of the SC ’23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*.

- SC-W '23. New York, NY, USA: Association for Computing Machinery, pp. 165–176. ISBN: 9798400707858. 10.1145/3624062.3624588. <https://dl.acm.org/doi/10.1145/3624062.3624588> [retrieval: 17.2.2024].
- Multus (2024a)**: *K8snetworkplumbingwg/Multus-Cni*. k8snetworkplumbingwg. <https://github.com/k8snetworkplumbingwg/multus-cni> [retrieval: 1.5.2024].
- **(2024b)**: GitHub - K8snetworkplumbingwg/Multus-Cni: A CNI Meta-Plugin for Multi-Homed Pods in Kubernetes. <https://github.com/k8snetworkplumbingwg/multus-cni> [retrieval: 16.4.2024].
- Murphy, R. C./Wheeler, K. B./Barrett, B. W./Ang, J. A. (2010)**: Introducing the Graph 500. In: *Cray Users Group (CUG)* 19.45-74, p. 22. <http://www.richardmurphy.net/archive/cug-may2010.pdf> [retrieval: 21.3.2024].
- Nahren Khizeran (2024)**: Hewlett Packard Enterprise Acquires Pachyderm to Expand AI-at-scale Capabilities with Reproducible AI. <https://www.hpe.com/us/en/newsroom/press-release/2023/01/hewlett-packard-enterprise-acquires-pachyderm-to-expand-ai-at-scale-capabilities-with-reproducible-ai.html> [retrieval: 25.3.2024].
- Nalawala, H. S./Shah, J./Agrawal, S./Oza, P. (2022)**: A Comprehensive Study of “EtcD”—An Open-Source Distributed Key-Value Store with Relevant Distributed Databases. In: *Emerging Technologies for Computing, Communication and Smart Cities*. Ed. by Pradeep Kumar Singh/Maheshkumar H. Kolekar/Sudeep Tanwar/Sławomir T. Wierchoń/Raj K. Bhatnagar. Lecture Notes in Electrical Engineering. Singapore: Springer Nature, pp. 481–489. ISBN: 978-981-19028-4-0. 10.1007/978-981-19-0284-0_35.
- Naumann, J. D./Jenkins, A. M. (1982)**: Prototyping: The New Paradigm for Systems Development. In: *MIS Quarterly* 6.3, pp. 29–44. ISSN: 0276-7783. 10.2307/248654. JSTOR: 248654. <https://www.jstor.org/stable/248654> [retrieval: 3.11.2023].
- Niklas Wittmer (2018)**: Parallelisierung Mit OpenMP. Hamburg: University of Hamburg. https://wr.informatik.uni-hamburg.de/_media/teaching/wintersemester_2017_2018/ep-1718-wittmer-openmp-ausarbeitung.pdf.
- NVIDIA (2024)**: Network Operator. NVIDIA Docs. <https://docs.nvidia.com/networking/display/COKAN10/Network+Operator> [retrieval: 16.4.2024].
- Open Container Initiative (2024)**: *Image-Spec/Spec.Md at Main · Opencontainers/Image-Spec*. Version v1.1.0. <https://github.com/opencontainers/image-spec/releases/download/v1.1.0/oci-image-spec-v1.1.0.pdf> [retrieval: 26.4.2024].
- Open MPI, director (2014)**: *What Is Open MPI*. <https://www.youtube.com/watch?v=D0-xSWBGNAw> [retrieval: 6.4.2024].
- Perkel, J. M. (2018)**: Why Jupyter Is Data Scientists’ Computational Notebook of Choice. In: *Nature* 563.7729, pp. 145–146. 10.1038/d41586-018-07196-1. <https://www.nature.com/articles/d41586-018-07196-1> [retrieval: 8.4.2024].
- Pfister, G. F. (w.y.a)**: An Introduction to the InfiniBand™ Architecture. In.
- **(w.y.b)**: An Introduction to the InfiniBand™ Architecture. In.

- Ploskas, N./Samaras, N. (2016):** ‘Introduction’. In: *GPU Programming in MATLAB*. Elsevier, pp. 1–19. ISBN: 978-0-12-805132-0. 10.1016/B978-0-12-805132-0.00001-1. <https://linkinghub.elsevier.com/retrieve/pii/B9780128051320000011> [retrieval: 27.3.2024].
- Priedhorsky, R./Randles, T. (2017):** Charliecloud: Unprivileged Containers for User-Defined Software Stacks in HPC. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '17: The International Conference for High Performance Computing, Networking, Storage and Analysis. Denver Colorado: ACM, pp. 1–10. ISBN: 978-1-4503-5114-0. 10.1145/3126908.3126925. <https://dl.acm.org/doi/10.1145/3126908.3126925> [retrieval: 13.2.2024].
- Prof. Dr. Ben H. Juurlink, director (2018):** *Lecture on Cache Coherence*. <https://www.youtube.com/watch?v=BNH-3TwmVN0> [retrieval: 28.3.2024].
- Rabenseifner, R./Hager, G./Jost, G. (2009):** Hybrid MPI/OpenMP Parallel Programming on Clusters of Multi-Core SMP Nodes. In: *2009 17th Euromicro International Conference on Parallel, Distributed and Network-based Processing*. 2009 17th Euromicro International Conference on Parallel, Distributed and Network-based Processing. Weimar: IEEE, pp. 427–436. ISBN: 978-0-7695-3544-9. 10.1109/PDP.2009.43. <https://ieeexplore.ieee.org/document/4912964/> [retrieval: 31.3.2024].
- Rancher (2024):** Innovate Everywhere. Rancher Labs. <http://www.rancher.com> [retrieval: 12.4.2024].
- Randal, A. (2021):** The Ideal Versus the Real: Revisiting the History of Virtual Machines and Containers. In: *ACM Comput. Surv.* 53.1, pp. 1–31. ISSN: 0360-0300, 1557-7341. 10.1145/3365199. <https://dl.acm.org/doi/10.1145/3365199> [retrieval: 13.2.2024].
- RDMA Consortium (2003):** RDMA Consortium FAQs. http://rdmaconsortium.org/home/FAQs_Apr25.htm [retrieval: 1.4.2024].
- RedHat (2024a):** Making High Performance Networking Applications Work on Hybrid Clouds. <https://www.redhat.com/en/blog/making-high-performance-networking-applications-work-hybrid-clouds> [retrieval: 1.5.2024].
- **(2024b):** Red Hat OpenShift Container Platform. <https://www.redhat.com/en/technologies/cloud-computing/openshift/container-platform> [retrieval: 12.4.2024].
- RedHat - Openshift (2024):** About Single Root I/O Virtualization (SR-IOV) Hardware Networks - Hardware Networks | Networking | OpenShift Container Platform 4.15. https://docs.openshift.com/container-platform/4.15/networking/hardware_networks/about-sriov.html [retrieval: 3.5.2024].
- Rejiba, Z./Chamanara, J. (2023):** Custom Scheduling in Kubernetes: A Survey on Common Problems and Solution Approaches. In: *ACM Comput. Surv.* 55.7, pp. 1–37. ISSN: 0360-0300, 1557-7341. 10.1145/3544788. <https://dl.acm.org/doi/10.1145/3544788> [retrieval: 20.3.2024].
- Rohit Gupta (2024):** Linux: The Operating System Dominating the World’s Fastest Supercomputers. <https://www.linkedin.com/pulse/discover-power-speed-linux-operating-system-dominating-rohit-gupta> [retrieval: 2.4.2024].
- Ronaghan, E. (2020):** Squeezing Performance out of Arkouda. In: *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2020 IEEE Interna-

- tional Parallel and Distributed Processing Symposium Workshops (IPDPSW). New Orleans, LA, USA: IEEE, pp. 677–677. ISBN: 978-1-72817-445-7. 10.1109/IPDPSW50202.2020.00119. <https://ieeexplore.ieee.org/document/9150410/> [retrieval: 20.4.2024].
- Ruud Van Der Pas (w.y.):** ‘Speed Up Your OpenMP Code Without Doing Much’. PPCES (Santa Clara).
- Ruud van der Pas, director (2014):** *Getting OpenMP Up To Speed*. <https://www.youtube.com/watch?v=u6N33FhfgZU> [retrieval: 4.4.2024].
- Schadt, E. E./Linderman, M. D./Sorenson, J./Lee, L./Nolan, G. P. (2010):** Computational Solutions to Large-Scale Data Management and Analysis. In: *Nature reviews genetics* 11.9, pp. 647–657. <https://www.nature.com/articles/nrg2857> [retrieval: 21.3.2024].
- SCHEDMD (2024a):** Slurm Workload Manager - Overview. <https://slurm.schedmd.com/overview.html> [retrieval: 7.4.2024].
- **(2024b):** Slurm Workload Manager - Scheduling Configuration Guide. https://slurm.schedmd.com/sched_config.html [retrieval: 7.4.2024].
 - **(2024c):** Slurm Workload Manager - Topology Guide. <https://slurm.schedmd.com/topology.html> [retrieval: 7.4.2024].
- Schreiner, M./Hess, T./Benlian, A. (w.y.):** Gestaltungsorientierter Kern oder Tendenz zur Empirie? Zur neueren methodischen Entwicklung der Wirtschaftsinformatik. In.
- Singularity - Docs (2024a):** Fakeroot Feature — Singularity Container 3.5 Documentation. <https://docs.sylabs.io/guides/3.5/user-guide/fakeroot.html> [retrieval: 12.4.2024].
- **(2024b):** Integrating with Kubernetes — Singularity CRI User Documentation 1.0.0 Documentation. <https://docs.sylabs.io/guides/cri/1.0/user-guide/k8s.html> [retrieval: 23.4.2024].
 - **(2024c):** Introduction to Singularity — Singularity Container 3.5 Documentation. <https://docs.sylabs.io/guides/3.5/user-guide/introduction.html#why-use-singularity> [retrieval: 12.4.2024].
- Singularity - Repo (2024):** Sylabs/Singularity-Cri: The Singularity Implementation of the Kubernetes Container Runtime Interface. <https://github.com/sylabs/singularity-cri> [retrieval: 23.4.2024].
- Sookocheff, K. (2018):** A Guide to the Kubernetes Networking Model. <https://sookocheff.com/post/kubernetes/understanding-kubernetes-networking-model/> [retrieval: 20.2.2024].
- IB-SRIOV-CNI - Repo (2024):** *K8snetworkplumbingwg/Ib-Sriov-Cni: InfiniBand SR-IOV CNI*. <https://github.com/k8snetworkplumbingwg/ib-sriov-cni> [retrieval: 2.5.2024].
- Sriov-Network-Operator - Repo (2024):** *Sriov-Network-Operator/Bindata at Master*. <https://github.com/k8snetworkplumbingwg/sriov-network-operator/blob/c2d9e325ce8c2cab22f31eebe41742dae2398712/bindata/manifests/operator-webhook/003-webhook.yaml#L10> [retrieval: 3.5.2024].
- Sterling, T./Brodowicz, M./Anderson, M. (2017):** High Performance Computing: Modern Systems and Practices. Morgan Kaufmann. 720 pp. ISBN: 978-0-12-420215-3. Google Books: qQHIBAAAQBAJ.

- Steven Vaughan-Nichols (2024)**: Can the Internet Exist without Linux? ZDNET. <https://www.zdnet.com/home-and-office/networking/can-the-internet-exist-without-linux/> [retrieval: 12.4.2024].
- Sultan, S./Ahmad, I./Dimitriou, T. (2019)**: Container Security: Issues, Challenges, and the Road Ahead. In: *IEEE Access* 7, pp. 52976–52996. ISSN: 2169-3536. 10.1109/ACCESS.2019.2911732. <https://ieeexplore.ieee.org/document/8693491/> [retrieval: 16.2.2024].
- Sun, Y./Safford, D./Zohar, M./Pendarakis, D./Gu, Z./Jaeger, T. (2018)**: Security Namespace: Making Linux Security Frameworks Available to Containers. In: *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Security Symposium. Baltimore, MD: USENIX Association, pp. 1423–1439. ISBN: 978-1-939133-04-5. <https://www.usenix.org/conference/usenixsecurity18/presentation/sun>.
- The Linux Foundation (2024)**: CNI. <https://www.cni.dev/docs/spec/> [retrieval: 20.2.2024].
- Tigera (2024)**: Overlay Networking | Calico Documentation. <https://docs.tigera.io/calico/latest/networking/configuring/vxlan-ipip> [retrieval: 16.4.2024].
- TOP500 (2024a)**: List Statistics Architecture 1995 | TOP500. <https://www.top500.org/statistics/list/> [retrieval: 25.3.2024].
- **(2024b)**: List Statistics Architecture 2023 | TOP500. <https://www.top500.org/statistics/list/> [retrieval: 25.3.2024].
 - **(2024c)**: List Statistics Interconnect November 2023. <https://top500.org/statistics/list/> [retrieval: 1.4.2024].
 - **(2024d)**: Operating Systems Development over Time. <https://www.top500.org/statistics/over-time/> [retrieval: 2.4.2024].
- Truyen, E./Van Landuyt, D./Preuveneers, D./Lagaisse, B./Joosen, W. (2019)**: A Comprehensive Feature Comparison Study of Open-Source Container Orchestration Frameworks. In: *Applied Sciences* 9.5 (5), p. 931. ISSN: 2076-3417. 10.3390/app9050931. <https://www.mdpi.com/2076-3417/9/5/931> [retrieval: 18.2.2024].
- Uni.lu (2024)**: UL HPC Tutorial: Singularity with Infiniband. <https://ulhpc-tutorials.readthedocs.io/en/latest/containers/singularity-inf/> [retrieval: 12.4.2024].
- Vailshery, L. S. (2024)**: Leading Containerization Technologies 2023. Statista. <https://www.statista.com/statistics/1256245/containerization-technologies-software-market-share/> [retrieval: 17.2.2024].
- Velayudhan, N. (2024)**: A Visual Guide to Kubernetes Networking Fundamentals | Open-source.Com. <https://opensource.com/article/22/6/kubernetes-networking-fundamentals> [retrieval: 29.2.2024].
- Verma, A./Pedrosa, L./Korupolu, M./Oppenheimer, D./Tune, E./Wilkes, J. (2015)**: Large-Scale Cluster Management at Google with Borg. In: *Proceedings of the Tenth European Conference on Computer Systems*. EuroSys '15. New York, NY, USA: Association for Computing Machinery, pp. 1–17. ISBN: 978-1-4503-3238-5. 10.1145/2741948.2741964. <https://dl.acm.org/doi/10.1145/2741948.2741964> [retrieval: 17.2.2024].
- Vishu Vaishnav (2023)**: Message Passing in Distributed System. GeeksforGeeks. <https://www.geeksforgeeks.org/message-passing-in-distributed-system/> [retrieval: 30.3.2024].

- VMware (2020):** VMware Tanzu Kubernetes Grid - Enterprise Kubernetes Container Runtime. <https://tanzu.vmware.com/kubernetes-grid> [retrieval: 12.4.2024].
- Wilde, T./Hess, T. (w.y.):** Methodenspektrum der Wirtschaftsinformatik: Überblick und Portfoliobildung. In.
- Yoo, A. B./Jette, M. A./Grondona, M. (2003):** SLURM: Simple Linux Utility for Resource Management. In: *Job Scheduling Strategies for Parallel Processing*. Ed. by Dror Feitelson/Larry Rudolph/Uwe Schwiegelshohn. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, pp. 44–60. ISBN: 978-3-540-39727-4. 10.1007/10968987_3.
- Zempashi (2024):** Zempashi/Kube-Training. <https://github.com/Zempashi/kube-training> [retrieval: 18.2.2024].
- Zivuku, S. (2023):** In-Depth Analysis of Kubernetes Scheduling. Medium. <https://zivukushingai.medium.com/in-depth-analysis-of-kubernetes-scheduling-abf08f949924> [retrieval: 19.3.2024].